

Gerenciamento de Processos

Comunicação e Sincronização do Compartilhamento



Roteiro:

- Concorrência e Condições de Corrida
- Mecanismos de Sincronização:
 - Semáforo, Mutex e Variável de Condição
- Monitor
- Problemas Clássicos de Concorrência
- Envio de Mensagem
- Inter Process Communication em Unix:
 - Pipe, FIFO, Message Queue, Sockets

Comunicação Inter-Processos (IPC)

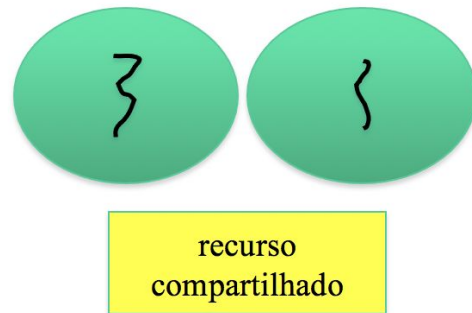
- Existem inúmeras situações em que processos do sistema (ou do usuário) precisam interagir para se comunicar ou sincronizar as suas ações, por exemplo, no acesso compartilhado a dados ou recursos.

Comunicação Inter-Processos envolve:

- sincronização de atividades
- troca de dados

Comunicação Inter-Processos(IPC)

- Processos (e threads) são entidades independentes que podem ser executados em qualquer ordem e interrompidos a qualquer momento
- A ordem de escalonamento é imprevisível
- Precisa-se de mecanismos para evitar problemas de inconsistência de dados compartilhados decorrentes da execução concorrente.



IPC entre processos

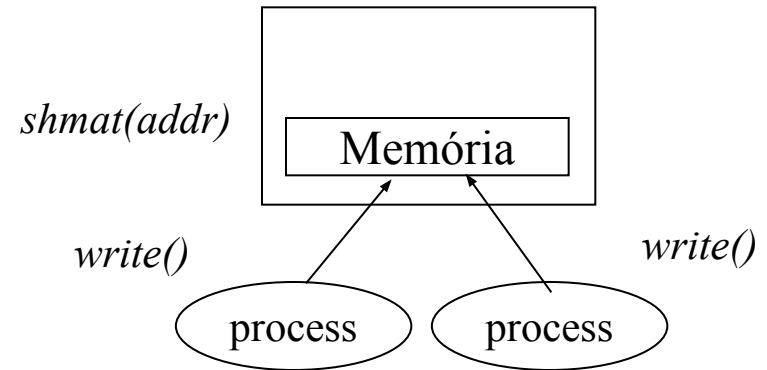


IPC entre threads

Comunicação e Sincronização entre Processos: duas Abordagens

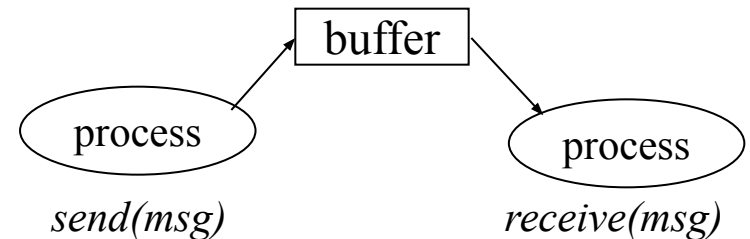
1. Baseada em memória compartilhada

- Comunicação é **implícita** (por dados compartilhados, sem canal de comunicação) mas
- Sincronização para acesso precisa ser feita **explicitamente**



2. Baseada em troca de mensagens

- Canal de comunicação é **explícito**;
- Sincronização é **implícita** (processos bloqueiam nas primitivas)



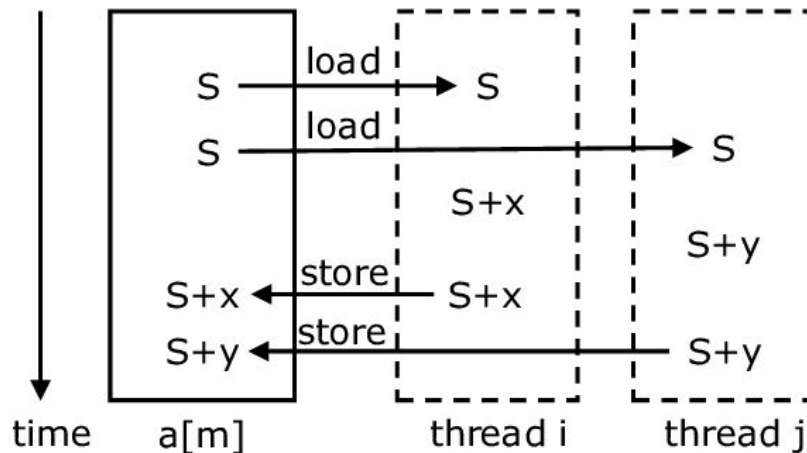
Processos concorrentes que compartilham um recurso sofrem do problema de **condição de corrida**.

Mecanismos de Comunicação e Sincronização em UNIX

- Shared Memory – compartilhamento sem sincronização
- Signals – notificações assíncronas p/ controle e sincronização
- Pipes – para comunicação fifo entre processo pai e filhos
- FIFO (Named pipes) – para comunicação fifo entre qq processos, e com persistência de dados
- Message Queues – para comunicação fifo, com mensagens rotuladas com um tipo
- Semaphore e Mutex – p/ sincronização
- Sockets – para comunicação fifo cliente-servidor

Condição de Corrida

- Ocorre quando mais de um processo/ thread acessam uma memória compartilhada
- É a situação em que dois processos lêem e escrevem um dado compartilhado e o resultado final da manipulação depende da ordem em que os processos/thread são executados.



Thread i:

`S = S+1;`

`LOAD S, %eax`

`ADD X, %eax`

`STO %eax, S`

Depurar programas que contém condições de corrida é muito difícil, pois não é possível prever em que sequência as instruções de cada thread serão executadas.

O problema: Condição de Corrida

Problema decorrente do **indeterminismo na ordem de acesso concorrente** ao **recurso/dados compartilhado**.

o escalonamento e a troca de contexto a qualquer momento geram indeterminismo na intercalação das instruções dos processos/threads concorrentes

Exemplo: Instruções de atribuição em linguagem de alto nível são traduzidos para instruções de máquina mais elementares.

Digamos que x começa com 2.

Processo A:

$x = x + 1;$

LOAD $x, \%eax$

ADD 1, $\%eax$

STO $\%eax, x$

ProcessoB:

$x = x - 1;$

LOAD $x, \%ebx$

SUB 1, $\%ebx$

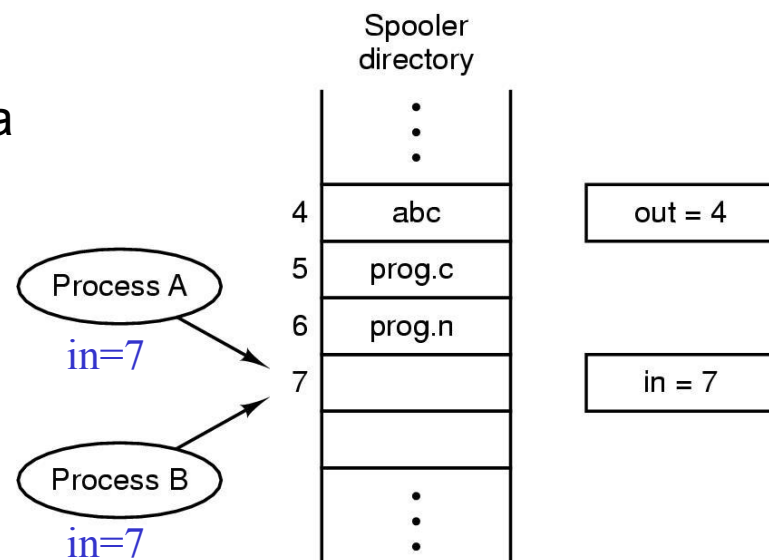
STO $\%ebx, x$

- Digamos que A é interrompido após ADD; B executa até final e A retoma execução.
- Problema: o valor de x carregado por A em $\%eax$ não refletirá o novo valor de x atribuído pelo B, e quando A retomar, irá sobreescrever o valor $x=3$ em cima do valor produzido por B.

Condição de Corrida

Exemplo: Dois processos concorrentes querem adicionar um job de impressão a fila de *spooler*:

- processo A lê memória compartilhada “in=7”, e logo depois é interrompido,
- Processo B faz o mesmo e inclui seu arquivo na posição 7 do spool de impressão
- Quando A retoma execução, sobrescreve a posição 7 com seu arquivo.



```
#include <stdlib.h>
#include <stdio.h>
#include <pthread.h>
```

```
int mails = 0;
```

```
void* routine() {
    for (int i = 0; i < 100000; i++) {
        mails++;
        // read mails
        // increment
        // write mails
    }
}
```

```
int main(int argc, char* argv[]) {
    pthread_t p1, p2, p3, p4;
    if (pthread_create(&p1, NULL, &routine, NULL)
    != 0) {
        return 1;
    }
    if (pthread_create(&p2, NULL, &routine, NULL)
    != 0) {
        return 2;
    }
    if (pthread_join(p1, NULL) != 0) {
        return 3;
    }
    if (pthread_join(p2, NULL) != 0) {
        return 4;
    }
    printf("Number of mails: %d\n", mails);
    return 0;
}
```

Condição de Corrida

4 threads acabam incrementando a variável mails em menos do que 40.0000 por que o trio é interrompido.

```
LOAD x,%eax
ADD 1,%eax
STO %eax,x
```

Condição de Corrida

Exemplo: atualização de conta bancária conjunta
(se o valor do saldo e a operação forem armazenados/executados em
cada processo)

1º. Titular da conta:

Consulta saldo
R\$ 100,00

Faz retirada
R\$ 50,00

Consulta saldo
R\$ -30,00 ???

2º. Titular da conta:

Consulta saldo
R\$ 100,00

Faz retirada
R\$ 80,00

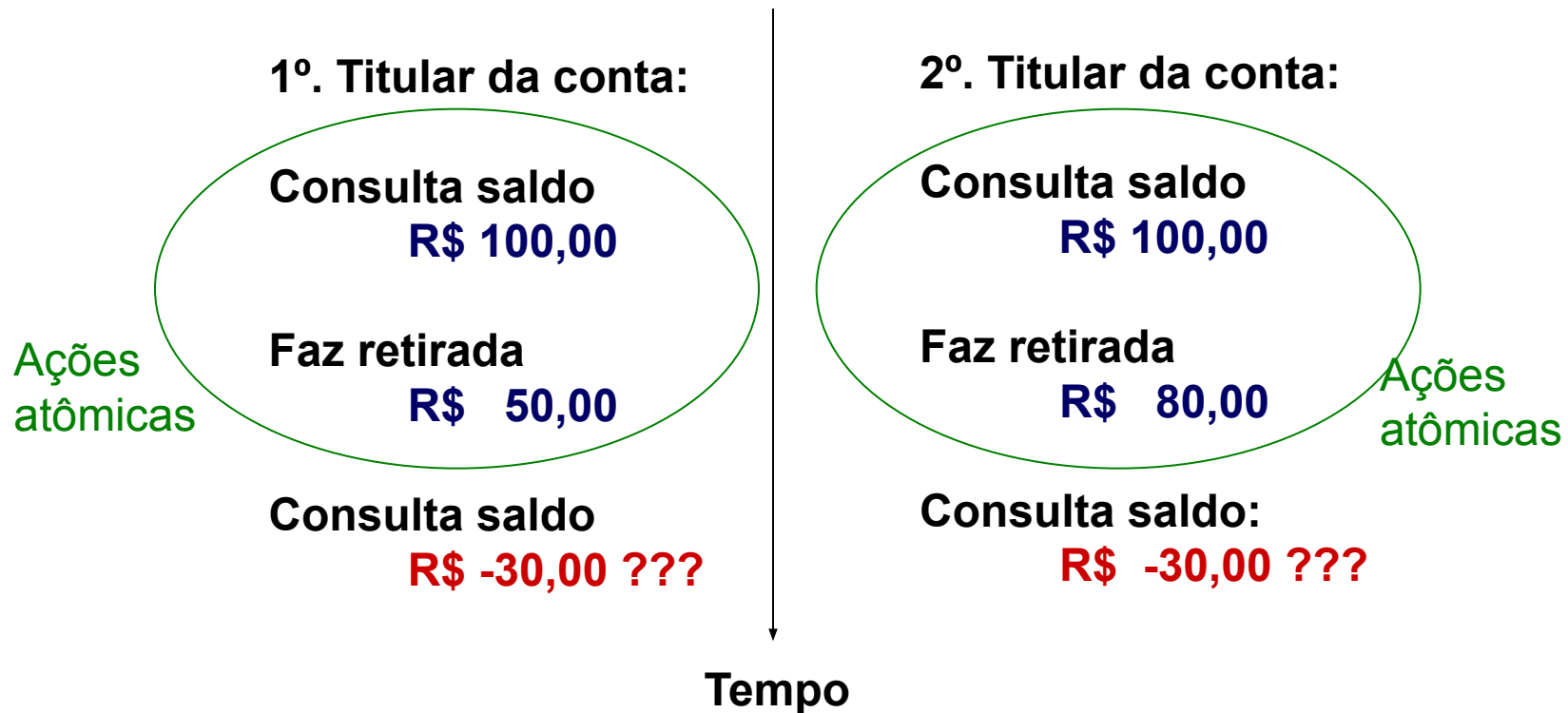
Consulta saldo:
R\$ -30,00 ???

Tempo

Explique o que aconteceu e indique como evitar que a conta fique negativa

Condição de Corrida

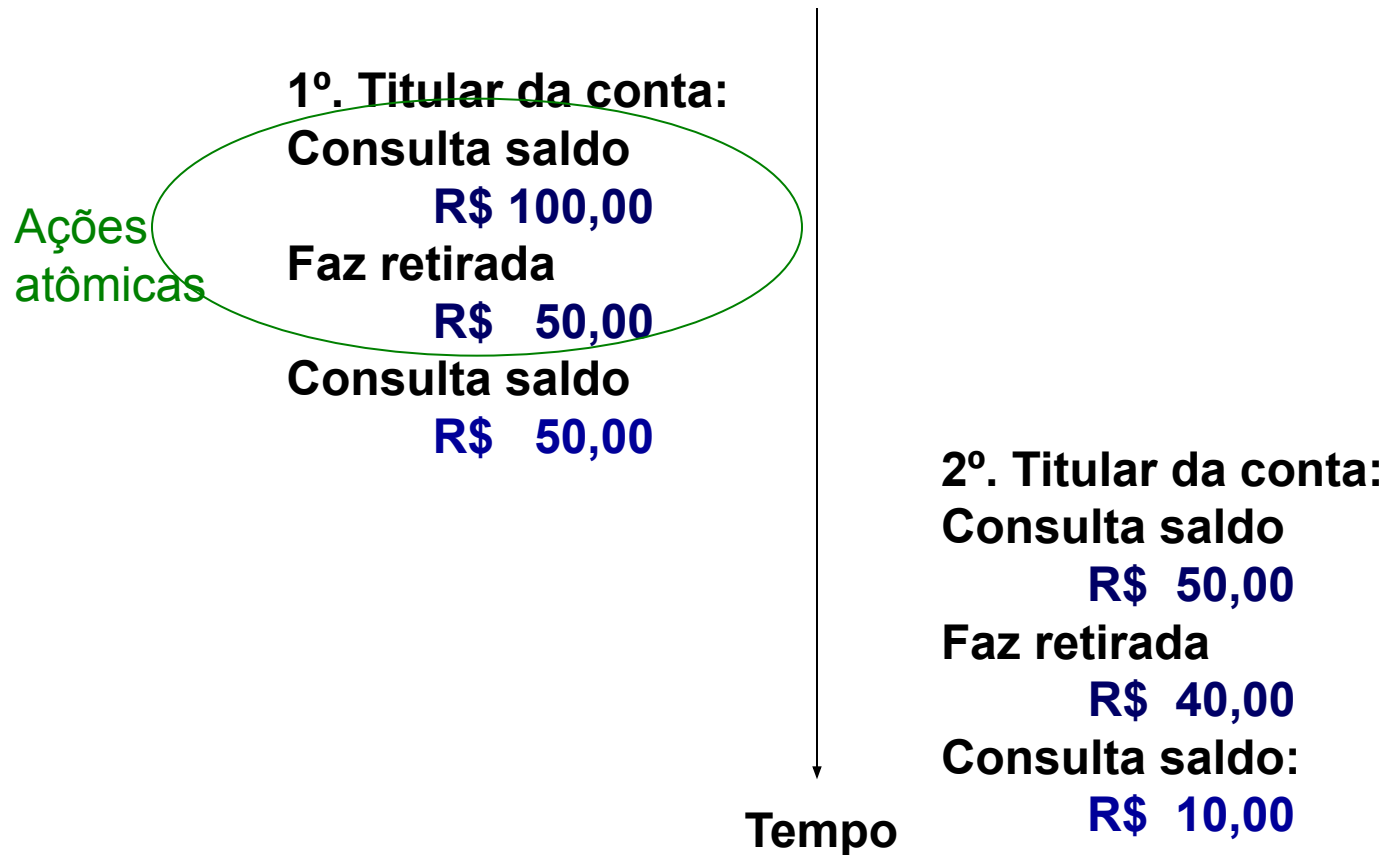
Exemplo: atualização de conta bancária conjunta



Explique o que aconteceu e indique como evitar que a conta fique negativa

Condição de Corrida

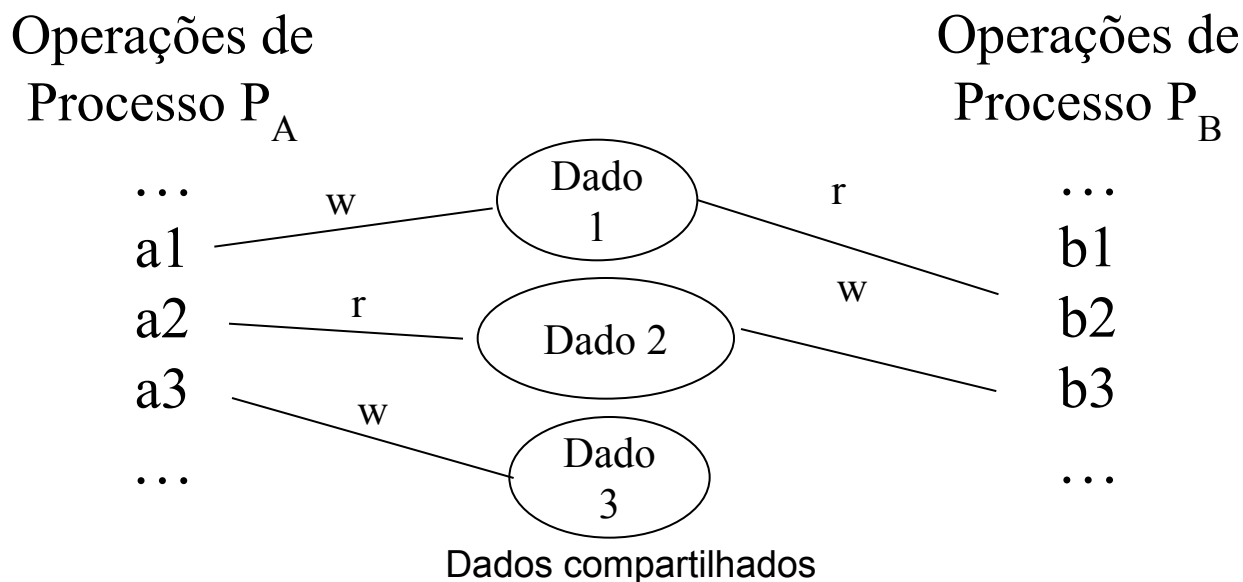
Exemplo: atualização de conta bancária conjunta



Condição de Corrida

Ocorre sempre que...

- Cada processo/thread precisa executar uma sequência de ações ($a_1, \dots, a_N$) que envolvem mais de um dado/recurso compartilhado, e
- os dados/recursos precisam manter um estado consistente entre si;
- e quando a execução de uma sequência de operações, um dos processos é interrompido pelo outro

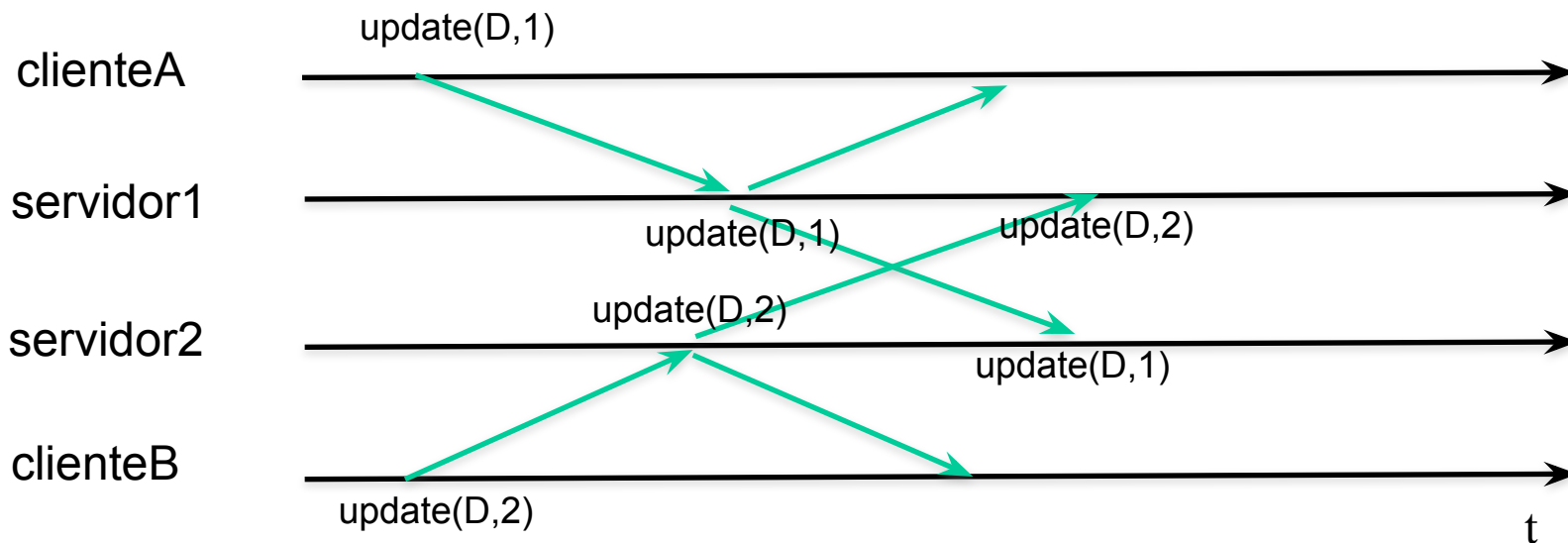


Condição de Corrida

Também vale para troca de mensagens (entre processos clientes e processos servidores)

Exemplo:

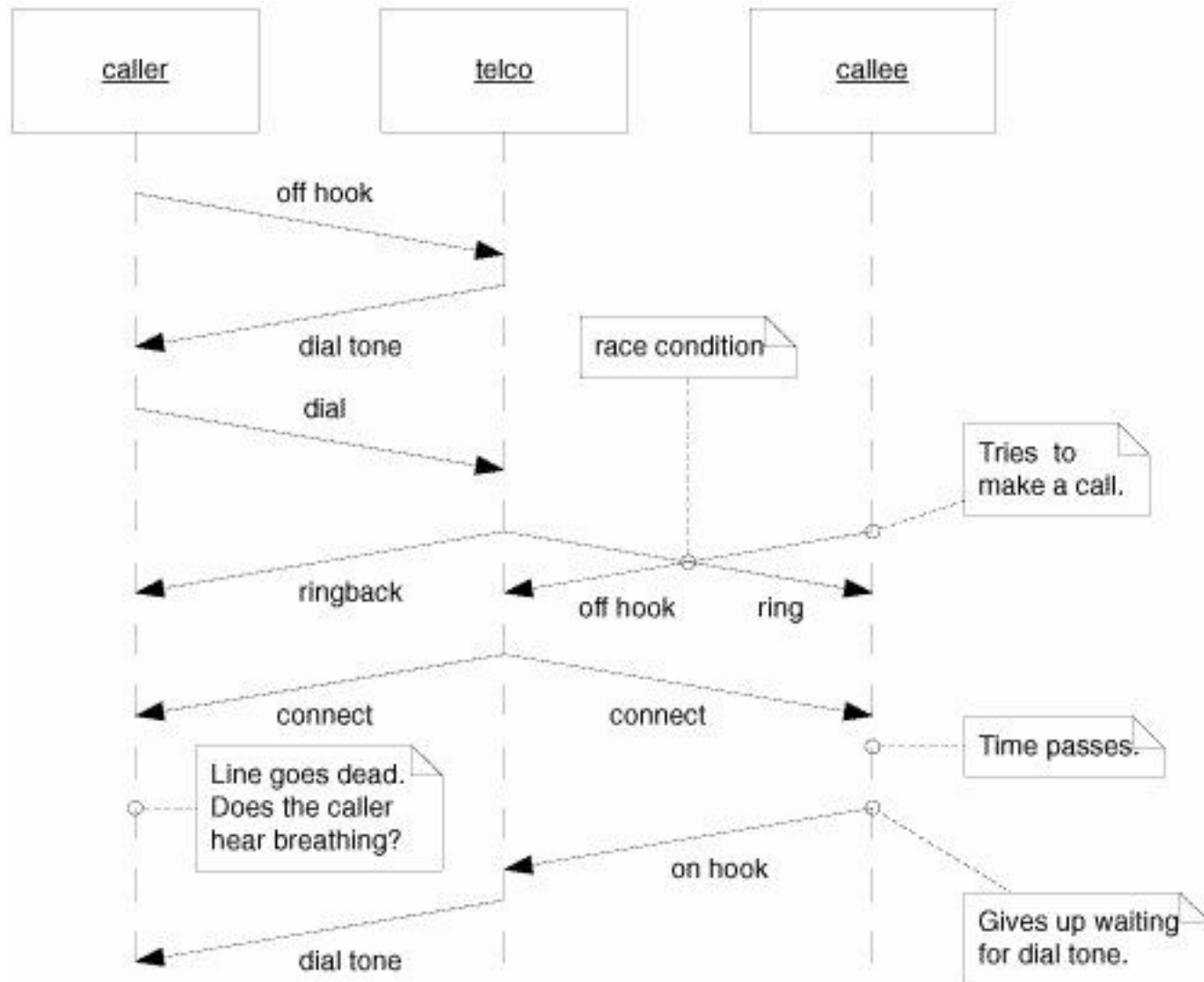
1. Suponha um serviço tolerante a falhas que seja implementado com replicação (distribuída) de dados.
2. Assim que recebe uma requisição, um servidor responde ao cliente e envia a atualização seu dados para o outro servidor, que atualiza seu dado correspondentemente



Os servidores precisam entrar em um consenso sobre a ordem final das operações.

Condição de Corrida

exemplo do dia-a-dia: ligação telefônica



Condição de Corrida

Para evitar os problemas de condição de corrida os processos/threads que compartilham algum dado/recurso:

1. precisam “bloquear” temporariamente o recurso, impedindo que os outros processos interfiram no seu acesso ao recurso. (ou seja, garantir a **Exclusão mútua**)
2. precisam ser capazes de “notificar” os demais processos quando o recurso foi “liberado” ou quando está no estado que permita uma determinada operação

por exemplo, quando um buffer não está cheio e permite a escrita de mais um elemento)

Esses “bloquear”, “liberar” e “notificar” precisam ser atômico.

Condição de Corrida

Para permitir uma cooperação correta de processos sem condição de corrida dados compartilhados (evitando condições de corrida) é preciso:

- Fazer um processo esperar pelo outro (sincronização entre processos)
- Garantir que algumas sequências de operações ocorram de forma atômica (indivisível)

Região Crítica

*Região crítica (ou Sessão crítica) é uma parte do código (programa) que contém as operações sobre dados compartilhados, a serem executadas em **exclusão mútua**.*

Para implementar regiões críticas **alguns requisitos básicos** precisam ser satisfeitos:

Exclusão Mútua: Se um processo P está executando sua região crítica nenhum outro poderá executar a sua região crítica (**segurança = safety**)

Progresso: Um processo executando dentro da região crítica não pode ser bloqueado/terminado por outro processo (**progresso = liveness**).

Espera Limitada: Um processo não deve esperar indefinidamente para entrar em sua região crítica (**justiça**)

Região Crítica

Para implementar uma região crítica deve haver um mecanismo/protocolo para garantir a entrada e saída segura (sincronizada, coordenada) nesta parte do código.

Código em um processo:

```
...  
Enter-region; // bloqueia se outro processo estiver dentro  
    instr 1;  
    instr 2;  
    instr 3;  
    ...  
Exit-region; // sai da região, e libera outros processos esperando  
...
```



Veremos algumas possíveis abordagens e mecanismos para se garantir exclusão mútua de RCs

Usando Regiões Críticas

para resolver a concorrência no acesso a fila de *spooler*:

```
Process A/B {
```

```
...
```

```
    Enter-Region()
```

```
    copy file to Spooler[in];
```

```
    in = (in+1)%slots;
```

```
    Exit-Region()
```

```
...
```

```
}
```

```
Spooler {
```

```
    while(1) {
```

```
        Enter-Region();
```

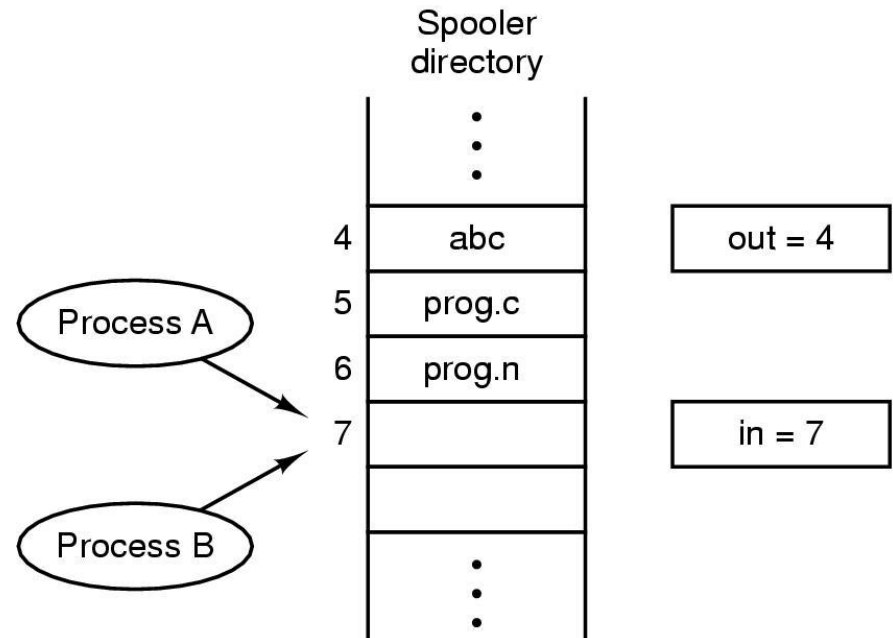
```
        print Spooler[out];
```

```
        out = (out+1)%slots;
```

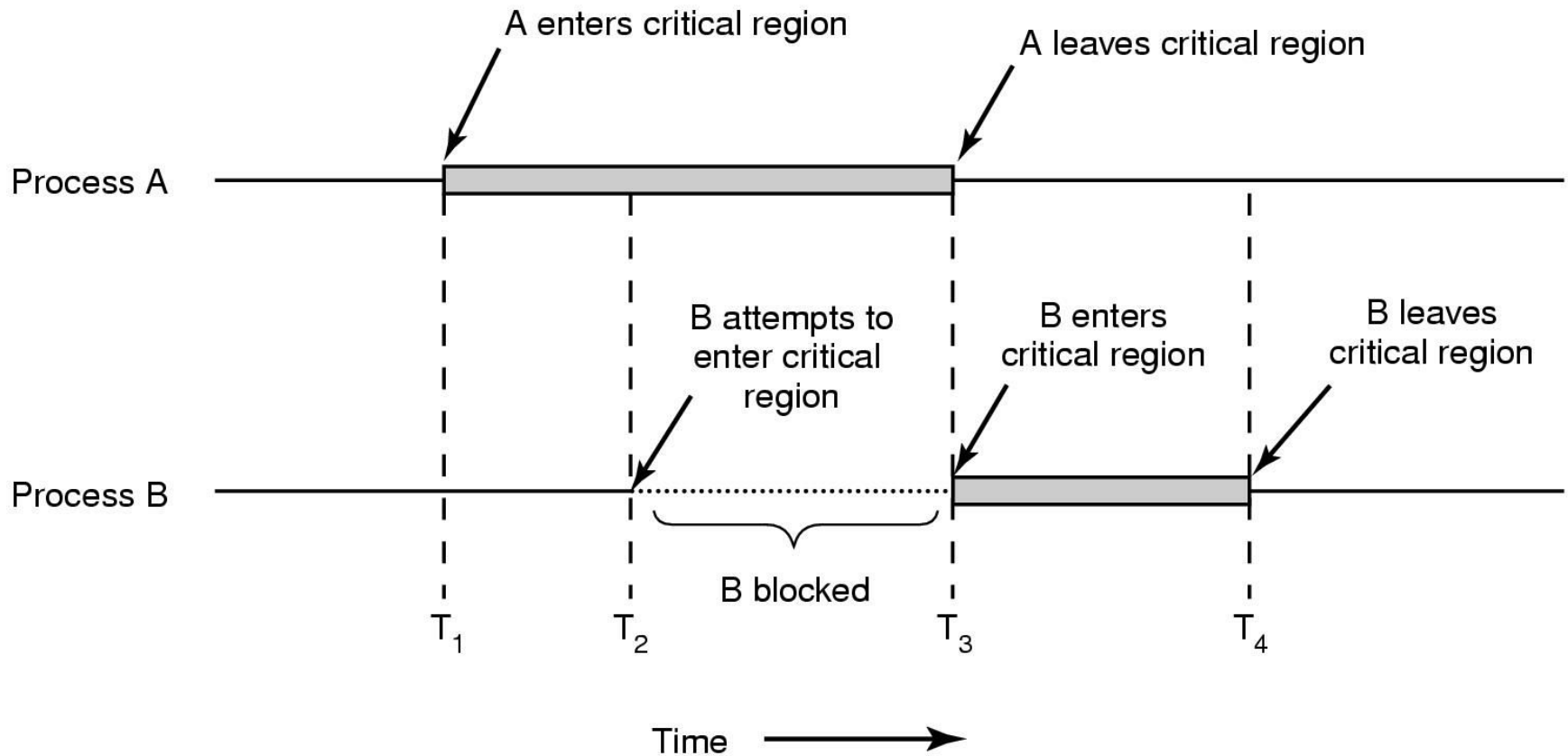
```
        Exit-Region();
```

```
    }
```

```
}
```



Região Crítica



Exclusão mútua usando Regiões Críticas

Formas de Implementar Regiões Críticas

Alternativas:

1. Desabilitar interrupções de hardware
➡ Pode ser feito pelo núcleo, mas não por um processo em modo usuário
2. Processos compartilham uma flag “lock”: se lock=0, trocar valor para 1 e processo entra RC, senão processo espera
➡ Se leitura e atribuição do lock não for atômica, então exclusão mútua não é garantida
3. Alternância regular de acesso por dois processos (PID= 0; PID= 1)
➡ Só funciona para 2 processos e apenas se a frequência de acesso ao recurso for mais ou menos a mesma.

```
while (TRUE) {  
    while (turn != 0)    /* loop */ ;  
    critical_region( );  
    turn = 1;  
    noncritical_region( );  
}
```

```
while (TRUE) {  
    while (turn != 1)    /* loop */ ;  
    critical_region( );  
    turn = 0;  
    noncritical_region( );  
}
```

Região Crítica com Espera Ocupada

```
#define FALSE 0
#define TRUE 1
#define N      2          /* number of processes */

int turn;                  /* whose turn is it? */
int interested[N];         /* all values initially 0 (FALSE) */

void enter_region(int process); /* process is 0 or 1 */
{
    int other;              /* number of the other process */

    other = 1 - process;    /* the opposite of process */
    interested[process] = TRUE; /* show that you are interested */
    turn = process;         /* set flag */
    while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process) /* process: who is leaving */
{
    interested[process] = FALSE; /* indicate departure from critical region */
}
```

Solução de Peterson:

- Variável *turn* e o vetor *interested[]* são variáveis compartilhadas
- Se dois processos $PID = \{0,1\}$ executam simultaneamente *enter_region*, o primeiro valor de *turn* será sobreescrito (e o processo correspondente vai entrar), mas *interested[first]* vai manter o registro do interesse do segundo processo

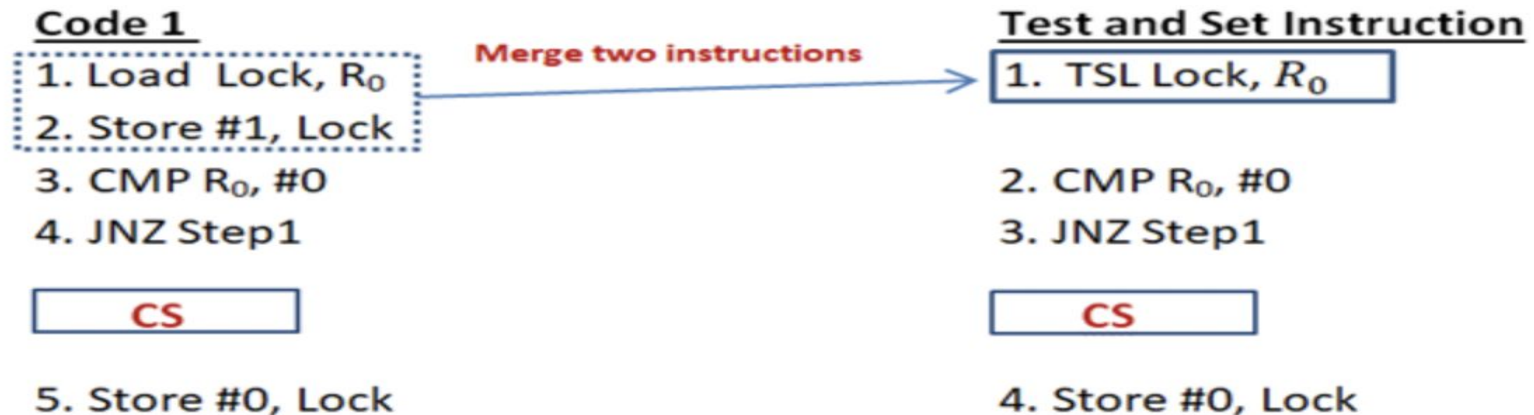
Exclusão Mútua com Espera Ocupada - Test and Set Lock

Algumas arquiteturas possuem uma instrução de máquina especial, **Test-and-Set-Lock (TSL)** para cópia de um lock para um registrador e simultaneamente a atribuição de um valor diferente de zero ($\neq 0$), **de forma atômica!**

Processos que desejam entrar RC executam TSL como forma de testar e setar a variavel lock atomicamente:

- se registrador (e lock=0), então entram na RC, senão esperam em loop

Initially Lock = 0



Espera Ocupada vs. Bloqueio em system calls

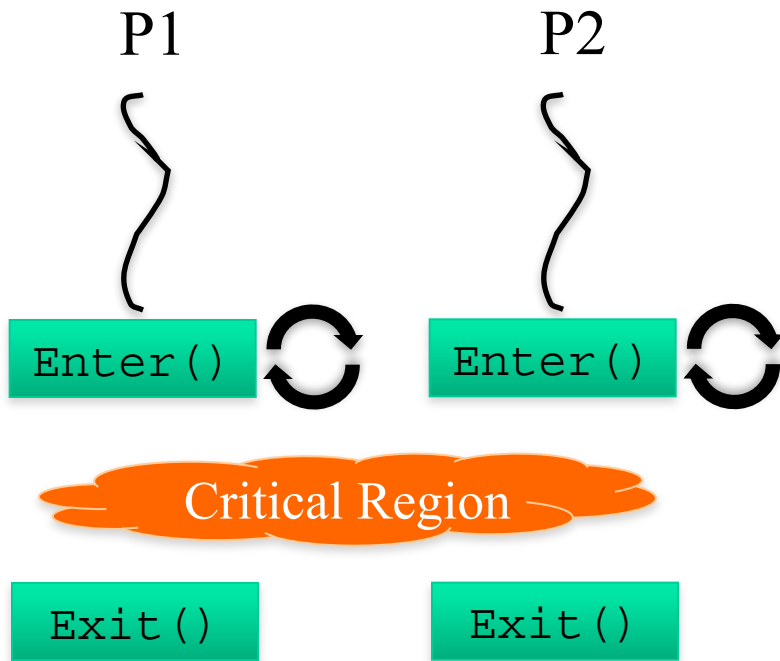
- Solução de Peterson e Test-and-Set-Lock têm o problema da espera ocupada que consome ciclos de processamento durante a espera de `enter_region()`
- Outro problema da espera Ocupada: Inversão de prioridades
Se um processo com baixa prioridade estiver na RC, demorará mais a ser escalonado (e a sair da RC), pois processos de maior prioridade estarão executando em espera ocupada (para entrar na RC).

A alternativa: System calls que bloqueiam o processo e o fazem esperar por um sinal de outro processo

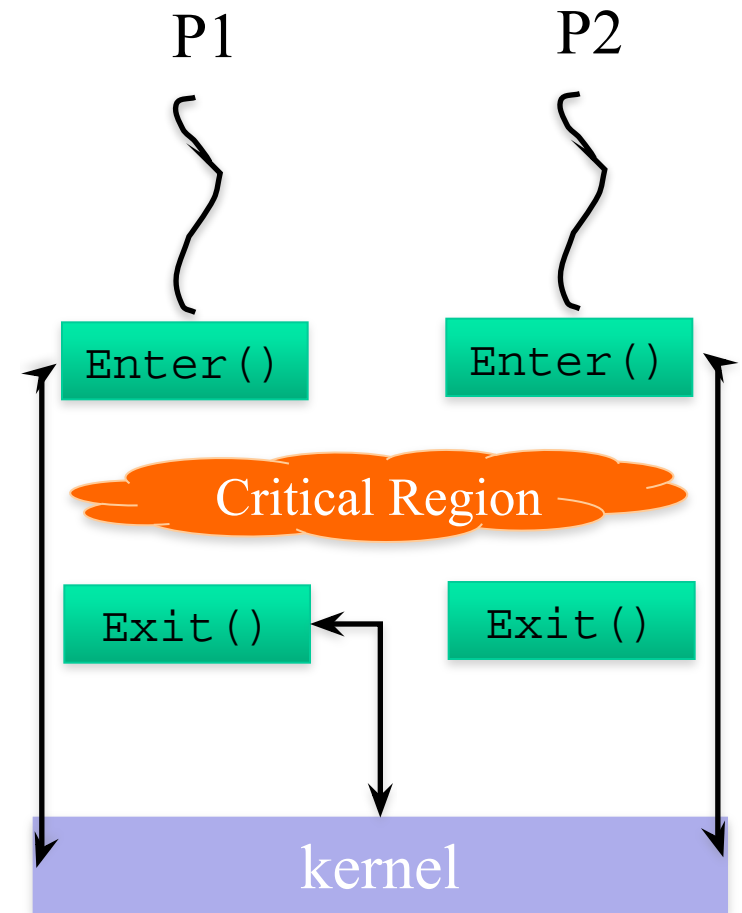
Por exemplo:

- *sleep:: suspende o processo até que seja acordado*
- *wakeup(PID):: sinal para acordar o processo PID*

Espera Ocupada *versus* system call para o núcleo



Espera Ocupada: consome ciclos de processamento.



system calls: o núcleo garante a atomicidade

Primitivas sleep & wakeup

As soluções anteriores são corretas mas utilizam a “espera ocupada” (“espera ociosa”)

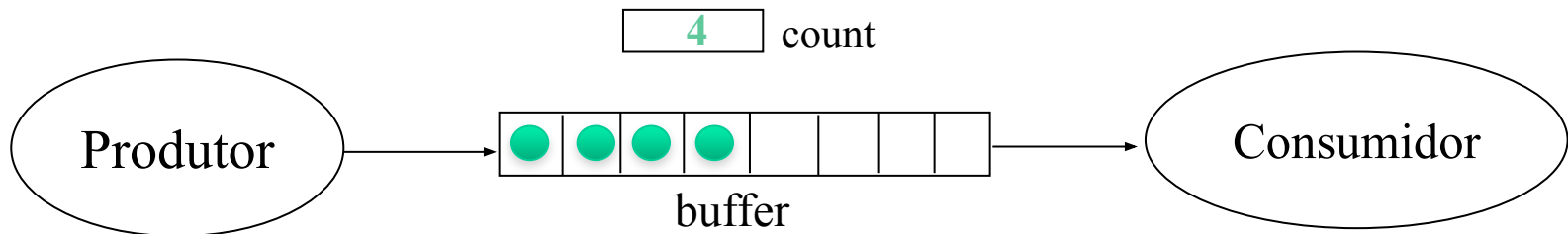
- O processo não consegue entrar na região crítica fica preso em um loop consumindo tempo de CPU
- podemos usar chamadas do SO sleep e wakeup
- em vez de espera ocupada, o sistema operacional pode bloquear o processo no sleep
- Sincronizando o produtor consumidor

Problema do Produtor e Consumidor

Sincronização de dois processos (um consumidor, outro produtor de itens) que compartilham um buffer em uma região crítica, e que usam uma variável compartilhada *count* para controlar o fluxo de controle.

- se $count=N$, produtor deve esperar, e
- se $count=0$ consumidor deve esperar,

Cada processo deve “acordar” o outro processo quando o conteúdo do buffer mudar a ponto de permitir o prosseguimento do processamento



Esse tipo de sincronização está relacionada ao estado do recurso (o buffer):

no caso, se itens podem ser retirados ou adicionados do buffer

-> **Sincronização de condição**

Problema do Produtor e Consumidor usando sleep & wakeup

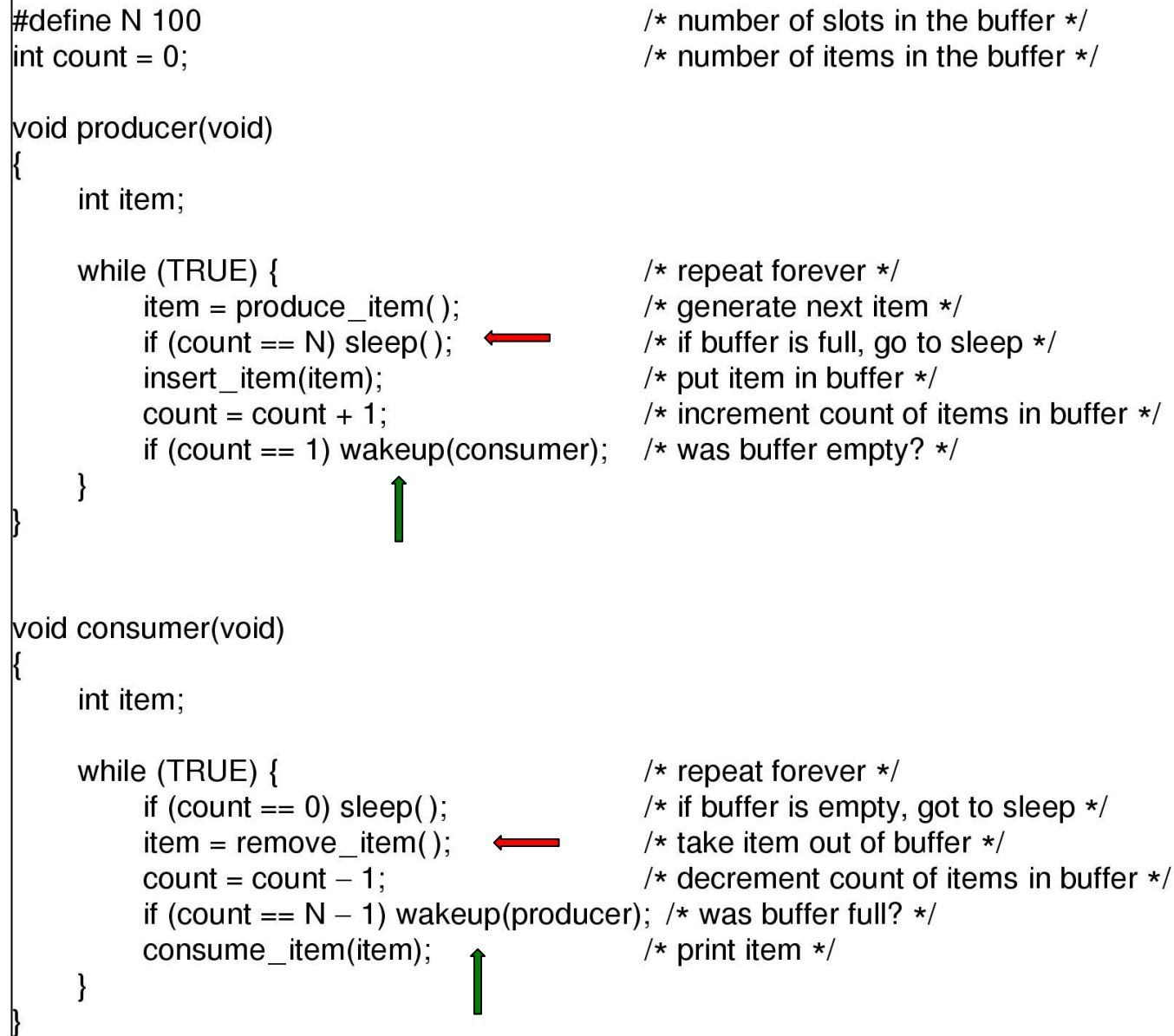
```
#define N 100                                     /* number of slots in the buffer */
int count = 0;                                   /* number of items in the buffer */

void producer(void)
{
    int item;

    while (TRUE) {                               /* repeat forever */
        item = produce_item();                  /* generate next item */
        if (count == N) sleep();                /* if buffer is full, go to sleep */
        insert_item(item);                      /* put item in buffer */
        count = count + 1;                      /* increment count of items in buffer */
        if (count == 1) wakeup(consumer);      /* was buffer empty? */
    }
}

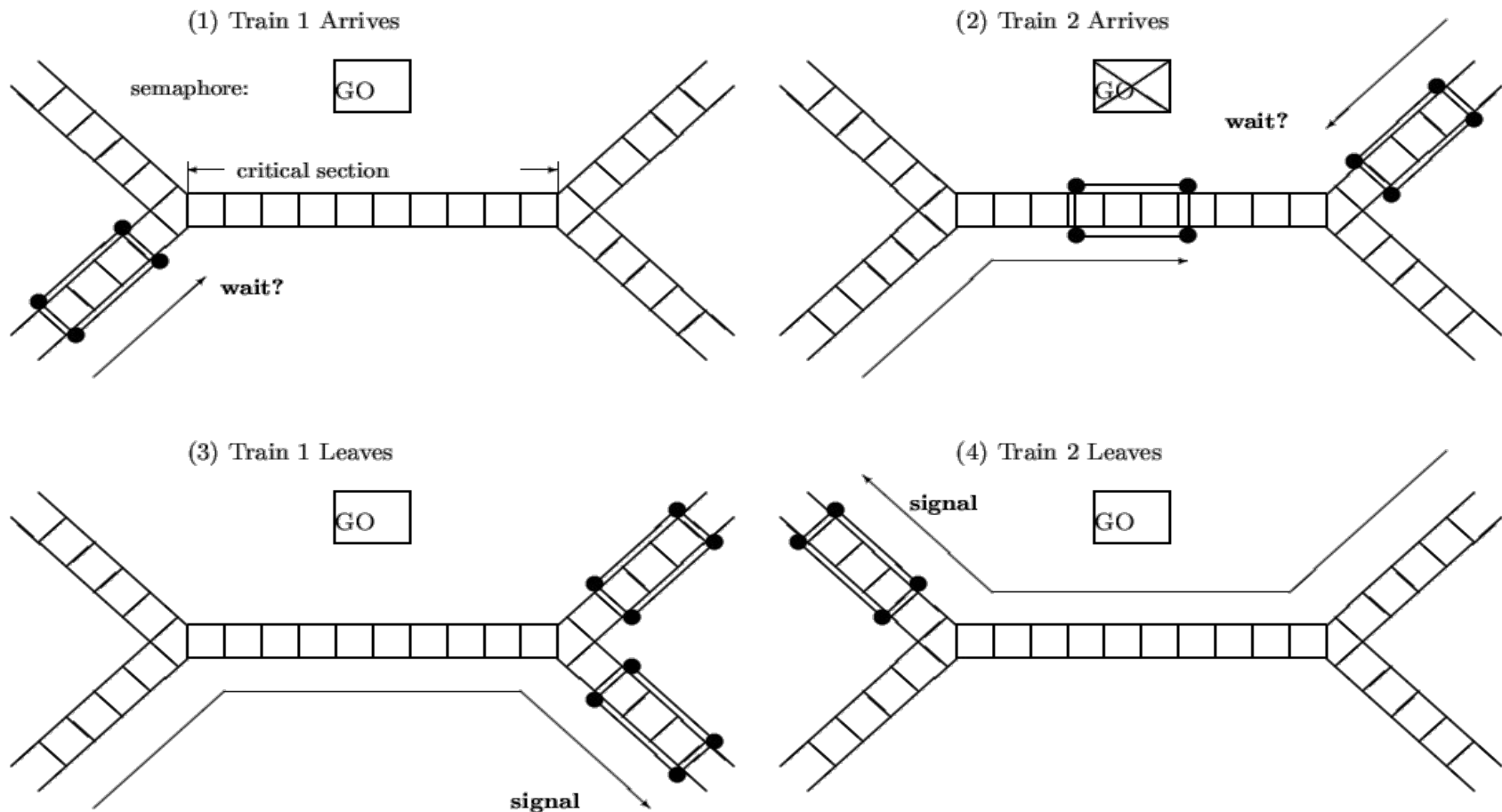
void consumer(void)
{
    int item;

    while (TRUE) {                               /* repeat forever */
        if (count == 0) sleep();                /* if buffer is empty, got to sleep */
        item = remove_item();                  /* take item out of buffer */
        count = count - 1;                     /* decrement count of items in buffer */
        if (count == N - 1) wakeup(producer);  /* was buffer full? */
        consume_item(item);                   /* print item */
    }
}
```



Semáforos

Em 1965 E.W. Dijkstra propôs o conceito de semáforos como mecanismo básico para sincronização entre processos. A inspiração: sinais de trens.



Edsger W. Dijkstra

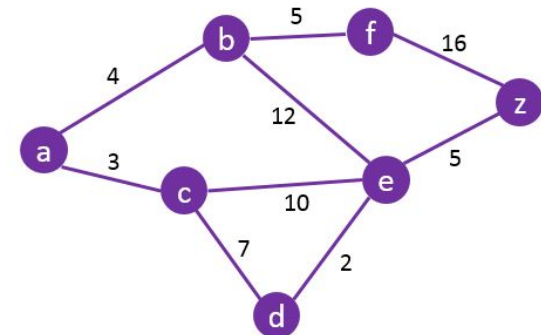


Edsger Wybe Dijkstra foi um cientista de sistemas de computação holandês, programador, engenheiro de software, ensaísta científico e pioneiro na ciência da computação. Físico teórico por formação, trabalhou como programador no Mathematisch Centrum de 1952 a 1962.

Nascido: 1930, Faleceu: 2002

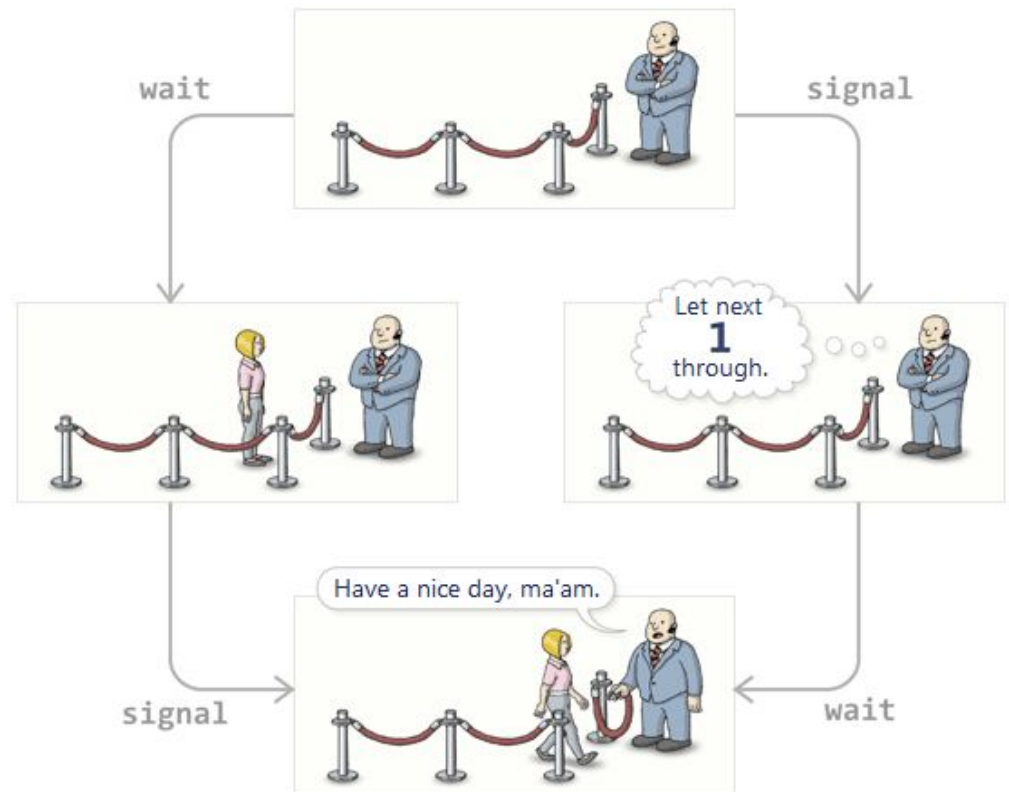
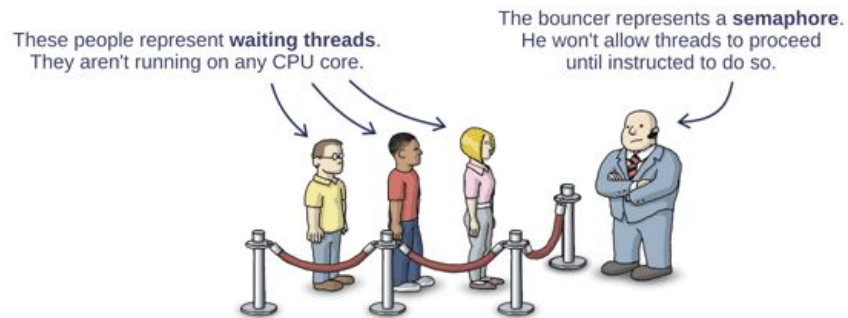
Sua contribuição mais conhecida?

- Algoritmo de Dijkstra – achar menor caminho de um ponto A a um ponto B em um grafo direcionado.



Semáforos

Outra Interpretação:



Semáforos

Um semáforo é um objeto que consiste de um contador, uma lista de espera de processos, e que pode ser manipulado por duas chamadas de sistema $P()$ e $V()$ (ou $down()$ e $up()$)

As duas operações **são atômicas! Não podem ser interrompidas**

- No caso da exclusão mútua as instruções *down* e *up* funcionam como protocolos de entrada e saída das regiões críticas:
 - ***down* (= *wait/ P*)**: é executada quando o processo deseja entrar na região crítica. Decrementa o semáforo de 1. Se valor ficar negativo, processo bloqueia.
 - ***up* (= *signal/ V*)**: quando o processo sai da sua região crítica, incrementa o semáforo de 1, possivelmente liberando algum processo bloqueado.

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <semaphore.h>

#define THREAD_NUM 4
sem_t semaphore;

void* routine(void* args) {
    sem_wait(&semaphore);
    sleep(1);
    printf("Hello from thread %d\n", *(int*)args);
    sem_post(&semaphore);
    free(args);
}

int main(int argc, char *argv[]) {
    pthread_t th[THREAD_NUM];
    sem_init(&semaphore, 0, 2);
    int i;
    for (i = 0; i < THREAD_NUM; i++) {
        int* a = malloc(sizeof(int));
        *a = i;
        if (pthread_create(&th[i], NULL, &routine, a) != 0) {
            perror("Failed to create thread");
        }
    }
    for (i = 0; i < THREAD_NUM; i++) {
        if (pthread_join(th[i], NULL) != 0) {
            perror("Failed to join thread");
        }
    }
    sem_destroy(&semaphore);
    return 0;
}

```

Exemplo:
4 threads executando print em paralelo a depender do valor so semáforo.

Hello from thread 3

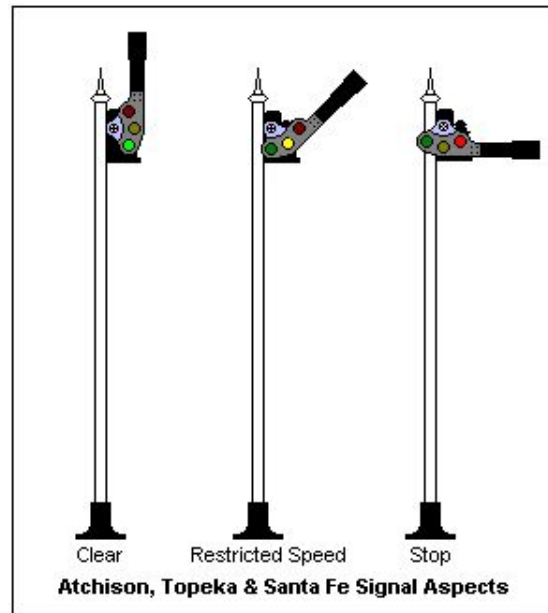
Hello from thread 1

Hello from thread 2

Hello from thread 0

Obs: sem_post (em pthreads) é equivalente a sem_signal

Semáforos - Operações



Um semáforo é gerenciado pelo núcleo.

- **down(&s)** :: verifica se pode entrar na Região crítica, e se não puder, bloqueia;
- **up(&s)** :: avisa que deixou região crítica, e causa o desbloqueio de um único processo bloqueado no semáforo;

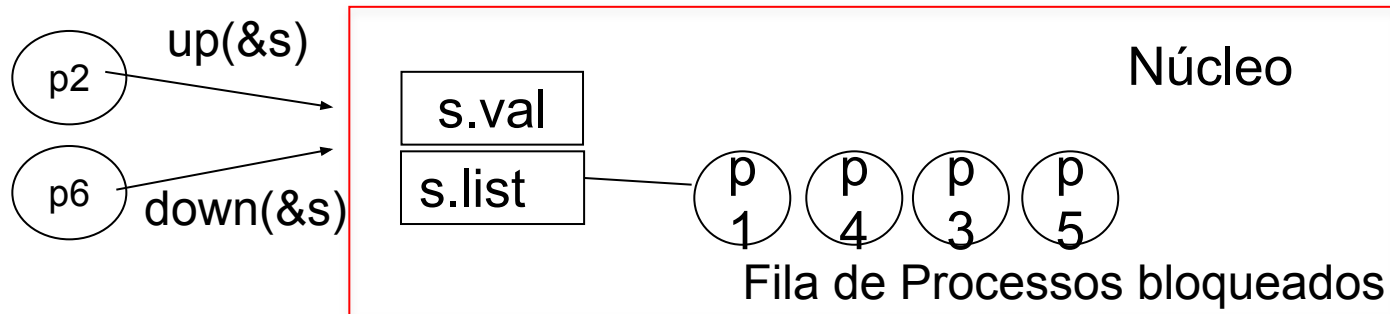
Semáforo - Implementação

val: é um contador que representa o número de processos/threads que podem entrar em uma Região Crítica.

list: A cada semáforo está associado uma lista de processos bloqueados.

Semântica das operações:

- **down(&s)** :: decrementa s.val. Se s.val < 0 processo invocador bloqueia nesta chamada e é colocado no final da fila. Senão, continua execução
- **up(&s)** :: Incrementa s.val, desbloqueia um dos processos bloqueados (se houver) e continua execução.



Obs: down/wait e up/signal são implementadas como chamadas do núcleo (system call), e durante a sua execução o núcleo desabilita temporariamente as interrupções (para garantir a atomicidade).

Semáforos - Implementação

Semaphore Structure:

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

Down Operation

```
down (semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}
```

Up Operation :

```
up (semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

Semáforos

- Para usar em C:
 - `#include </usr/include/semaphore.h>`
- Declarações e uso:
 - `sem_t semaphore`
 - `sem_init (&semaphore, 0, some_value);`
 - `sem_wait (&semaphore);`
 - `sem_post (&semaphore);`
- Obs: em Pthreads usa-se:
 - **wait** no lugar de down; **post** em vez de up.

Mutex: semáforos binários

Um **Mutex**, é um semáforo binário (que tem somente dois estados):

Ou seja, os valores s.val estão em $[-1,0]$, representando “ocupado” e “livre”

E as operações recebem outro nome:

- **mutex_lock** (é equivalente ao down)
- **mutex_unlock** (é equivalente ao up)

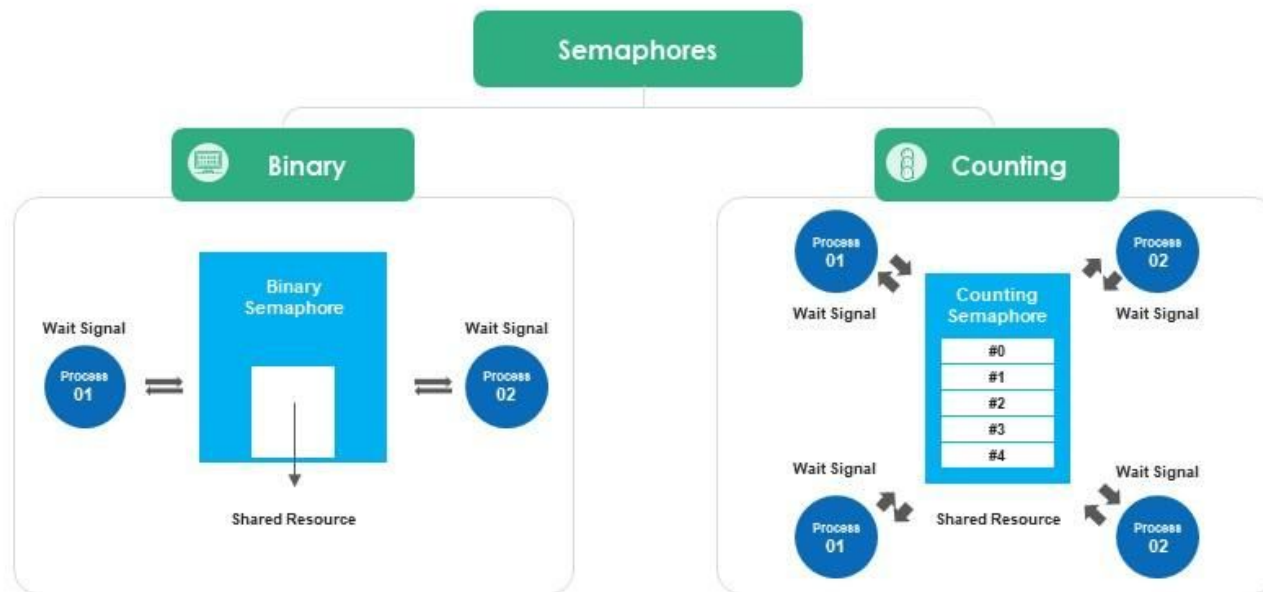
Os mutexes são usados para implementar exclusão mútua simples, isto é, **onde apenas um processo pode estar na região crítica.**

Obs: mutex_lock e mutex_unlock precisam ser chamados sempre pela mesma thread

Semáforos contadores vs Semáforos binários

Semaphore Implementation Process in Operating System

This slide shows implementation process of semaphore in operating system. It also includes two types of semaphores such as binary and counting along with various conditions while implementing semaphore.



Semaphore Implementation-

- Two variations of operations i.e. Wait and signal
- The semaphore value of wait is decreased by 1
- The semaphore value of signal is increased by 1
- Text here



Conditions for Implementing Semaphore

- Initialized to 1 before start
- Only non negative values allowed
- Text here
- Text here

Problema do Produtor e consumidor

Precisamos garantir:

- exclusão mútua no acesso ao buffer compartilhado
- produtor não deve inserir elementos no buffer quando este está cheio
- consumidor não deve consumir itens do buffer se este estiver vazio

a ideia seria usar

- **mutex** para controlar o acesso exclusivo ao buffer
- semáforo **empty** (como contador dos campos vazios do buffer) e
- semáforo **full** (como contador dos campos cheios do buffer)

Semáforos: Exemplo de uso

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

```
/* number of slots in the buffer */
/* semaphores are a special kind of int */
/* controls access to critical region */
/* counts empty buffer slots */
/* counts full buffer slots */
```

```
/* TRUE is the constant 1 */
/* generate something to put in buffer */
/* decrement empty count */
/* enter critical region */
/* put new item in buffer */
/* leave critical region */
/* increment count of full slots */
```

```
/* infinite loop */
/* decrement full count */
/* enter critical region */
/* take item from buffer */
/* leave critical region */
/* increment count of empty slots */
/* do something with the item */
```

Semáforos:

sem_empty = contador
de campos vazios
sem_full = contador de
campos preenchidos
mutex = exclusão mutua

Produtor consumidor com threads e semáforos

```
#include <semaphore.h>
#define THREAD_NUM 8

sem_t semEmpty;
sem_t semFull;

pthread_mutex_t mutexBuffer;

int buffer[10];
int count = 0;

void* producer(void* args) {
    while (1) {
        int x = rand() % 100;
        sleep(1);
        // Add to the buffer
        sem_wait(&semEmpty);
        pthread_mutex_lock(&mutexBuffer);
        buffer[count] = x;
        count++;
        pthread_mutex_unlock(&mutexBuffer);
        sem_post(&semFull);
    }
}

void* consumer(void* args) {
    while (1) {
        int y;
        // Remove from the buffer
        sem_wait(&semFull);
        pthread_mutex_lock(&mutexBuffer);
        y = buffer[count - 1];
        count--;
        pthread_mutex_unlock(&mutexBuffer);
        sem_post(&semEmpty);
        // Consume
        printf("Got %d\n", y);
        sleep(1);
    }
}
```

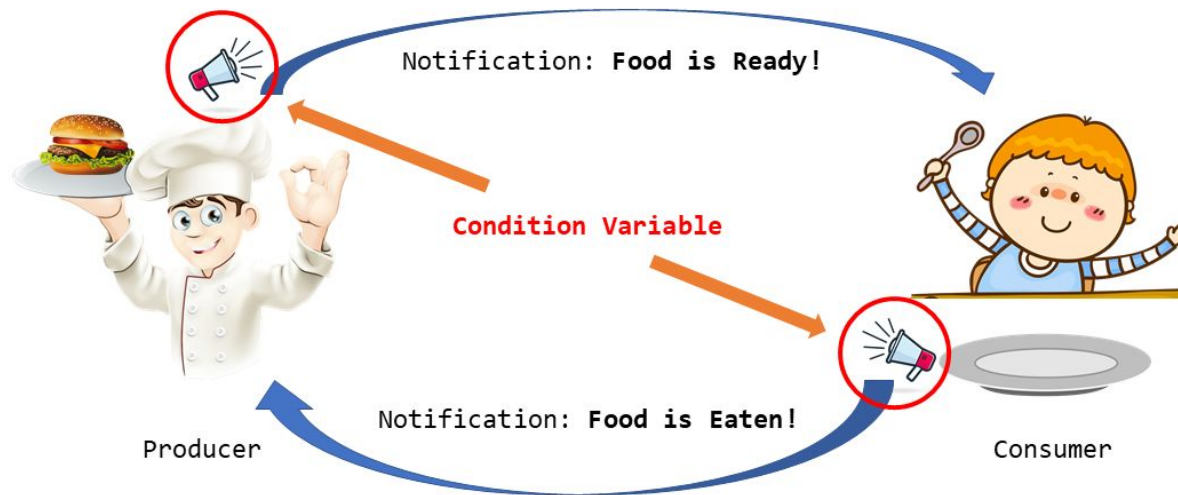
```
int main(int argc, char* argv[]) {
    srand(time(NULL));
    pthread_t th[THREAD_NUM];
    pthread_mutex_init(&mutexBuffer, NULL);
    sem_init(&semEmpty, 0, 10);
    sem_init(&semFull, 0, 0);
    int i;
    for (i = 0; i < THREAD_NUM; i++) {
        if (i > 0) {
            if (pthread_create(&th[i], NULL, &producer, NULL) != 0) {
                perror("Failed to create thread");
            }
        } else {
            if (pthread_create(&th[i], NULL, &consumer, NULL) != 0) {
                perror("Failed to create thread");
            }
        }
    }
    for (i = 0; i < THREAD_NUM; i++) {
        if (pthread_join(th[i], NULL) != 0) {
            perror("Failed to join thread");
        }
    }
    sem_destroy(&semEmpty);
    sem_destroy(&semFull);
    pthread_mutex_destroy(&mutexBuffer);
    return 0;
}
```

Variável de Condição

Uma variável de condição é uma variável usada para notificar outros processos/threads. Para comunicar threads quando determinadas condições se tornam satisfeitas.

Uma variável de condição é usada dentro de uma região crítica,

- processos/threads se colocam em uma fila esperando uma notificação de outra thread que acabou de sair da região crítica. (**cond_wait()**)
- Essa thread irá sinalizar a mudança de condição para as threads que estão esperando pela condição. (**cond_signal()**)



Condition Variables

Em programas multi-threaded, é frequentemente útil para uma thread esperar que alguma condição se torne verdadeira antes de prosseguir.

A abordagem simples, de apenas executar em loop (busy waiting) até que a condição se torne verdadeira, é ineficiente e desperdiça ciclos de CPU.

Em vez disso, melhor é a thread se colocar para dormir até que a condição para executar tenha sido satisfeita.

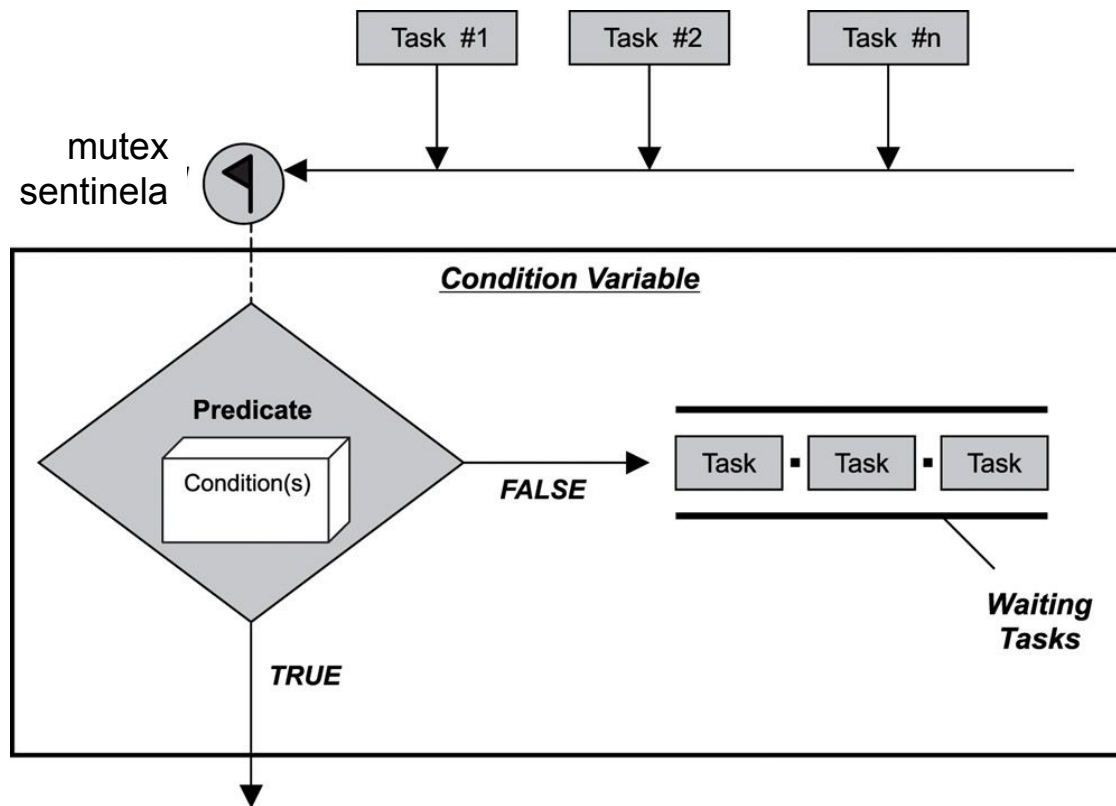
Uma variável de condição é uma fila em que as threads se colocam quando estão esperando por uma condição que é modificável por outra thread. (**cond_wait()**)

Quando isso acontece essa thread pode sinalizar a mudança de condição para as threads que estão esperando pela condição.

(**cond_signal()**)

Variável de Condição

Ideia central: bloquear e esperar ao mesmo tempo para uma condição ser satisfeita e o acesso exclusivo a uma sessão crítica



Ao contrário de Semáforos, variáveis de condição não possuem um contador (não têm uma memória)

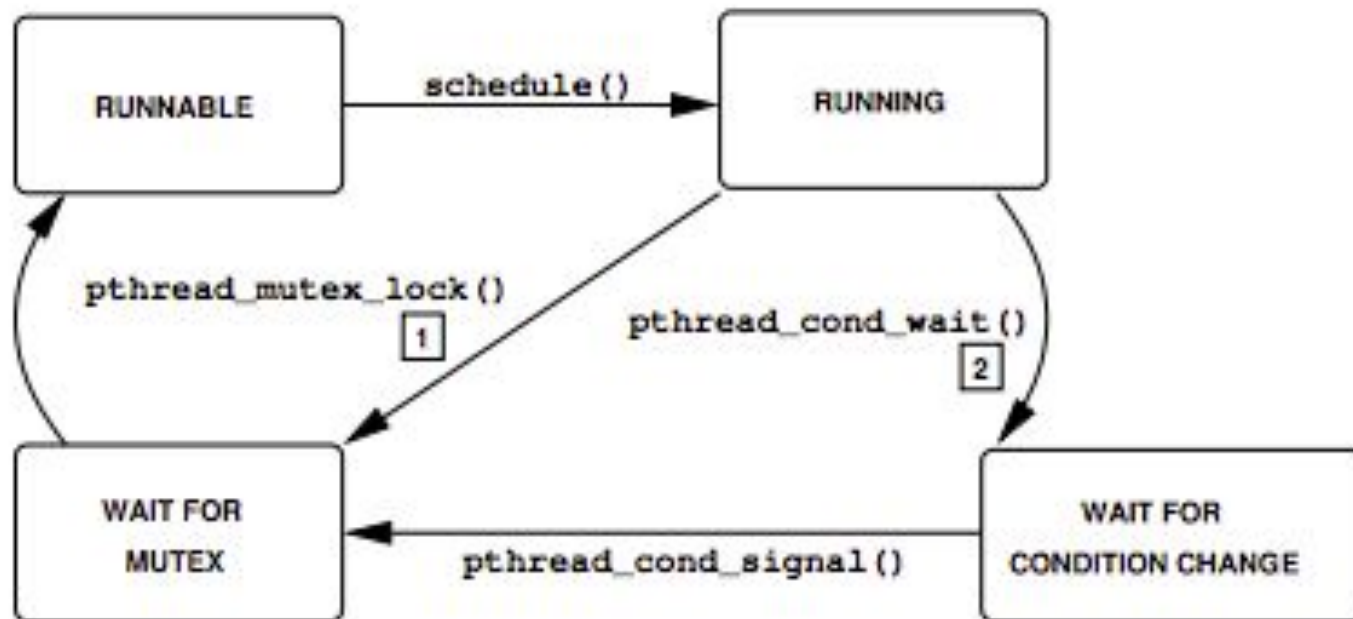
Fonte: <http://www.embeddedlinux.org.cn/>

Variável de Condição

- Variáveis de condição sempre estão associadas a um mutex.
- Permite que threads em uma região crítica sejam suspensos até uma condição estiver satisfeita.
- Threads/processos que estejam na região crítica abrem mão do lock da exclusão mútua deixando um outro processo entrar (para que possa fazer a condição ficar satisfeita)
- Operações: **wait**, **signal** (ou **notify**) e **broadcast**

Espera por Mutex e por Variável de Condição

Diagrama de estados:



- Primeiro, adquire o lock do mutex e depois espera o sinal
- Nessa espera, o lock é mantido, mas outro processo pode entrar na Região Crítica

Produtor consumidor com variáveis de condição

```
variáveis compartilhadas
cond_t not_empty, not_full;
int slots = 0;
mutex_t lock;
```

Produtor:

```
Produce_one(void)
mutex_lock(&lock)
while (slots == N)
    cond_wait(&not_full, &lock);
my = get_empty(&buffer);
Fill(&buffer[my];
slots++;
cond_signal(&not_empty);
mutex_unlock(&lock);
}
```

Consumidor:

```
Consume_one(void)
mutex_lock(&lock)
while (slots == 0)
    cond_wait(&not_empty, &lock);
my = get_filled(&buffer);
Use(&buffer[my];
slots--;
cond_signal(&not_full);
mutex_unlock(&lock);
}
```

Exemplo de Variáveis de Condição

Thread `fuel_filling` notifica a thread `car` quando o abastecimento terminou

```
pthread_mutex_t mutexFuel;
pthread_cond_t condFuel;
int fuel = 0;

void* fuel_filling(void* arg) {
    for (int i = 0; i < 5; i++) {
        pthread_mutex_lock(&mutexFuel);
        fuel += 15;
        printf("Filled fuel... %d\n", fuel);
        pthread_mutex_unlock(&mutexFuel);
        pthread_cond_signal(&condFuel);
        sleep(1);
    }
}

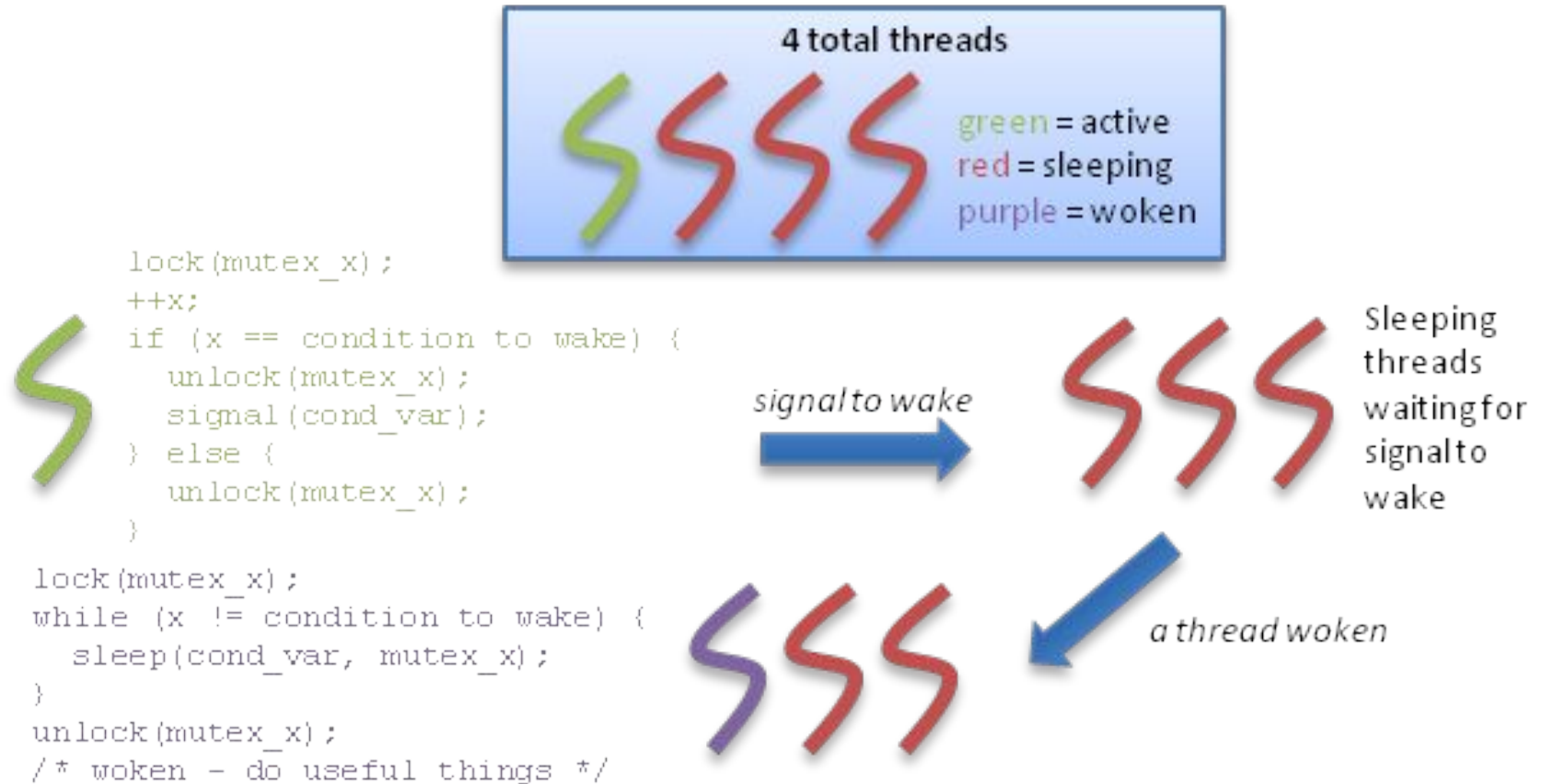
void* car(void* arg) {
    pthread_mutex_lock(&mutexFuel);
    while (fuel < 40) {
        printf("No fuel. Waiting...\n");
        pthread_cond_wait(&condFuel, &mutexFuel);
        // Equivalent to:
        // pthread_mutex_unlock(&mutexFuel);
        // wait for signal on condFuel
        // pthread_mutex_lock(&mutexFuel);
    }
    fuel -= 40;
    printf("Got fuel. Now left: %d\n", fuel);
    pthread_mutex_unlock(&mutexFuel);
}

int main(int argc, char* argv[]) {
    pthread_t th[2];
    pthread_mutex_init(&mutexFuel, NULL);
    pthread_cond_init(&condFuel, NULL);
    for (int i = 0; i < 2; i++) {
        if (i == 1) {
            if (pthread_create(&th[i], NULL,
                &fuel_filling, NULL) != 0) {
                perror("Failed to create thread");
            }
        } else {
            if (pthread_create(&th[i], NULL, &car,
                NULL) != 0) {
                perror("Failed to create thread");
            }
        }
    }

    for (int i = 0; i < 2; i++) {
        if (pthread_join(th[i], NULL) != 0) {
            perror("Failed to join thread");
        }
    }

    pthread_mutex_destroy(&mutexFuel);
    pthread_cond_destroy(&condFuel);
    return 0;
}
```

Uso típico de mutex e Variáveis de Condição



Variáveis de Condição em outras linguagens

Java:

```
private Lock aLock = new ReentrantLock();
private Condition condVar =
aLock.newCondition();

public int get(int who) {
    aLock.lock();
    try {
        while (available == false) {
            try {
                condVar.await();
            } catch (InterruptedException e)
            { }
        }
        available = false;
        condVar.signalAll();
    } finally {
        aLock.unlock();
        return contents;
    }
}
```

Existe a versão bloqueante e com timeout:

- `await()`
- `await(long timeout)` – se notificação não ocorre até timeout, retorna

Semáforos em UNIX

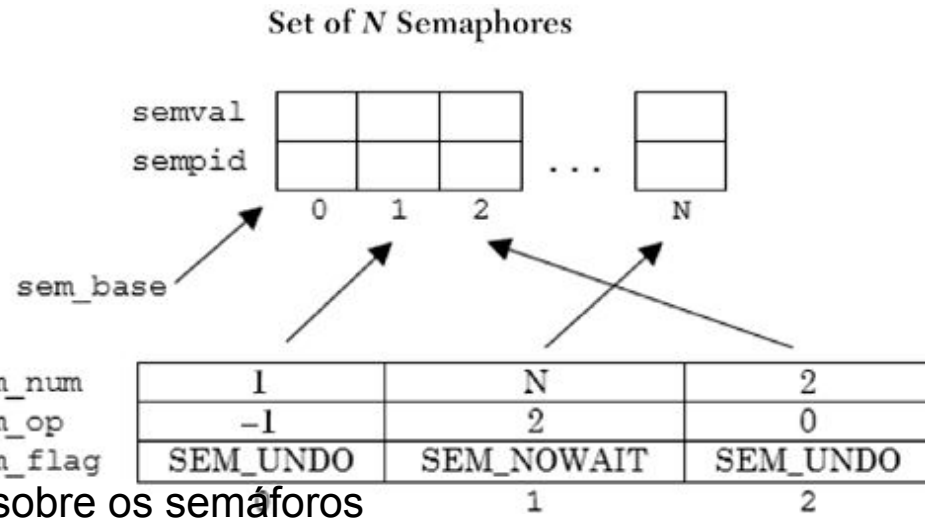
Aloca-se um vetor de semáforos

• `int semget( key, nsems, semflg)`

- key: chave do set
- nsems: número de semáforos
- semflg: `IPC_CREAT` | `IPC_EXCL`
- Retorna `semID`

• `int semop( semID, &sops, nsops)`

- `semID`
- `sops`: ponteiro para vetor de operações sobre os semáforos
- `nsops`: número de operações



Núcleo garante atomicidade de todas as operações sobre o vetor.

• `int semctl ( semID, int semnum, cmd, arg );`

- Operações de controle sobre o conjunto de semáforos.
- Destruição de um vetor de semáforos:

```
semctl(semid, 0, IPC_RMID, 0);
```

O Mecanismo Monitor

Monitor é um mecanismo de sincronização que:

- permite que threads tenham tanto a exclusão mútua quanto a capacidade de esperar que uma determinada condição se torne verdadeira.
- possuem um mecanismo para sinalizar a outros threads que sua condição foi atendida.

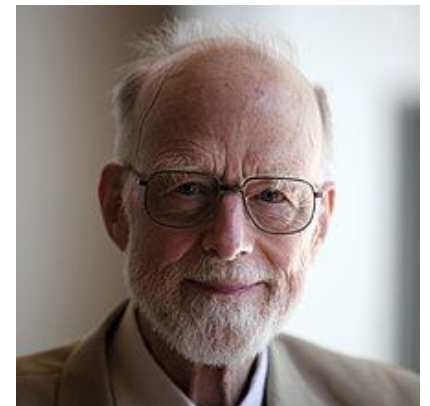
Um monitor consiste em um objeto mutex e variáveis de condição.

- possibilitam que threads desistam temporariamente do acesso exclusivo a fim de esperar que alguma condição seja atendida, antes de recuperar o acesso exclusivo e retomar sua tarefa.

Foi inventado por Per Brinch Hansen e C. A. R. Hoare, e incorporado como primitiva em linguagens Concurrent Pascal, Modula2 e Modula3



Per Brinch Hansen



Sir Tony Hoare

Monitor – Objeto com sincronização

Idéia central:

- Usar o princípio de encapsulamento de dados também para a exclusão mútua,
- Os dados internos do monitor só são acessados através dos **procedimentos** do Monitor;
- **A cada momento, apenas um único procedimento do monitor pode ser executado;**
- O monitor gerencia as threads esperando pelo término da execução do procedimento “em execução”

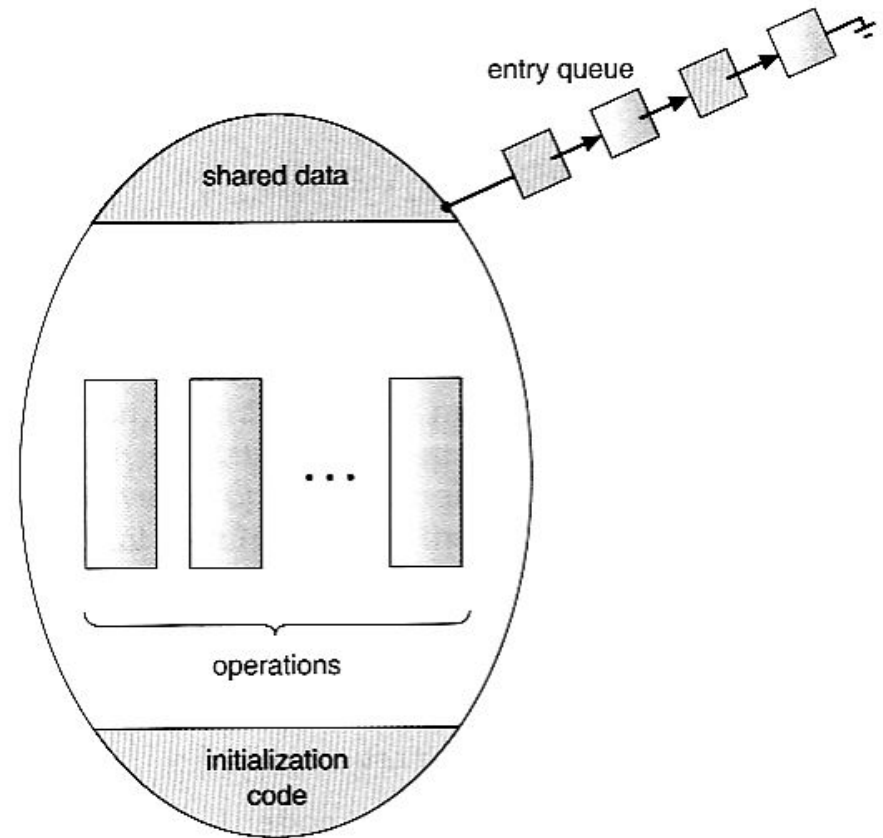
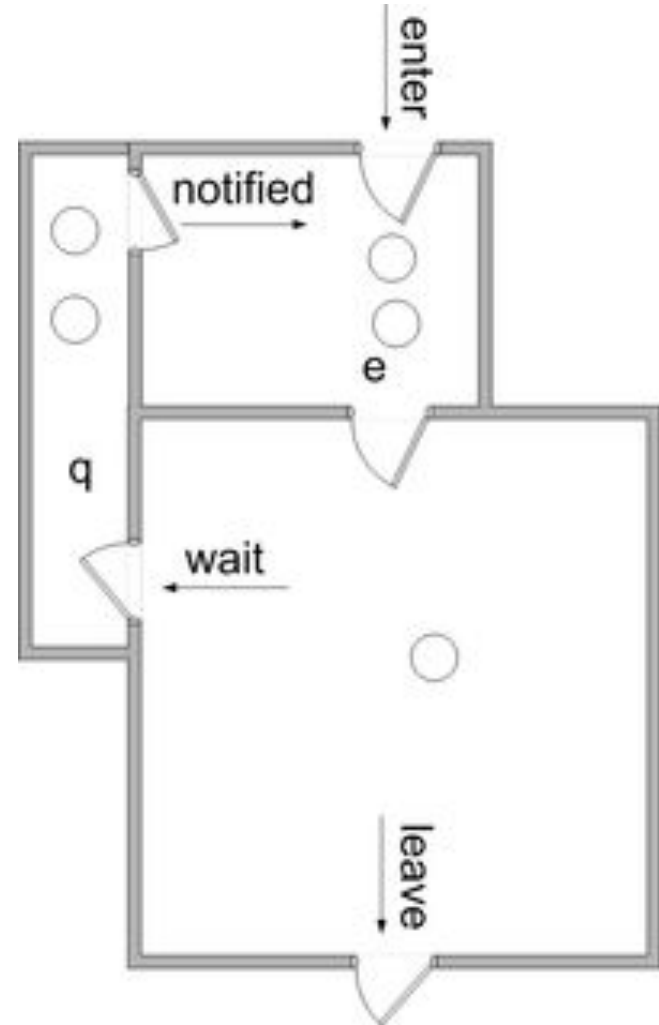


Figure 6.17 Schematic view of a monitor.

Monitor

Monitor é como um restaurante fast food, onde:

- voce (thread) entra,
- espera pela comida,
- eventualmente é notificado que a comida está pronta,
- come e paga e
- depois sai do restaurante.



Monitores

```
monitor monitor-name
{
    declaração de variáveis compartilhadas
    procedure P1 (...) {
        ...
    }
    procedure P2 (...) {
        ...
    }
    procedure Pn (...) {
        ...
    }

    {
        código de inicialização
    }
}
```

A cada Monitor está associado a um **mutex implícito**. A cada chamada de procedimento é executado um Down, e a cada retorno um Up.

Monitor

- É um objeto com uma coleção de procedimentos que manipulam dados compartilhados internos ao monitor.
- Existe um único mutex associado ao Monitor.
- Cada procedimento adquire o `mutex.lock` antes de começar a executar e faz `mutex.unlock` ao retornar.
- Assim, cada procedimento pode acessar os dados compartilhados do Monitor em regime de exclusão mútua.
- Variáveis de condição são usadas para bloquear procedimentos que necessitem de certa condição.

Monitor

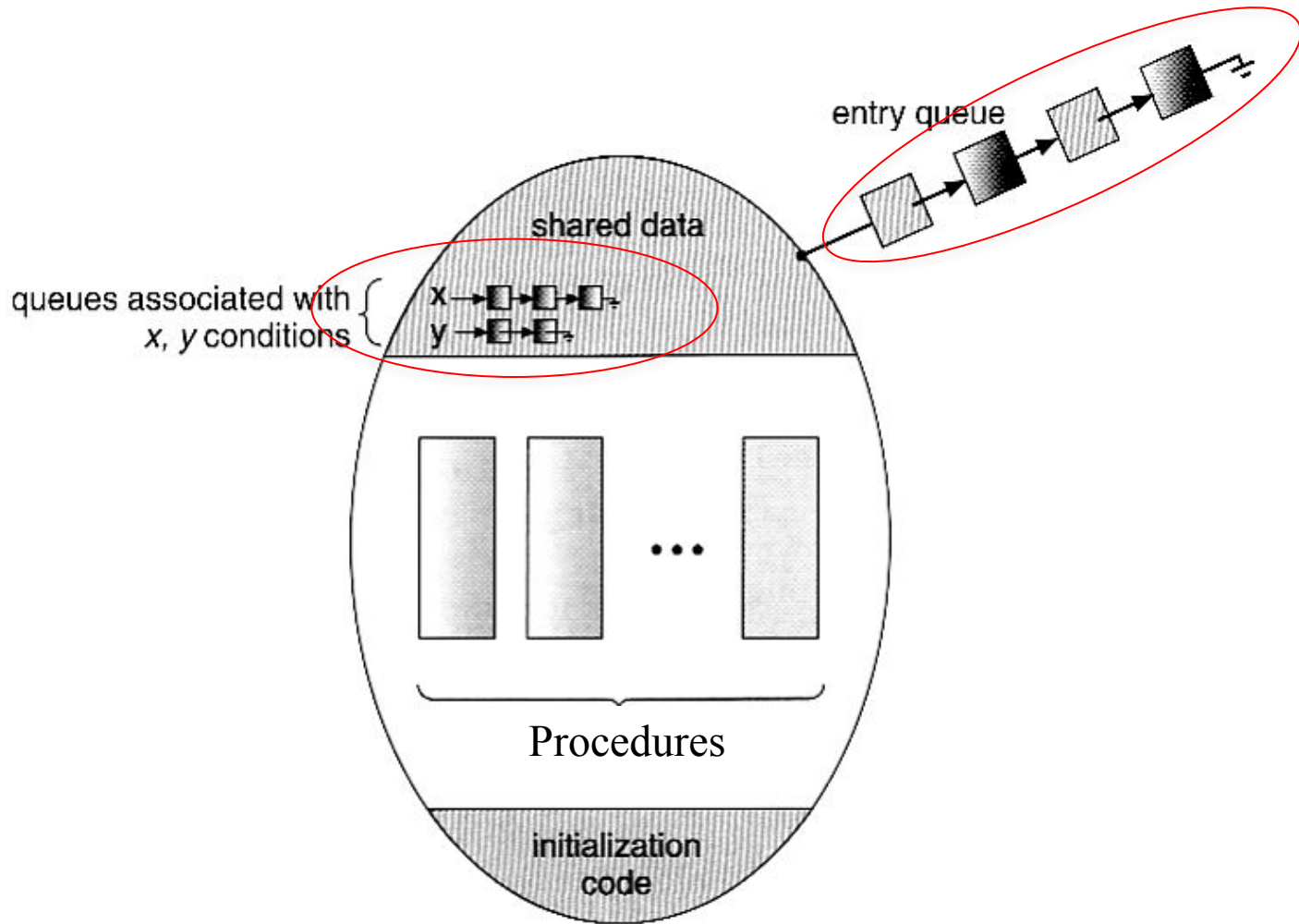


Figure 6.18 Monitor with condition variables.

Monitor

```
monitor example
  integer i;
  condition c;

  procedure producer( );

  .
  . wait(c)
  .
  end;

  procedure consumer( );

  .
  . signal(c)
  .
  end;
end monitor;
```

- Monitor é um elemento da linguagem de programação que combina o encapsulamento de dados com o controle para acesso sincronizado
- Usa-se variáveis de condição (com operações **wait** e **signal**), quando o procedimento em execução não consegue completar e precisa que outro procedimento seja completado;

Monitor

Para implementar a sincronização em Monitores é necessário utilizar variáveis de condição:

- *wait (Cond)*: suspende a execução da thread, fazendo ela liberar o mutex, e colocando-a em estado de espera associado a condição C
- *signal (Cond)*: permite que uma thread bloqueada por *wait(C)* continue a sua execução.
Se existirem mais de uma thread bloqueado, apenas uma é liberada. Senão, não tem efeito.
- *broadcast(Cond)* acorde todas as threads esperando na condição.

Monitor: Exemplo de Uso

```
monitor ProducerConsumer
  condition full, empty;
  integer count;
  procedure insert(item: integer);
  begin
    if count = N then wait(full);
    insert_item(item);
    count := count + 1;
    if count = 1 then signal(empty)
  end;
  function remove: integer;
  begin
    if count = 0 then wait(empty);
    remove = remove_item;
    count := count - 1;
    if count = N - 1 then signal(full)
  end;
  count := 0;
end monitor;
```

```
procedure producer;
begin
  while true do
    begin
      item = produce_item;
      ProducerConsumer.insert(item)
    end
  end;
procedure consumer;
begin
  while true do
    begin
      item = ProducerConsumer.remove;
      consume_item(item)
    end
  end;
```

Resolvendo o problema do produtor-consumidor com monitores

- Exclusão mútua dentro do monitor e controle explícito de sincronização garante a coerência do estado do buffer
- buffer tem N entradas

Exemplo: Monitor em Java

```
class Buffer {
    private char [] buffer;
    private int count = 0, in = 0, out = 0;
    Buffer(int size) {
        buffer = new char[size];
    }
    public synchronized void Put(char c) {
        while(count == buffer.length)
        {
            try { wait(); }
            catch (InterruptedException e) { }
            finally { }
        }
        System.out.println("Producing "+c+" ...");
        buffer[in] = c;
        in = (in + 1) % buffer.length;
        count++;
        notify();
    }
    ...
}

public synchronized char Get() {
    while (count == 0)
    {
        try { wait(); }
        catch (InterruptedException e) { }
        finally { }
    }
    char c = buffer[out];
    out = (out + 1) % buffer.length;
    count--;
    System.out.println("Consuming " + c + " ...");
    notify();
    return c;
}
```

Principal Diferença entre Monitores e Semáforos

Monitor:

- só serve para threads, já que processos não têm acesso a dados compartilhados (as instâncias de monitor)
- Monitor é um mecanismo mais abstrato, que estende o encapsulamento para a exclusão mútua
-

Semáforo:

- É um mecanismo de sincronização mais básico e pode ser provido por núcleos de S.O. ou runtime de uma linguagem programação (i.e. máquina virtual)
- podem ser usados por processos e threads

Problemas Clássicos de Concorrência

- Produtor-Consumidor
- Leitores e Escritores excludentes
- Barbeiro Dorminhoco
- Jantar dos Filósofos
- Sincronização de Barreira

Impasse e Inanição (Starvation)

As duas situações a serem evitadas em problemas de concorrência são:

Impasse (Definição): Um conjunto de processos está em situação de impasse (deadlock) se cada entidade pertencente ao conjunto está bloqueada esperando por um evento (ou recurso) que somente outra entidade no mesmo conjunto pode gerar (ou liberar).

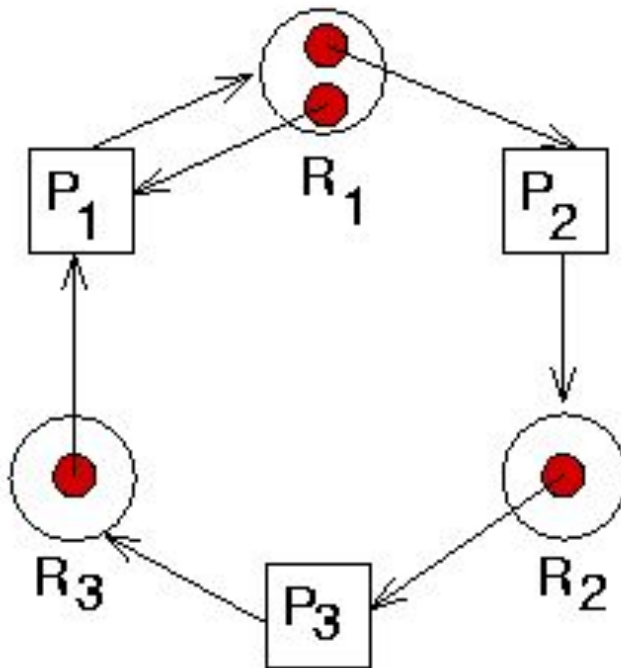
Inanição (Definição): Um processo está em inanição (starvation) quando ocorrem negativas de acesso a um determinado recurso perpetuamente, impedindo que o processo execute o restante das tarefas, sem que o mesmo esteja bloqueado.

P.S.: quando o processo está com baixa prioridade ou as demandas dos outros processos são muito mais frequentes

Grafo de Alocação de Recursos

Representação de uma situação de alocação e demanda por recursos por processos

Se houver um ciclo, então têm-se um impasse

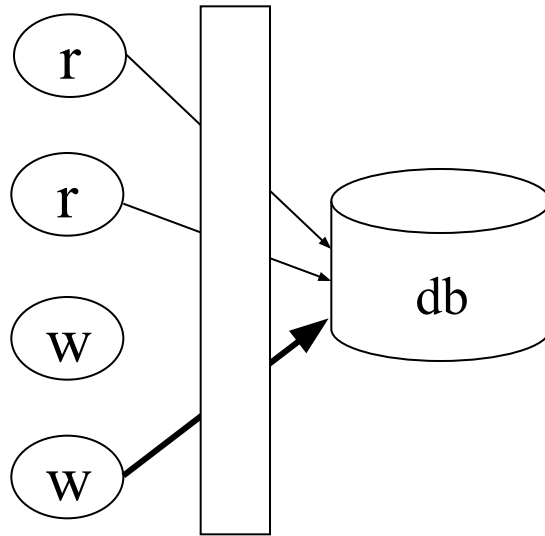


$R \rightarrow P$: recurso R
alocado ao processo P

$P \rightarrow R$: processo P
requisitando o recurso R

Problema dos Leitores e Escritores Excludentes: solução com semáforos

Objetivo: Vários leitores podem entrar a Sessão Crítica ao mesmo tempo, mas escritores precisam executar em exclusão mútua.



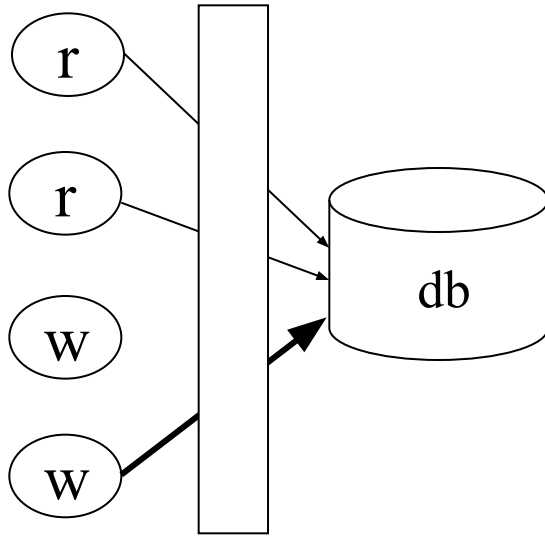
acesso exclusivo
para cada escritor

```
typedef int semaphore;           /* use your imagination */
semaphore mutex = 1;             /* controls access to 'rc' */
semaphore db = 1;                /* controls access to the database */
int rc = 0;                      /* # of processes reading or wanting to */
```

```
void reader(void)
{
    while (TRUE) {               /* repeat forever */
        down(&mutex);            /* get exclusive access to 'rc' */
        rc = rc + 1;             /* one reader more now */
        if (rc == 1) down(&db);  /* if this is the first reader ... */
        up(&mutex);              /* release exclusive access to 'rc' */
        read_data_base();        /* access the data */
        down(&mutex);            /* get exclusive access to 'rc' */
        rc = rc - 1;             /* one reader fewer now */
        if (rc == 0) up(&db);    /* if this is the last reader ... */
        up(&mutex);              /* release exclusive access to 'rc' */
        use_data_read();         /* noncritical region */
    }
}
```

```
void writer(void)
{
    while (TRUE) {               /* repeat forever */
        think_up_data();         /* noncritical region */
        down(&db);               /* get exclusive access */
        write_data_base();       /* update the data */
        up(&db);                 /* release exclusive access */
    }
}
```

Problema dos Leitores e Escritores Excludentes: solução com semáforos



Há aqui um risco potencial de impasse?

Mas pode-se definir prioridades diferentes.

Nessa variante, dá-se prioridade para os leitores, que só são bloqueados se um escritor (writer) estiver acessando db.

```
typedef int semaphore;           /* use your imagination */
semaphore mutex = 1;            /* controls access to 'rc' */
semaphore db = 1;               /* controls access to the database */
int rc = 0;                     /* # of processes reading or wanting to */

void reader(void)
{
    while (TRUE) {              /* repeat forever */
        down(&mutex);           /* get exclusive access to 'rc' */
        rc = rc + 1;            /* one reader more now */
        if (rc == 1) down(&db); /* if this is the first reader ... */
        up(&mutex);             /* release exclusive access to 'rc' */
        read_data_base();       /* access the data */
        down(&mutex);           /* get exclusive access to 'rc' */
        rc = rc - 1;            /* one reader fewer now */
        if (rc == 0) up(&db);    /* if this is the last reader ... */
        up(&mutex);             /* release exclusive access to 'rc' */
        use_data_read();        /* noncritical region */
    }
}

void writer(void)
{
    while (TRUE) {              /* repeat forever */
        think_up_data();        /* noncritical region */
        down(&db);              /* get exclusive access */
        write_data_base();      /* update the data */
        up(&db);                /* release exclusive access */
    }
}
```

O Problema do Barbeiro Dorminhoco

Pode haver até 5 clientes esperando pelo serviço.

Se todas cadeiras estão ocupadas, cliente segue adiante sem cortar cabelo.

Se todos os barbeiros estão ocupados, cliente espera.

Se não há clientes, barbeiro tira uma soneca, até que chega um novo cliente



O Barbeiro Dorminhoco: usando Semáforos

```
#define CHAIRS 5                /* # chairs for waiting customers */

typedef int semaphore;          /* use your imagination */

semaphore customers = 0;        /* # of customers waiting for service */
semaphore barbers = 0;         /* # of barbers waiting for customers */
semaphore mutex = 1;           /* for mutual exclusion */
int waiting = 0;               /* customers are waiting (not being cut) */
```

```
void barber(void)
{
    while (TRUE) {
        down(&customers);      /* go to sleep if # of customers is 0 */
        down(&mutex);           /* acquire access to 'waiting' */
        waiting = waiting - 1;  /* decrement count of waiting customers */
        up(&barbers);           /* one barber is now ready to cut hair */
        up(&mutex);             /* release 'waiting' */
        cut_hair();             /* cut hair (outside critical region) */
    }
}
```

⇐ Espera por clientes

⇐ avisando o barbeiro está pronto

```
void customer(void)
{
    down(&mutex);               /* enter critical region */
    if (waiting < CHAIRS) {     /* if there are no free chairs, leave */
        waiting = waiting + 1; /* increment count of waiting customers */
        up(&customers);        /* wake up barber if necessary */
        up(&mutex);            /* release access to 'waiting' */
        down(&barbers);        /* go to sleep if # of free barbers is 0 */
        get_haircut();         /* be seated and be serviced */
    } else {
        up(&mutex);             /* shop is full; do not wait */
    }
}
```

⇐ avisando que vc é um cliente

⇐ Espera por barbeiros

Barbeiro Dorminhoco usando Monitor

mon: monitor
sleeping_barber

Barber code:

while (true)

begin

mon.get_customer(); customers */

cut_hair();

end;

customer code:

mon.hair_cut();

Monitor:

type sleeping_barber = monitor

var number_of_customer: integer;

var x_barber: condition;

var x_customer: condition;

procedure get_customer()

begin

if (number_of_customer = 0)

x_barber.wait; /* barber has to wait for

number_of_customers:=number_of_customers - 1;

x_customer.signal; /* inform customer that

barber is ready */

end

procedure hair_cut()

begin

number_of_customer := number_of_customer + 1;

x_barber.signal; /* inform the barber that a

customer is in */

x_customer.wait; /* wait for the barber */

do_haircut();

end

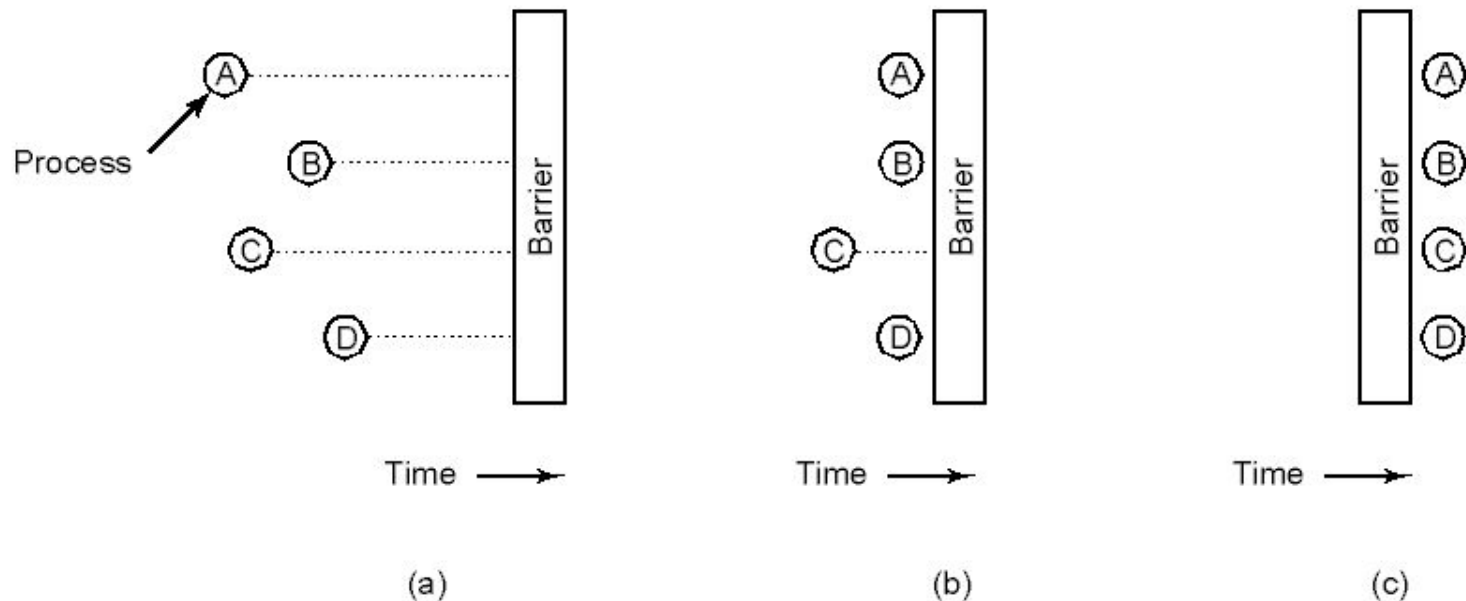
begin /*initialization*/

number_of_customer = 0;

end

end /* monitor */

Sincronização de Barreira



- Quando todos os processos precisam alcançar um mesmo estado, antes de prosseguir (exemplo: processamento paralelo “em rodadas” com troca de informações)
 - Processos progridem a taxas distintas
 - Todos que chegam a barreira, são bloqueados para esperar pelos demais
 - Quando o retardatário chega, todos são desbloqueados e podem prosseguir

Sincronização de Barreira

Sejam N processos que precisam esperar mutuamente na barreira.

```
Process {
    bool last = false;
    Semaphore barrier;
    Mutex m;
    Int count = N
    Init (&barrier, 0)

    down(&m)
    count--;
    if (count == 0) last= true;
    up(&m)
    if (NOT last) down (&barrier); // espera pelos demais processos
    else for (i=0; i< N; i++) up (&barrier); // desbloqueia todos
    ...
}
```

O Jantar dos Filósofos

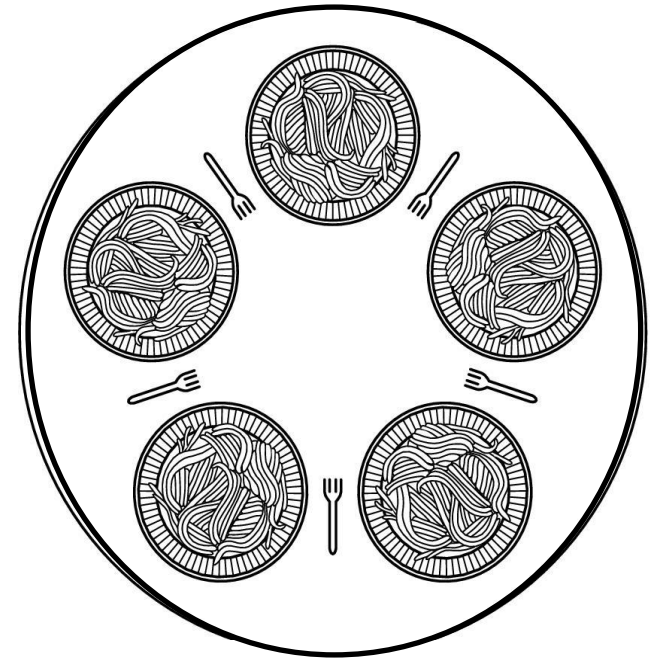
Sincronização para compartilhamento de recursos 2 a 2

Definição do Problema:

- Filósofos podem estar em um de 3 possíveis estados:
 - *com_fome*, *comendo* e *pensando*;
- Para comer, precisam de dois garfos, cada qual compartilhado com os seus vizinhos;
- Só é possível pegar um garfo por vez, isso é feito aleatoriamente.
- Assim que obtém um garfo, um filósofo não solta o mesmo

Questões globais:

- Como evitar um impasse (deadlock)?
- Como garantir que nenhum filósofo morra de fome? (starvation)



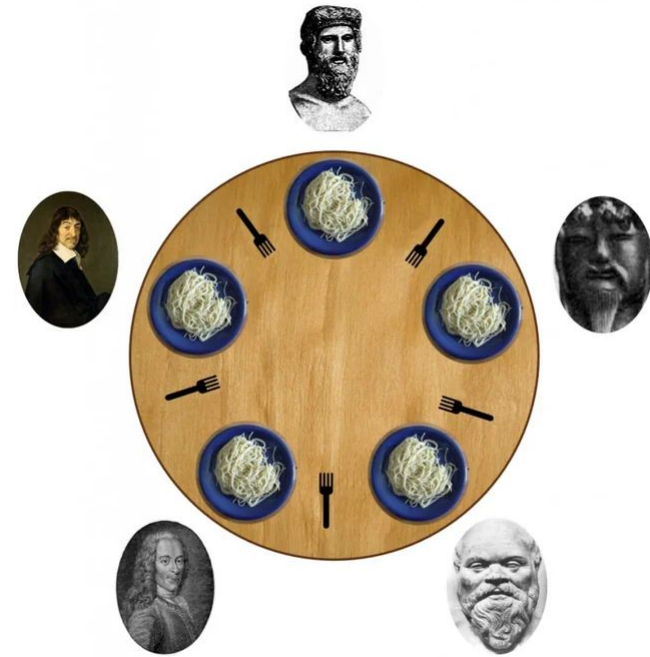
Esse problema é considerado um clássico de sincronização em S.O., mesmo não ocorrendo exatamente assim no mundo real.

Motivo: exemplifica muitas situações em que os processos precisam adquirir mais de um recurso ao mesmo tempo, recursos compartilhados com outros processos

O Jantar dos Filósofos

Uma solução trivial:

- Se há N recursos (garfos), deixar que no máximo $N-1$ filósofos tenham o direito de ter fome ao mesmo tempo.
 - Então, mesmo que filósofos vizinhos disputem um garfo, os filósofos das pontas sempre conseguirão comer, devolverão os garfos, e seus vizinhos conseguiram comer, etc.
 - Para isso, usar um semáforo contador, “mesa”, e inicializá-lo com o valor $N-1$ para limitar o numero de filosofos querendo comer
- O sistema fica livre de impasses



O Jantar dos Filósofos

Tentativa 1:

```
#define N 5                                /* number of philosophers */

void philosopher(int i)                    /* i: philosopher number, from 0 to 4 */
{
    while (TRUE) {
        think();                          /* philosopher is thinking */
        take_fork(i);                     /* take left fork */
        take_fork((i+1) % N);              /* take right fork; % is modulo operator */
        eat();                             /* yum-yum, spaghetti */
        put_fork(i);                       /* put left fork back on the table */
        put_fork((i+1) % N);               /* put right fork back on the table */
    }
}
```

Cada filósofo tenta pegar primeiro o garfo esquerdo, e depois o garfo direito.

Problema: Se todos pegarem o esquerdo ao mesmo tempo, teremos um impasse?

Tentativa 2: Aguarde até obter garfo esquerdo; Se garfo direito estiver disponível, ok, senão devolve também o garfo esquerdo e volta a esperar pelo garfo esquerdo.

Qual é o problema?

Jantar dos Filósofos

```
#define N          5          /* number of philosophers */
#define LEFT      (i+N-1)%N   /* number of i's left neighbor */
#define RIGHT     (i+1)%N     /* number of i's right neighbor */
#define THINKING  0          /* philosopher is thinking */
#define HUNGRY     1          /* philosopher is trying to get forks */
#define EATING     2          /* philosopher is eating */
typedef int semaphore;        /* semaphores are a special kind of int */
int state[N];                /* array to keep track of everyone's state */
semaphore mutex = 1;         /* mutual exclusion for critical regions */
semaphore s[N];              /* one semaphore per philosopher */

void philosopher(int i)      /* i: philosopher number, from 0 to N-1 */
{
    while (TRUE) {           /* repeat forever */
        think( );            /* philosopher is thinking */
        take_forks(i);        /* acquire two forks or block */
        eat( );               /* yum-yum, spaghetti */
        put_forks(i);         /* put both forks back on table */
    }
}
```

Solução (parte 1)

Jantar dos Filósofos

Solução (parte 2): usar um vetor `state[]` para verificar o estado dos vizinhos e dos vizinhos dos vizinhos.

```
void take_forks(int i)                /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                      /* enter critical region */
    state[i] = HUNGRY;                 /* record fact that philosopher i is hungry */
    test(i);                          /* try to acquire 2 forks */
    up(&mutex);                        /* exit critical region */
    down(&s[i]);                       /* block if forks were not acquired */
}

void put_forks(i)                     /* i: philosopher number, from 0 to N-1 */
{
    down(&mutex);                      /* enter critical region */
    state[i] = THINKING;              /* philosopher has finished eating */
    test(LEFT);                       /* see if left neighbor can now eat */
    test(RIGHT);                      /* see if right neighbor can now eat */
    up(&mutex);                        /* exit critical region */
}

void test(i)                          /* i: philosopher number, from 0 to N-1 */
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}
```

Jantar dos Filósofos: outras soluções baseadas em prioridades

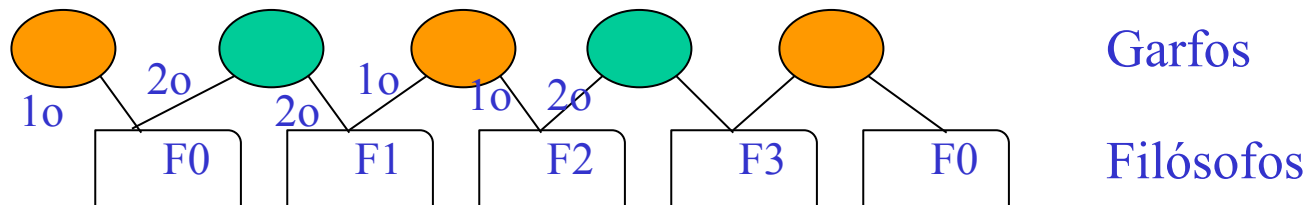
Solução de Dijkstra:

- Estabelecer uma ordem de prioridade dos garfos ($G_1, G_2, G_3, \dots, G_N$) e impor que os filósofos sempre peguem primeiro o garfo com menor ID – assim quebra-se a dependência circular! E garante-se que pelo menos um filósofo será capaz de comer.

Jantar dos Filósofos: Outra Solução

Ideia Central: Diferenciar o comportamento de filósofos pares e ímpares:

- os filósofos pares tentam pegar primeiro o garfo esquerdo, e depois o direito
- os filósofos ímpares tentam pegar primeiro o garfo direito e depois o esquerdo
- assim, os garfos (à direita do filósofo par) serão sempre os primeiros a serem tentados e nunca dois filósofos vizinhos diretos entrariam em um impasse



Exemplo:

F0 par obtém garfo à sua esquerda

F1 ímpar obtém garfo à sua direita

ou F0 ou F1 obtém o seu segundo garfo

.... (isso vale para todos os vizinhos)

Jantar dos Filósofos: Outras Soluções

A **solução do garçom** oferece uma maneira simples de resolver o problema do Jantar dos Filósofos, pressupondo uma entidade externa chamada *garçom*.

- Estratégia:
 - Cada filósofo deve solicitar cada um dos 2 garfos (compartilhados) a um garçom, que pode recusar a solicitação para evitar um impasse.
 - Por conveniência, presumimos que todos os filósofos solicitem primeiro o garfo esquerdo e depois o garfo direito.
 - O garçom sempre fornece os garfos quando solicitado , *a menos que* apenas um garfo não seja utilizado. Nesse caso, o garçom atende ao pedido somente se for solicitado um garfo *direito*; os pedidos de garfo esquerdo são *adiados* até que outro filósofo termine de comer.
 -
- Não haverá um *impasse*: O último garfo só será atribuído se o garçom tiver certeza de que pelo menos um filósofo pode terminar de comer (liberando os seus 2 garfos). Portanto, a "espera circular" necessária para o deadlock não pode ocorrer.
- Desvantagens: Escalabilidade (o garçom pode se tornar um gargalo se o número de processadores for grande).

Envio de Mensagens

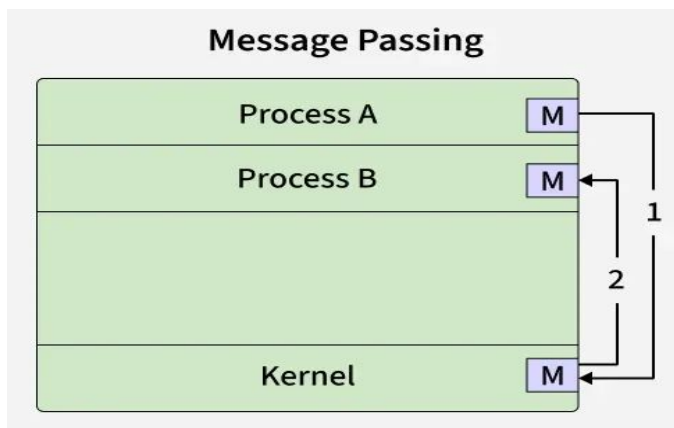
- É uma forma natural de interação entre processos
- Duas primitivas:
 - **send**(canal, &message) – pode ser bloqueante ou não-bloqueante
 - **receive**(canal, &message) – pode ser bloqueante ou não-bloqueante
- Processos não precisam indicar o processo da outra ponta
 - definem um canal lógico de comunicação
 - mensagens podem ser tipadas ou não
- O canal pode ser:
 - Entre processos co-localizados ou distribuídos
 - confiável ou não-confiável e talvez em ordem FIFO
 - Ponto-a-ponto ou Ponto-a-multiponto
 - ter uma capacidade máxima ou ser um *stream*

Envio de Mensagens

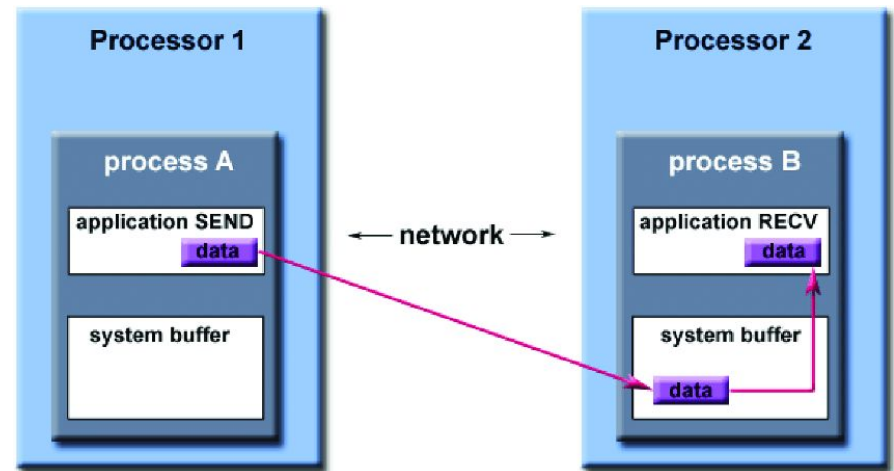
- Ao contrário de mecanismos de sincronização, envio de mensagens é um mecanismo de sincronização mais sofisticado, pois:
 - Permite também a troca/transmissão de dados
 - processos podem estar executando na mesma máquina ou em máquinas distintas
 - Qualquer solução de concorrência usando semáforos pode ser resolvida por envio de mensagens
 - considere: `down()` $\equiv$ `receive()` e `up()` $\equiv$ `send()`, e o valor do semáforo sendo o número de mensagens
- Decisões de projeto do mecanismo:
 - `receive()` sempre bloqueia se o outro processo não tiver executado o `send()`
 - Mas `send()` pode ser **assíncrono** (com buffer de mensagens) ou **síncrono** (*Rendezvous*)
 - Em comunicação do tipo *Rendezvous* podem ocorrer impasses
 - Comunicação nem sempre é confiável: quando através da rede, mensagens podem ser perdidas: □ são necessárias confirmações (acks) e re-transmissões

Envio de Mensagens

- Pipes, e Named Pipes (FIFO)
- Message Queue
- Remote Procedure/Method Call (RPC)
- Sockets
 - Connection oriented
 - Datagram Oriented
- Request-Reply

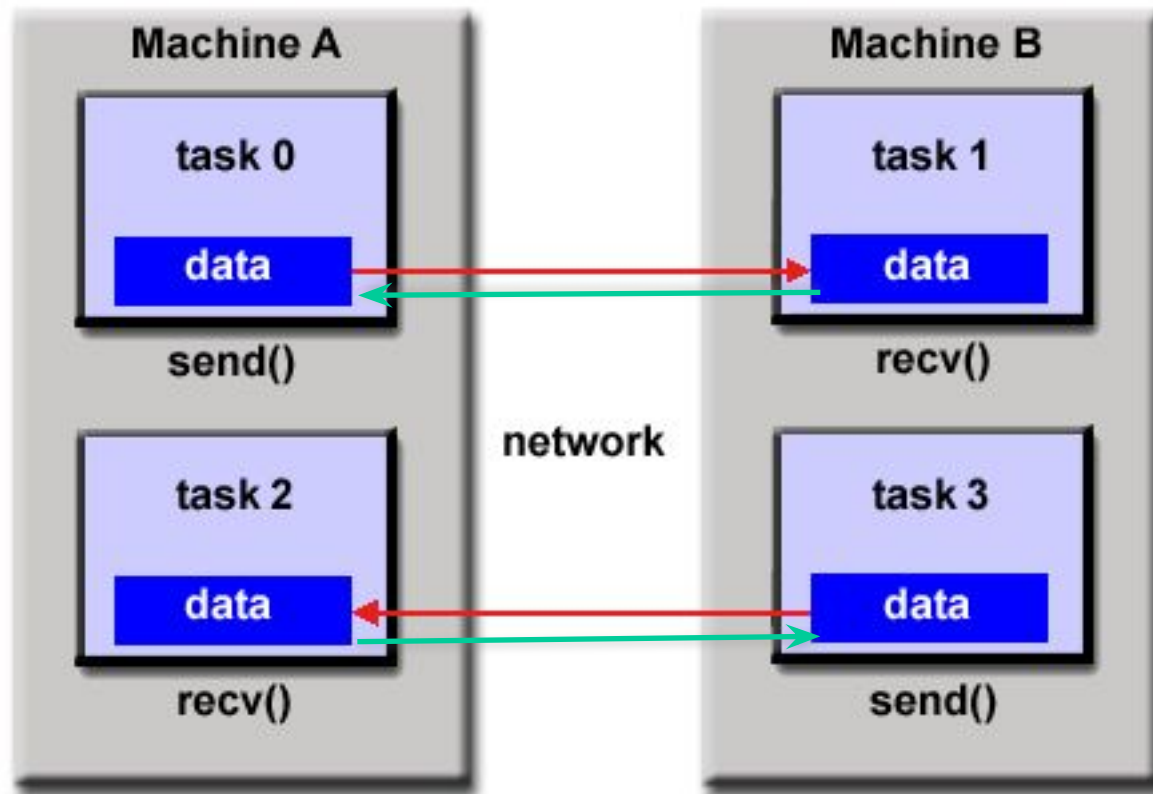


local



distribuida

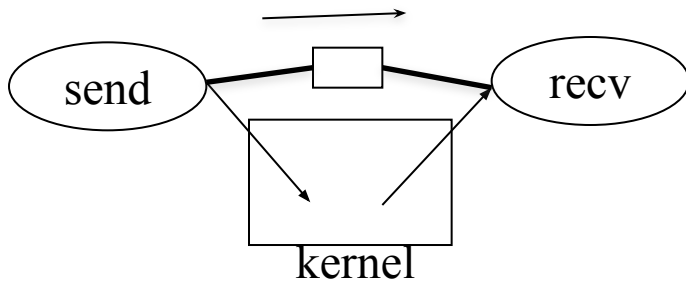
Envio de Mensagens



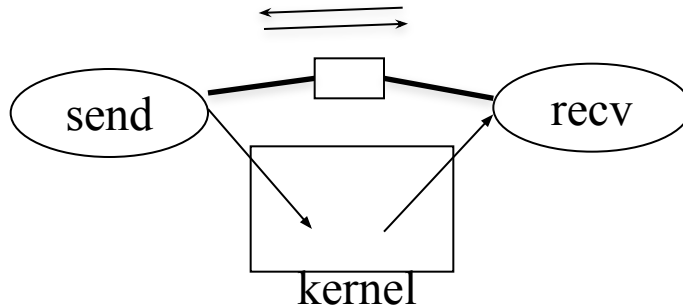
Tipos de Envios de Mensagem

Entre processos co-localizados

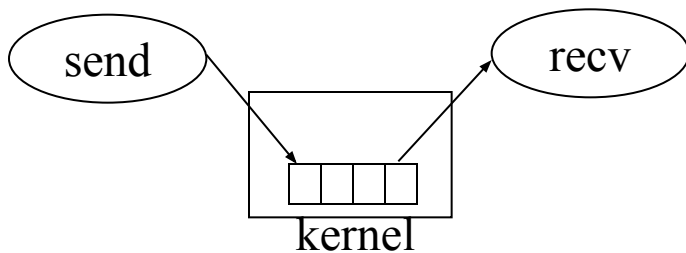
Rendezvous, unidirecional



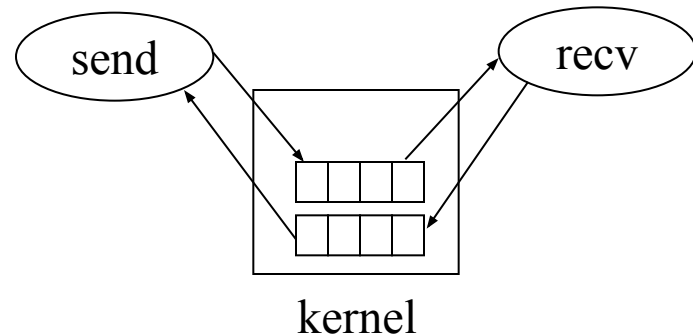
*Rendezvous, bidirecional
(Request-Reply)*



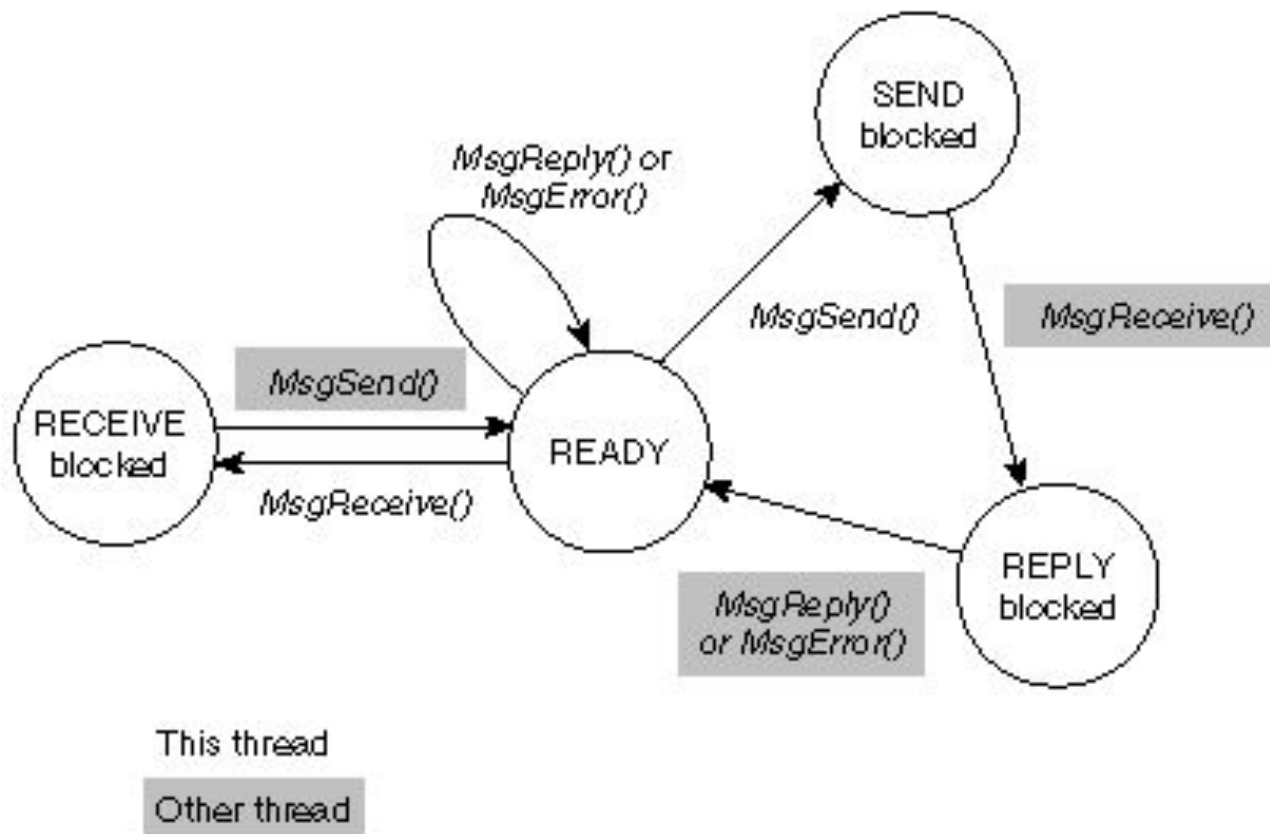
*Comunicação assíncrona,
unidirecional*



*Comunicação bidirecional,
(Request-Reply) assíncrono*



Request-Reply Síncrono: possíveis estados dos processos/threads



Problema do Produtor-Consumidor

com envio de N Mensagens

Idéia: consumidor envia mensagem vazia, e produtor responde com a mensagem preenchida.

```
#define N 100                                /* number of slots in the buffer */

void producer(void)
{
    int item;
    message m;                                /* message buffer */

    while (TRUE) {
        item = produce_item();                /* generate something to put in buffer */
        receive(consumer, &m);                /* wait for an empty to arrive */
        build_message(&m, item);              /* construct a message to send */
        send(consumer, &m);                    /* send item to consumer */
    }
}

void consumer(void)
{
    int item, i;
    message m;

    for (i = 0; i < N; i++) send(producer, &m); /* send N empties */
    while (TRUE) {
        receive(producer, &m);                /* get message containing item */
        item = extract_item(&m);              /* extract item from message */
        send(producer, &m);                    /* send back empty reply */
        consume_item(item);                    /* do something with the item */
    }
}
```

Problema do Produtor-Consumidor com envio de N Mensagens

Idéia: consumidor envia mensagem vazia, e produtor responde com a mensagem preenchida.

```
#define N 100                                /* number of slots in the buffer */

void producer(void)
{
    int item;
    message m;                               /* message buffer */

    while (TRUE) {
        item = produce_item();               /* generate something to put in buffer */
        receive(consumer, &m);               /* wait for an empty to arrive */
        build_message(&m, item);             /* construct a message to send */
        send(consumer, &m);                  /* send item to consumer */
    }
}

void consumer(void)
{
    int item, i;
    message m;

    for (i = 0; i < N; i++) send(producer, &m); /* send N empties */
    while (TRUE) {
        receive(producer, &m);               /* get message containing item */
        item = extract_item(&m);             /* extract item from message */
        send(producer, &m);                  /* send back empty reply */
        consume_item(item);                  /* do something with the item */
    }
}
```

IPC básico no Unix

- Sinais – notificações assíncronas entre processos de um mesmo usuário (e do núcleo para processos)
 - Ação default: terminar o processo
 - Processo pode definir um tratador de sinal, que reage independentemente à execução corrente do processo sinalizado
 - Limitações: execução tem alto overhead, e sinais não permitem transferir dados
- Pipes: fluxo de dados unidirecional, entre processos quaisquer (no mesmo host)
 - Fluxo First-in-First_out (FIFO), não tipado (fluxo de bytes),
 - Ao criar uma pipe, são retornados um descritor de arquivo para escrita e outro para leitura (usado com `read()` e `write()`)

Problema do Produtor-Consumidor com envio de N mensagens

Idéia: Para a sincronização de condição, o consumidor envia mensagem vazia, e produtor responde com a mensagem preenchida.

```
#define N 100                                /* number of slots in the buffer */

void producer(void)
{
    int item;
    message m;                               /* message buffer */

    while (TRUE) {
        item = produce_item();               /* generate something to put in buffer */
        receive(consumer, &m);               /* wait for an empty to arrive */
        build_message(&m, item);             /* construct a message to send */
        send(consumer, &m);                  /* send item to consumer */
    }
}

void consumer(void)
{
    int item, i;
    message m;

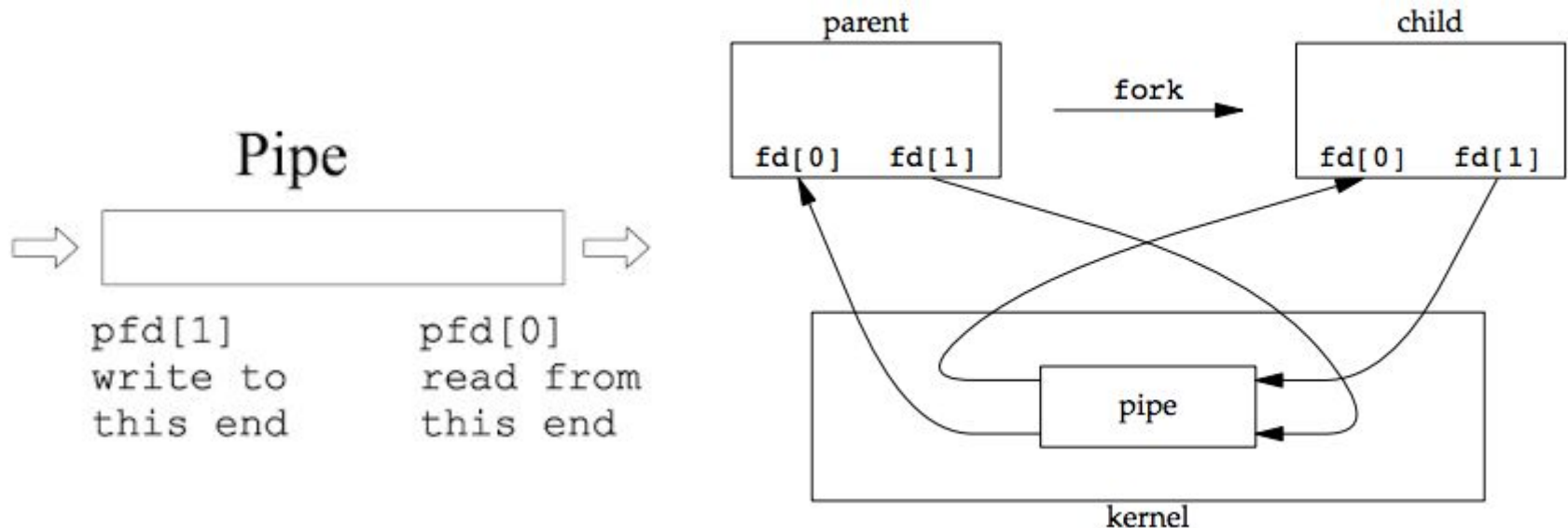
    for (i = 0; i < N; i++) send(producer, &m); /* send N empties */
    while (TRUE) {
        receive(producer, &m);               /* get message containing item */
        item = extract_item(&m);             /* extract item from message */
        send(producer, &m);                 /* send back empty reply */
        consume_item(item);                  /* do something with the item */
    }
}
```

Mecanismos de Comunicação em Unix

- Pipes/ FIFO
- Message Queues
- Sockets

Pipe

- Pipe é uma conexão de software temporária entre dois processos ligando o STDOUT de um processo com o STDIN do outro processo.
- Em Unix, pipes podem funcionar de forma bidirecional, o que significa que permite direcionar o fluxo de dados para qualquer um dos "processos conectados".
- Além disso, são anônimos no sentido que os descritores serão temporários e existirão apenas enquanto a pipe permanecer criada.



Exemplo de comunicação de pai para filho

```
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <sys/wait.h>
#include <stdlib.h>

const char* process = NULL;
void read_from_fd(int fd) {
    char c;
    printf ("Process [%s] Reading from\n", process, fd);
    while(read(fd, &c, 1) != 0) {
        printf("%c", c);
    }
}

int main(int argc, char** argv) {
    process = "PARENT";
    int fd[2];
    if (pipe(fd) != 0) {
        printf ("Error creating pipe. %s", strerror(errno));
        exit(errno);
    }
    int pid = fork();
    if (pid == -1) {
        printf ("Error forking. %s", strerror(errno));
        exit(errno);
    } else if (pid == 0) {
        //child
        process = "CHILD";
        /* Close the write end of the pipe in child process.*/
        close(fd[1]);
        read_from_fd(fd[0]);
        exit(0);
    } else {
        /* Close the read end of the pipe in parent process.*/
        close(fd[0]);
        const char* data = "Data";
        printf("Process [%s] Writing data to pipe\n", process);
        write(fd[1], data, strlen(data));
        /* After writing all data, close the write end */
        close(fd[1]);
        /* Wait for the child to finish */
        waitpid(pid, 0, 0);
        exit(0);
    }
}
```

```
#>./a.out
```

```
Process [PARENT] Writing data to pipe
```

```
Process [CHILD] Reading from descriptor
```

```
3
```

```
Data
```

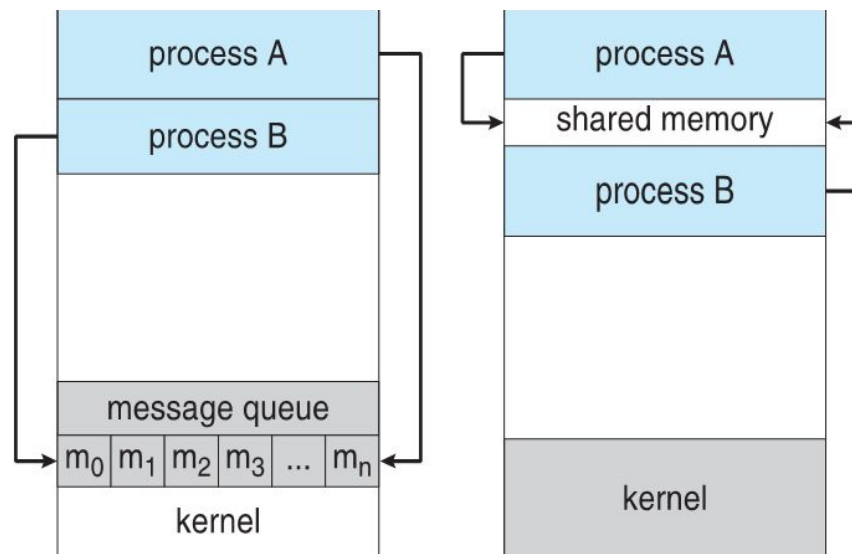
FIFO (Named Pipe)

- Similar ao pipe, só que persiste além de seu tempo de uso pelos processos que o criaram.
- Como se fossem um arquivo no diretório corrente do sistema de arquivos.
- Pode ser usado por quaisquer dois processos
- Para criação:
 - `mknod`,
 - `mknod("/tmp/mypipe", S_IFIFO, 0)`
- Abre-se e fecha-se como se fosse um arquivo para leitura e/ou escrita
 - `fd = open ("/tmp/mypipe", 0)`
- Usa as permissões `rwX` de arquivos para o controle de acesso - qualquer processo que saiba o seu nome e tenha autorização de acesso pode acessá-lo para enviar ou receber dados.
- Usa `unlink` para remover o named pipe

Message Queue

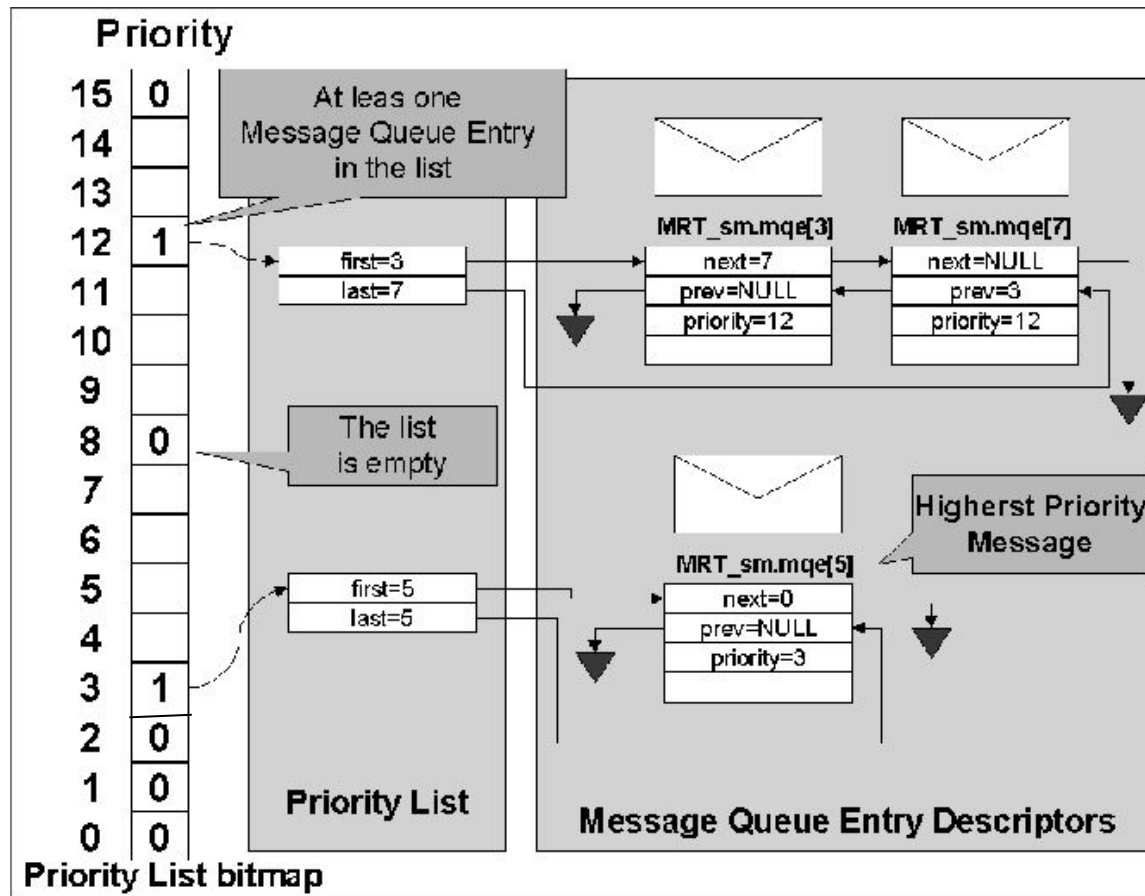
- É uma fila de mensagens baseada em prioridades.
- Cada mensagem é simplesmente um pequeno bloco de memória que contém uma mensagem definida pela aplicação.
- Quando uma mensagem é enviada para a Message Queue, ela é armazenada na memória do kernel para que possa ser posteriormente lida pelo processo destinatário.
- Cada mensagem enfileirada tem uma prioridade.

Diferença entre Msg Queue e Shared Memory



Message Queue: Priority list bitmap

Todas as mensagens de uma mesma prioridade, em uma lista duplamente encadeada.



Message Queue

- Adota o paradigma de mailbox: mensagens podem ter qualquer tamanho.
 - msqID (chave que identifica a message queue)
- Cada mensagem tem os seguintes campos:
 - Type
 - Length
 - Data // a mensagem propriamente dita
- todos eles especificados em `msgsnd()` quando a mensagem é adicionada a message queue.
- `msqID` e `Type` são usados para identificar univocamente a mensagem (que pode ser consumida em qualquer ordem FCFS)

Outras operações

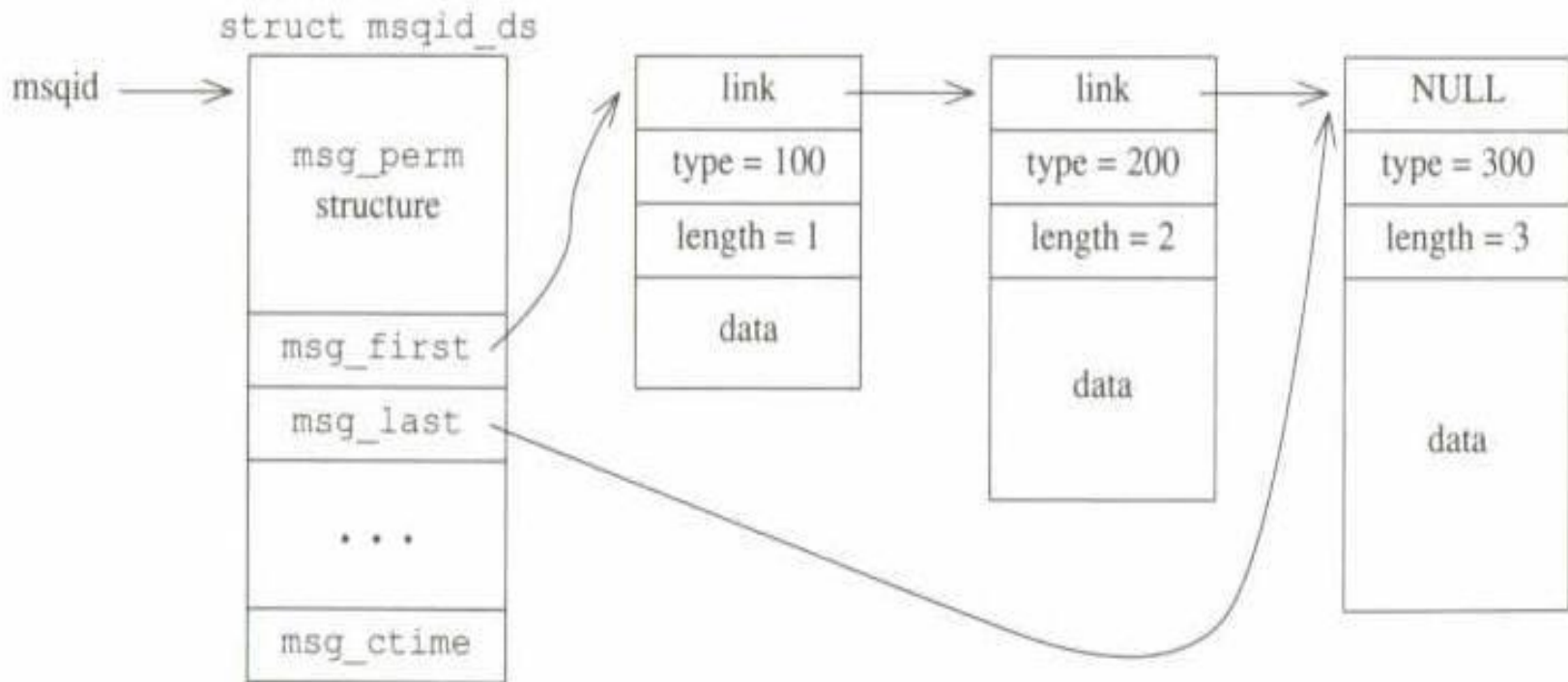
- `msgget()` – para criar uma msg queue
- `msgrcv()` – para receber uma msg
- `msgctl()` – para remover uma message queue

Message Queue: Chamadas de Sistema

- System calls:
 - `int msgget (key_t key, int flag)` - cria ou acessa uma a message queue e retorna o message queue identifier
 - `int msgsnd(int msqid, const void *msgp, size_t msgsz, int flag)` adiciona a mensagem msgp na message queue
 - `int msgrcv(int msqid, void *msgp, size_t msgsz, long msgtype, int msgflg)` recebe uma mensagem e armazena ela para msgp
 - **msgtype**: mensagens podem ser tipadas e cada tipo define um canal de comunicação
 - `int msgctl(int msqid, int cmd, struct msqid_ds *buf)` provê uma série de operações de controle sobre message queues (exemplo: remoção da mesma).
- Processos são bloqueados quando:
 - tentam ler de uma queue vazia
 - Tentam escrever em uma queue cheia
- Processos podem se restringir a receber mensagens apenas de certo tipo **msgtype** (portanto. consumo pode ser diferente de FCFS)

Estrutura da Message Queue

Estrutura mantida no núcleo:



Exemplo de Message Queue

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#define BUFSIZE 128
#define PERMS      0666
#define KEY ((key_t) 7777)
main() {
    int i, msqid;
    struct {
        long m_type;
        char m_text[BUFSIZE];
    } msgbufs, msgbuf;

    if ((msqid = msgget(KEY, PERMS | IPC_CREAT)) < 0) error("msgget perror");
    msgbufs.m_type = 1L;
    strcpy(msgbufs.m_text, "a REALLY boring message");
    if (msgsnd(msqid, &msgbufs, BUFSIZE, 0) < 0) perror("msgsnd error");
    printf("the message sent is: %s \n", msgbufs.m_text);
    if (msgrcv(msqid, &msgbuf, BUFSIZE, 0L, 0) != BUFSIZE) perror("msgrcv err");
    printf("the message received is: %s \n", msgbuf.m_text);
    // remove msg
    if (msgctl(msqid, IPC_RMID, (struct msqid_ds *)0) < 0) perror("IPC_RMID err");
    exit(0);
}
```

Sockets

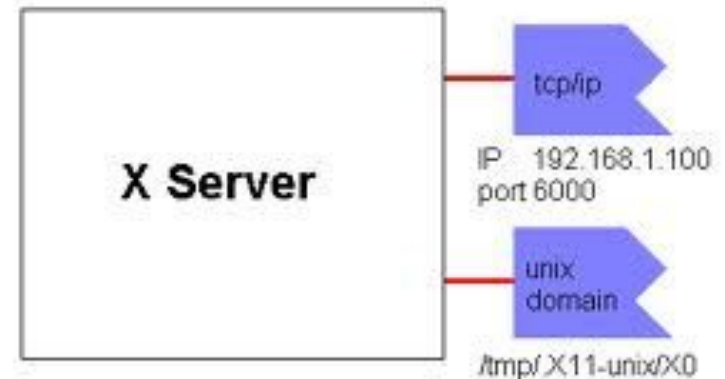
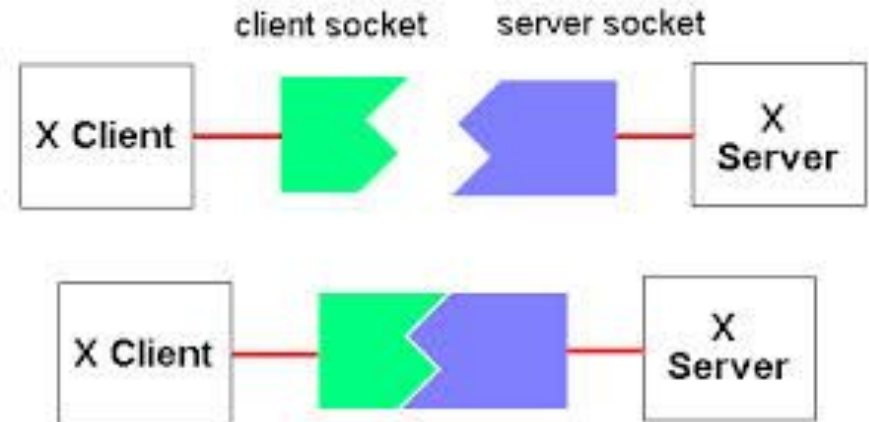
Um socket é um *ponto de comunicação bi-direcional* de um processo:

É um objeto lógico através do qual mensagens podem ser enviadas e recebidas.

Processo recebe um descritor de socket (similar a um file descriptor)

Principal Vantagem:

- não precisar conhecer o outro processo
- É genérico para diferentes domínios e protocolos de comunicação (
 - Internet domain (TCP/UDP),
 - Unix domain,



O que é um socket?

Socket é uma interface de software, uma abstração de um *ponto final de comunicação provida pelo kernel*.

- Pode ser usado como um descritor de arquivo.
- Através de um socket, mensagens podem ser enviadas e recebidas usando um protocolo de rede
- É uma interface independente do protocolo de comunicação inter-processos.
- Sockets são criados para um domínio de comunicação,
- **Domínio de comunicação** é a especificação de propriedades comuns da comunicação entre processos (p.ex. o formato dos endereços)
- Exemplos de domínio:
 - UNIX domain (para comunicação local),
 - Internet domain.

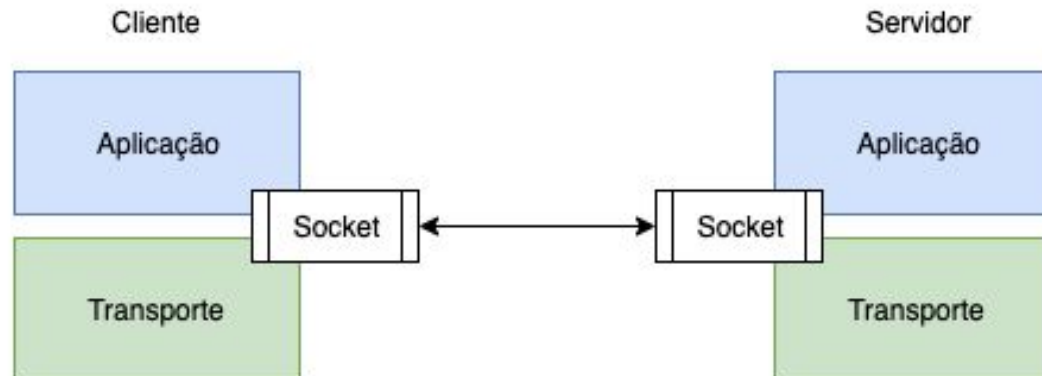
O que é um socket?

Socket provê a comunicação entre *duas pontas* (fonte e destino) entre dois processos que estejam na mesma máquina (*Unix Socket*) ou na rede (*TCP/IP Sockets*).

Na rede, a representação de um socket se dá por `ip:porta`, por exemplo:

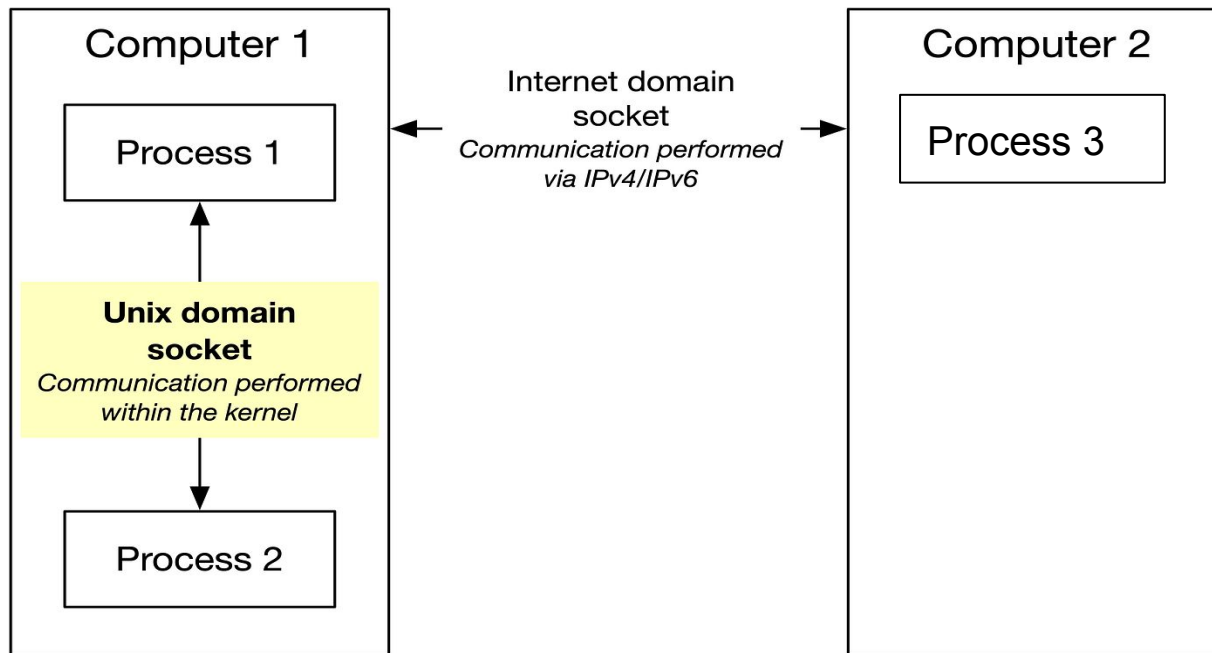
`127.0.0.1:4477` (IPv4).

Um socket se comporta como um arquivo, chamadas de sistema de leitura e escrita são aplicáveis, da mesma forma como funcionam em um arquivo comum.



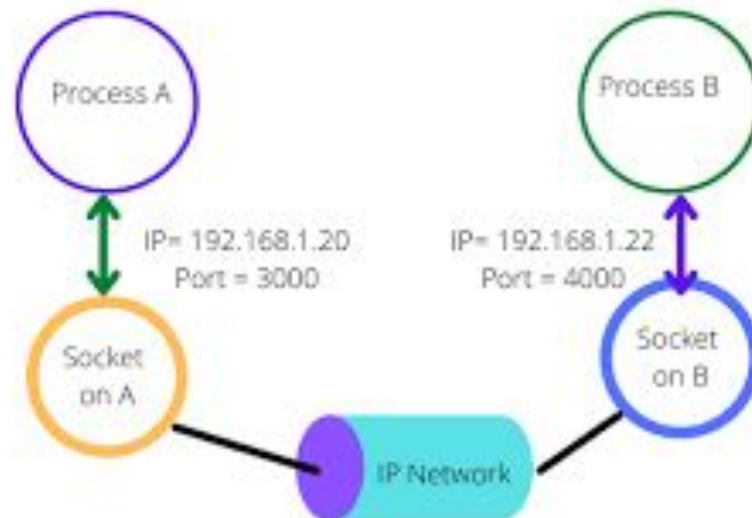
Socket: Domínios de Comunicação

A diferença entre um Unix Socket (*IPC Socket*) de um TCP/IP Socket é que o primeiro só permite a comunicação entre processos que estão na mesma máquina. TCP/IP Socket também permite a comunicação entre processos através da rede.

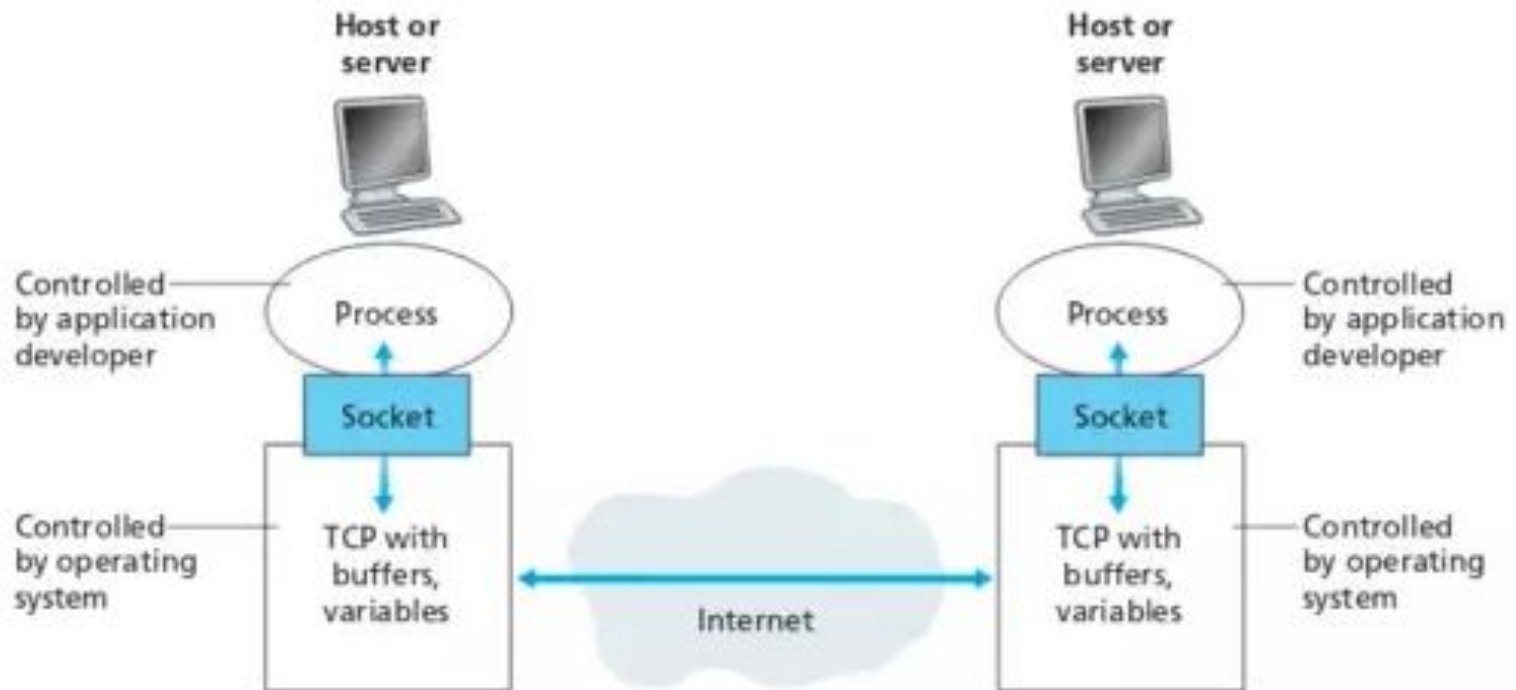


O que é um socket?

- Um socket de rede é uma estrutura de software em um nó da rede que serve de ponto final para enviar e receber dados através da rede.
- A estrutura e as propriedades de um socket são definidos por uma (API) para a arquitetura de rede.
- A interface de programação de aplicações (API) para a pilha de protocolos de rede cria um identificador para cada socket criado, um *descriptor de socket*.
- Este descriptor é um tipo de descriptor de arquivo. ele é mantido pelo processo de aplicação para as operações de leitura e escrita no canal de comunicação.



Sockets



Dois Tipos de Internet Sockets

- Stream socket
 - Orientado à conexão
 - Comunicação bidirecional
 - Confiável (sem perdas), entrega de mensagens na ordem, e com controle de fluxo
 - Tipicamente usa o Transmission Control Protocol (TCP)
 - Exemplos de Serviços: ftp, telnet, ssh, http
- Datagram socket
 - Orientado a pacotes, não mantém uma conexão aberta,
 - cada pacote (data-grama) é transmitido individualmente
 - Tipicamente, usa User Datagram Protocol (UDP)
 - Exemplos: telefonia IP (skype, etc.), wireless

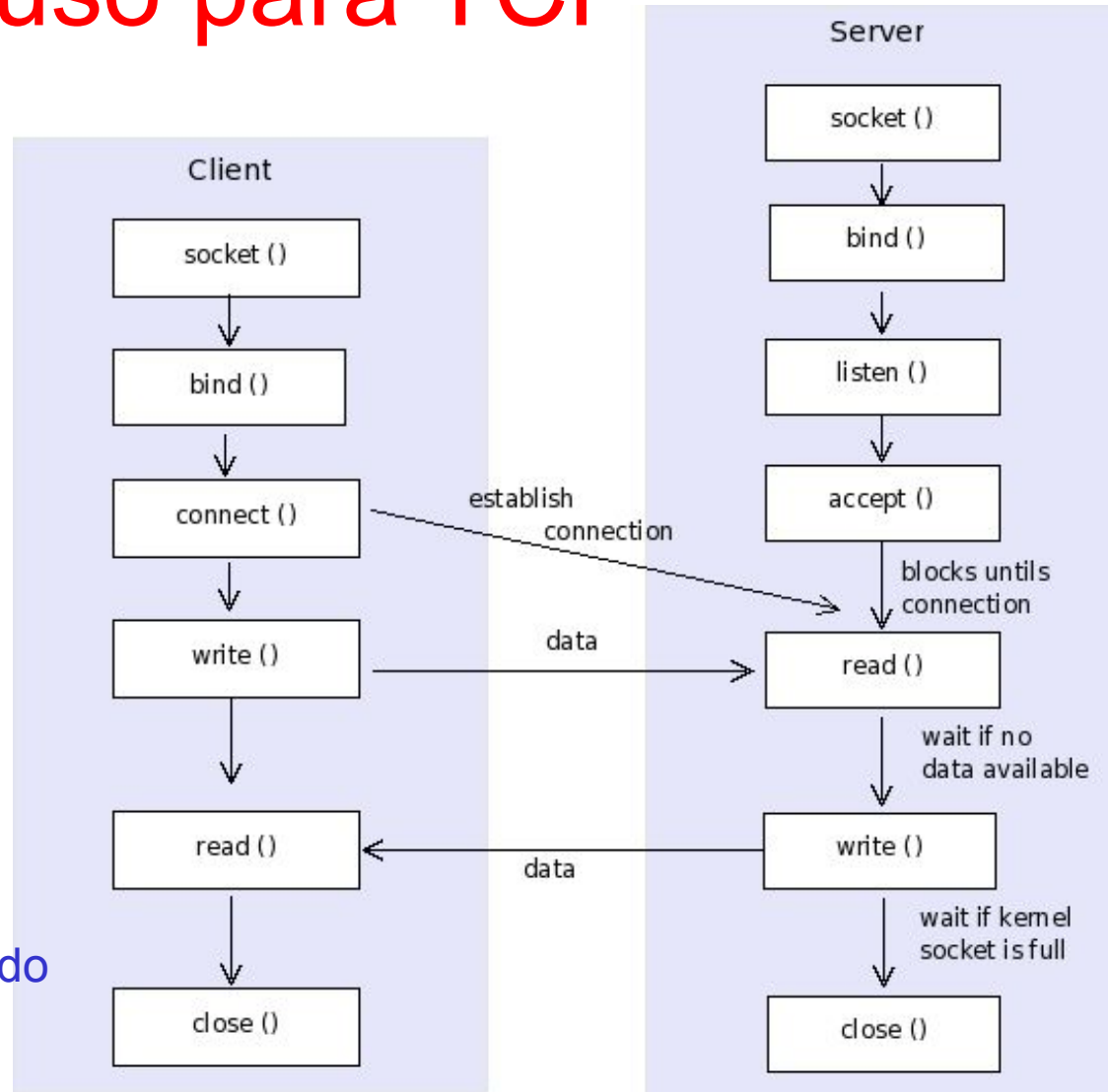
Sockets – uso para TCP

bind() atribui um endereço IP ao socket

- Nome depende do tipo de protocolo (address families):
- AF_INET
- AF_INET6
- AF_APPLETALK
- ...

connect() faz uma ligação com endereço (host_ID: porta) do servidor

listen() informa que o socket passado como argumento é passivo, ou seja, apenas é usado para aceitar novas conexões



accept() processa o próximo pedido de conexão e cria um novo socket conectado e retorna o descritor desse socket criado

Cliente TCP

- Obtendo o descritor do socket

```
soc= socket(AF_INET, SOCK_STREAM, 0);
```

- Setando o endereço (do servidor, no cliente)

```
struct sockaddr_in sin;  
struct hostent *host = gethostbyname (argv[1]);  
unsigned int server_address = *(unsigned long *) host->h_addr_list[0];  
unsigned short server_port = atoi (argv[2]);
```

```
memset (&sin, 0, sizeof (sin));  
sin.sin_family = AF_INET;  
sin.sin_addr.s_addr = server_address;  
sin.sin_port = htons (server_port);
```

No Cliente TCP (cont.)

- Fazendo a conexão

```
if (connect(chat_sock, (struct sockaddr *) &sin, sizeof (sin)) < 0) {  
    perror("connect");  
    printf("Cannot connect to server\n");  
    abort();  
}
```

- Enviando os dados pelo descritor

```
int send_packets(char *buffer, int buffer_len) {  
    sent_bytes = send(chat_sock, buffer, buffer_len, 0);  
    if (sent_bytes < 0) {  
        perror ("send");  
    }  
    return 0;  
}
```

Exemplo cliente

```
hSocket=socket(AF_INET,SOCK_STREAM,IPPROTO_TCP);

if(hSocket == SOCKET_ERROR)
{
    printf("\nCould not make a socket\n");
    return 0;
}

/* get IP address from name */
pHostInfo=gethostbyname(strHostName);
/* copy address into long */
memcpy(&nHostAddress,pHostInfo->h_addr,pHostInfo->h_length);

/* fill address struct */
Address.sin_addr.s_addr=nHostAddress;
Address.sin_port=htons(nHostPort);
Address.sin_family=AF_INET;

/* connect to host */
if(connect(hSocket,(struct sockaddr*)&Address,sizeof
(Address))
== SOCKET_ERROR)
{
    printf("\nCould not connect to host\n");
    return 0;
}

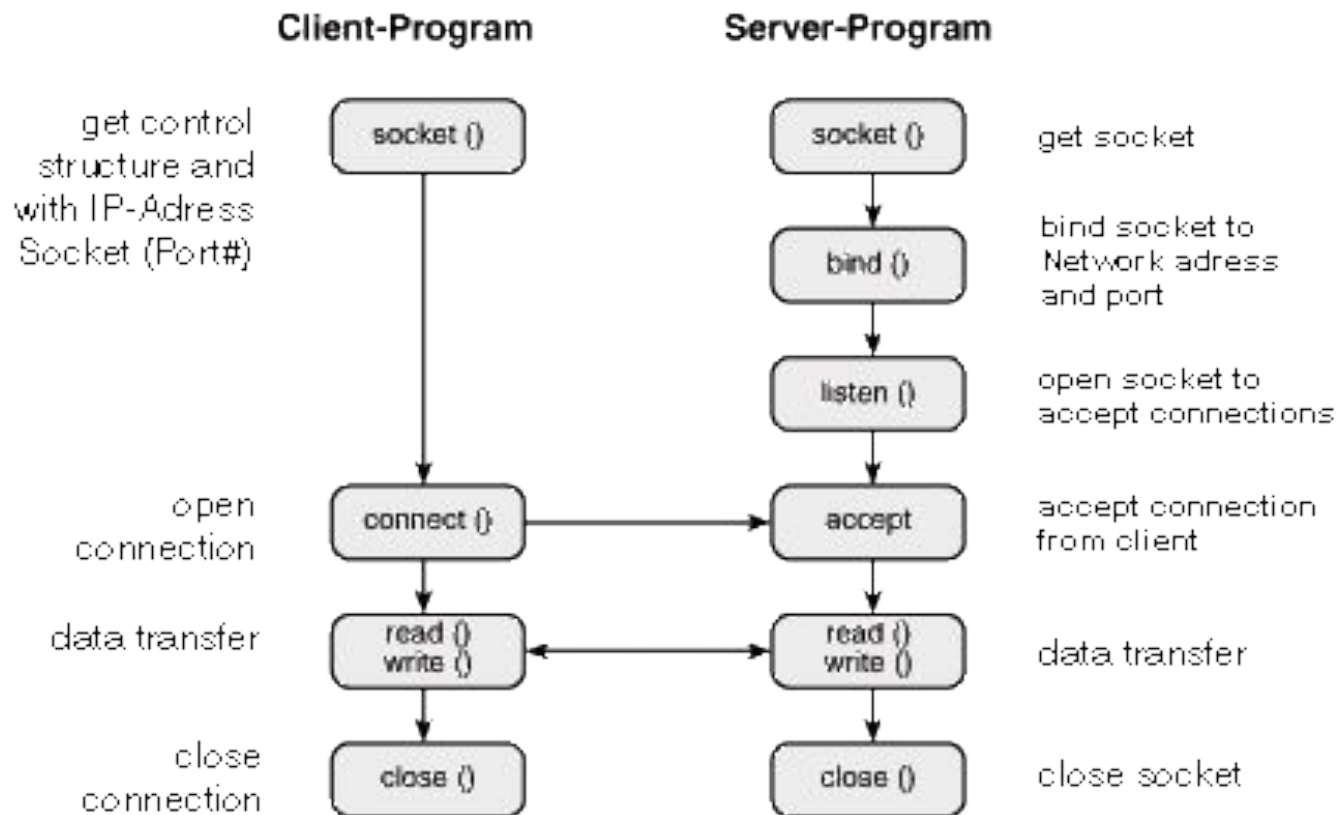
nReadAmount=read(hSocket,pBuffer,BUFFER_SIZE);
write(hSocket,pBuffer,nReadAmount);
```

No Servidor TCP

- Cria stream socket: *socket()*
- Atribui porta ao socket: *bind()*
- Espera por conexão de cliente: *listen()*
- *while(1)*
 - Aceita conexão de cliente e cria novo socket Nsock: *accept()*
 - Recebe dado do cliente: *recv(Nsock, ...)*
 - Envia resposta para client: *send(Nsock, ...)*

Sockets com TCP:

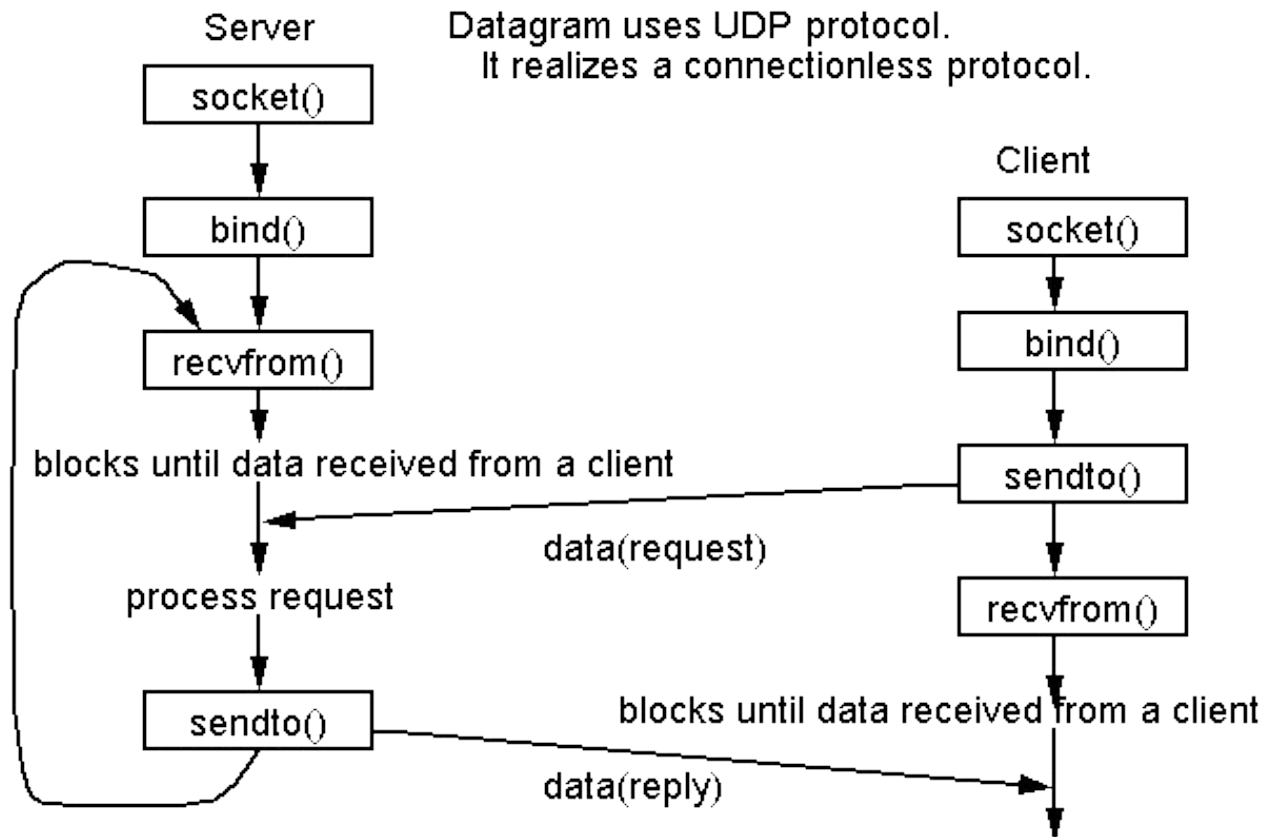
Interação Cliente-Servidor



Sockets - uso com UDP

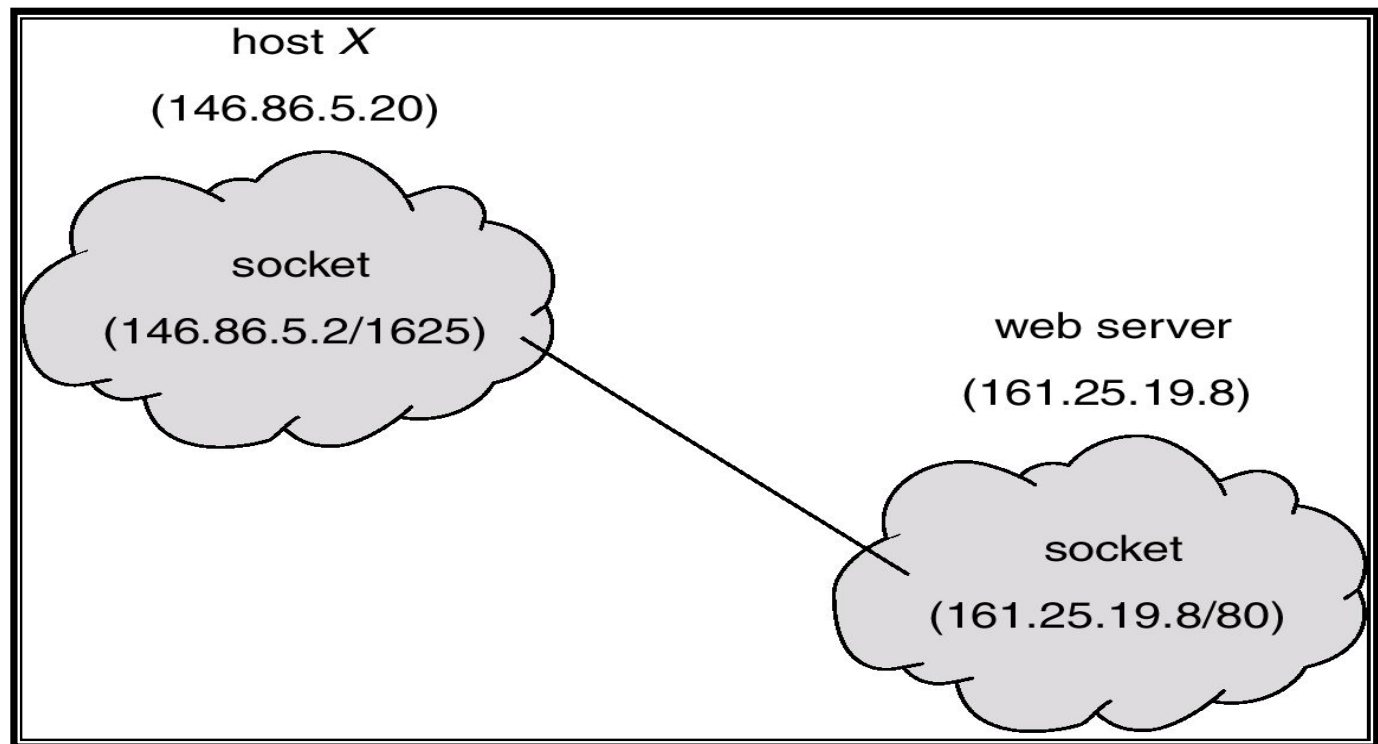
A comunicação é **orientada a datagrama**, sem que se crie uma conexão lógica entre os processos. Nesse caso, cada pacote é enviado e recebido separadamente.

O controle de fluxo, retransmissão e ordenação fica a cargo da aplicação.



Sockets e IP

- Concatenação de endereço IP address e porta: **161.25.19.8:1625** se refere a porta **1625** no host **161.25.19.8**
- **Localhost:** 127.0.0.1
- Uma conexão TCP consiste de um par de sockets



Portas reservadas para serviços Internet

Keyword	Number	Protocol(s)	Description
tcpmux	1	TCP, UDP	TCP Port Service Multiplexer
echo	7	TCP, UDP	Echo
discard	9	TCP, UDP	Discard
systat	11	TCP	Active Users
daytime	13	TCP, UDP	Daytime (RFC 867)
qotd	17	TCP	Quote of the Day
msp	18	TCP, UDP	message send protocol
chargen	19	TCP, UDP	Character Generator
ftp-data	20	TCP, UDP	File transfer default data
ftp	21	TCP, UDP	File transfer control
ssh	22	TCP, UDP	Remote login protocol
telnet	23	TCP, UDP	Telnet
smtp	25	TCP, UDP	Simple Mail Transfer
time	37	TCP, UDP	Time
rlp	39	TCP, UDP	Resource location protocol
nameserver	42	TCP, UDP	Host name server
whois	43	TCP, UDP	Who is
re-mail-ck	50	TCP, UDP	Remote mail checking protocol
domain	53	TCP, UDP	Domain name server
bootps	67	TCP, UDP	Bootstrap protocol server
bootpc	68	TCP, UDP	Bootstrap protocol client
tftp	69	TCP, UDP	Trivial file transfer protocol
gopher	70	TCP, UDP	Gopher
finger	79	TCP, UDP	Finger
www	80	TCP, UDP	World wide web or HTTP

kerberos	88	TCP, UDP	Kerberos
supdup	95	TCP, UDP	SUPDUP
hostname	101	TCP, UDP	NIC Host Name Server
iso-tsap	102	TCP, UDP	ISO-TSAP Class 0
csnet-ns	105	TCP, UDP	CCSO name server protocol
rtnet	107	TCP, UDP	Remote Telnet Service
pop-2	109	TCP, UDP	Post Office Protocol - Version 2
pop-3	110	TCP, UDP	Post Office Protocol - Version 3
sunrps	111	TCP, UDP	SUN Remote Procedure Call
auth	113	TCP, UDP	Authentication Service
sftp	115	TCP, UDP	Simple File Transfer Protocol
uucp-path	117	TCP, UDP	UUCP Path Service
nntp	119	TCP, UDP	Network News Transfer Protocol
nyp	123	TCP, UDP	Network Time Protocol
netbios-ne	137	TCP, UDP	NETBIOS Name Service
netbios-dgram	138	TCP, UDP	NETBIOS Datagram Service
netbios-ssn	139	TCP, UDP	NETBIOS Session Service
imap	143	TCP, UDP	Internet Message Access Protocol
snmp	161	TCP, UDP	SNMP
snmp-trap	162	TCP, UDP	SNMPTRAP
cmip-man	163	TCP, UDP	CMIP/TCP Manager
cmip-agent	164	TCP, UDP	CMIP/TCP Agent
xdmcp	177	TCP, UDP	X Display Manager Control Protocol
nextstep	178	TCP, UDP	NextStep Window Server
bgp	179	TCP, UDP	Border Gateway Protocol
prospero	191	TCP, UDP	Prospero Directory Service
irc	194	TCP, UDP	Internet Relay Chat Protocol
smux	199	TCP, UDP	SMUX

Outros Mecanismos de Comunicação

Mecanismos (interfaces) mais abstratos implementados usando sockets como base.

1. Chamada Remota de Procedimento/Métodos (RPC, RMI)
2. Message Passing Interface (MPI)
3. Publish/Subscribe

Chamada Remota de Procedimento

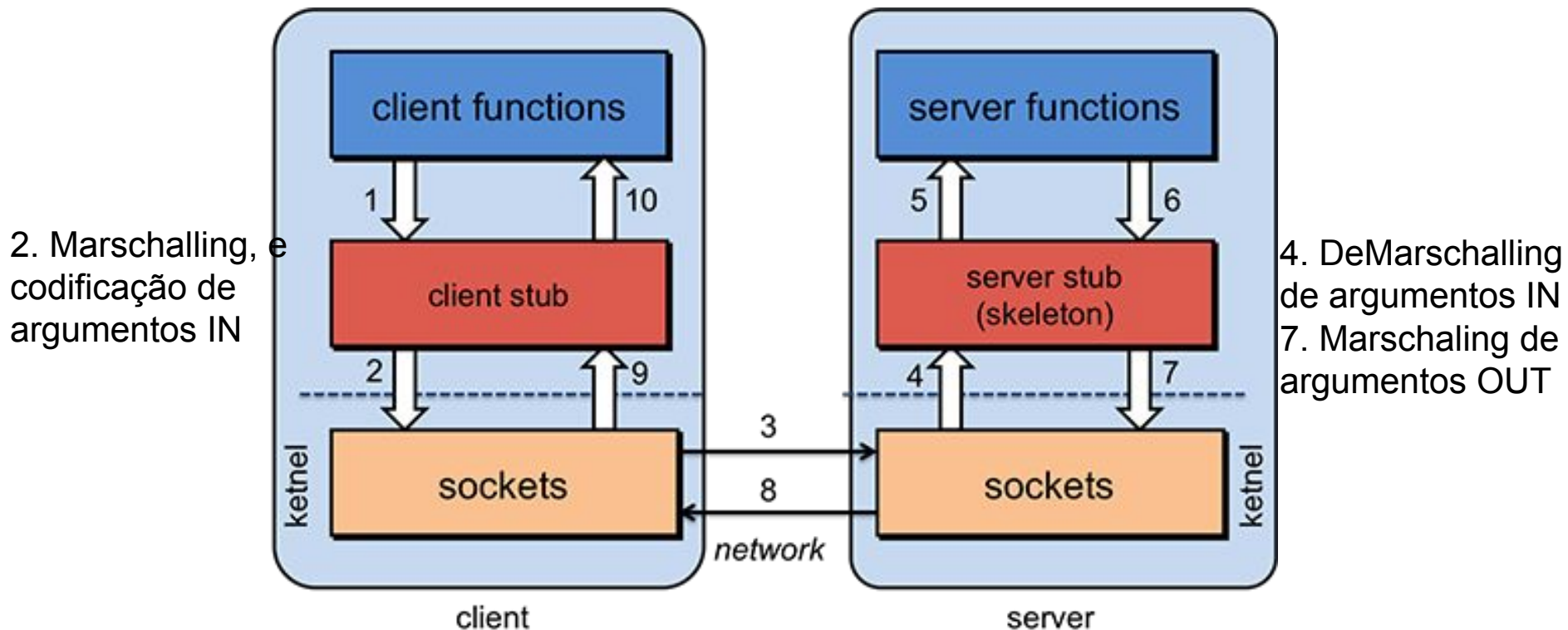
(Remote Procedure Call, Remote Method Invocation)

Remote Procedure Call (RPC) cria uma abstração de uma chamada de procedimento entre processos distribuídos executando na rede;

- **Stubs** – código “proxy” gerado (a partir da assinatura dos procedimentos do servidor) no lado cliente e no lado servidor:
 - **stub cliente** encontra o servidor, estabelece uma conexão e faz o marshalling dos parâmetros
 - **stub do servidor** (skeleton) recebe a mensagem, desempacota os parâmetros, faz a chamada do procedimento, empacota o resultado e envia de volta ao stub cliente.

Chamada Remota de Procedimentos

RPC implementado sobre sockets.



Interface Definition Language (IDL)

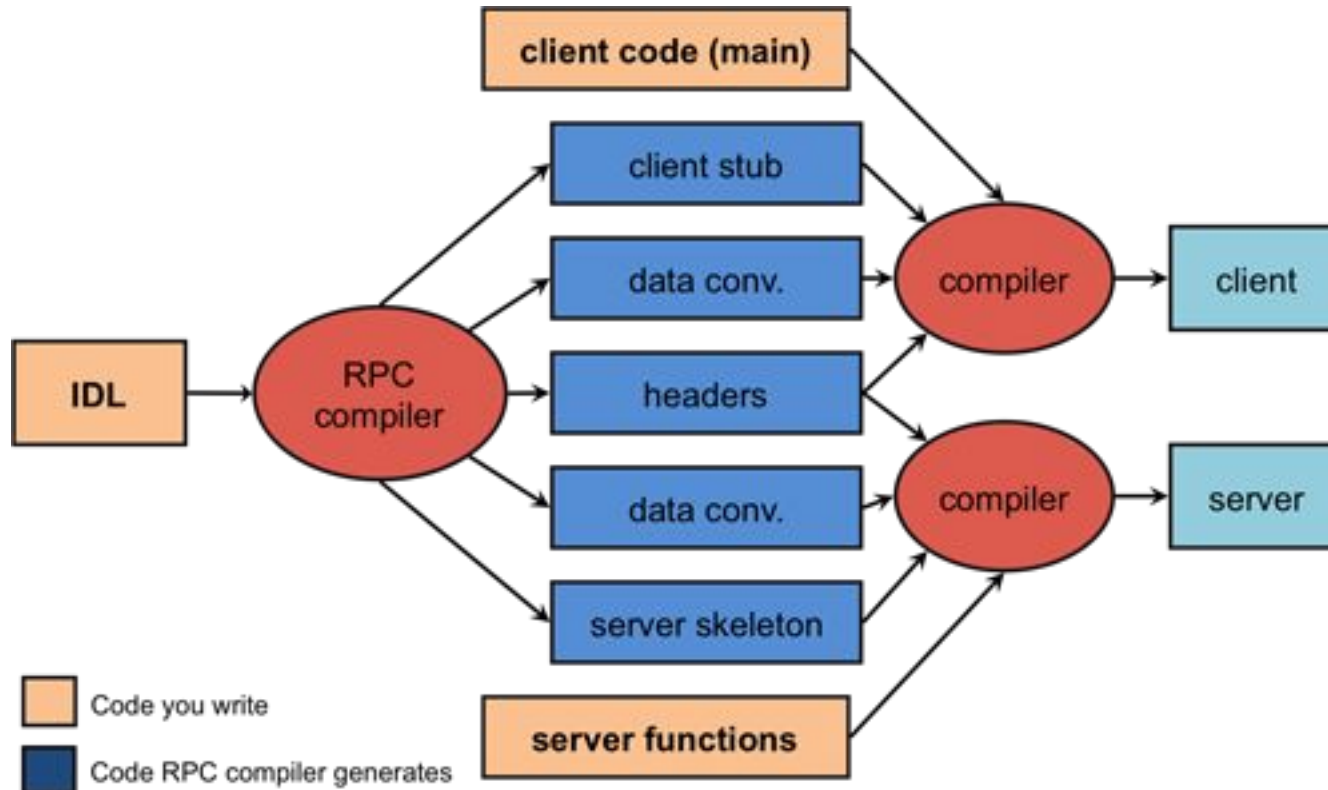
É a linguagem em que se especifica a interface de um processo (servidor). É independente de linguagem de programação.

Além dos tipos atômicos (long, double, string) e de structs para estruturas de dados mais complexas, também define os parâmetros de entrada, saída e tipos de retorno dos procedimentos remotos

```
module BankExample {  
    typedef float MoneyType;  
    struct NameType {  
        string first;  
        string last;  
    };  
    interface BankAccount {  
        MoneyType balance();  
        MoneyType deposit(  
            in MoneyType amount);  
        MoneyType withdraw(  
            in MoneyType amount);  
    };  
    interface CheckingAccount :  
                                   BankAccount {  
        exception BadCheck {  
            MoneyType fee;  
        };  
        MoneyType writeCheck(  
            in MoneyType amount)  
            raises (BadCheck);  
    };  
    interface BankManager {  
        CheckingAccount openAccount(  
            in NameType name,  
            in MoneyType deposit,  
            out MoneyType balance);  
    };  
};
```

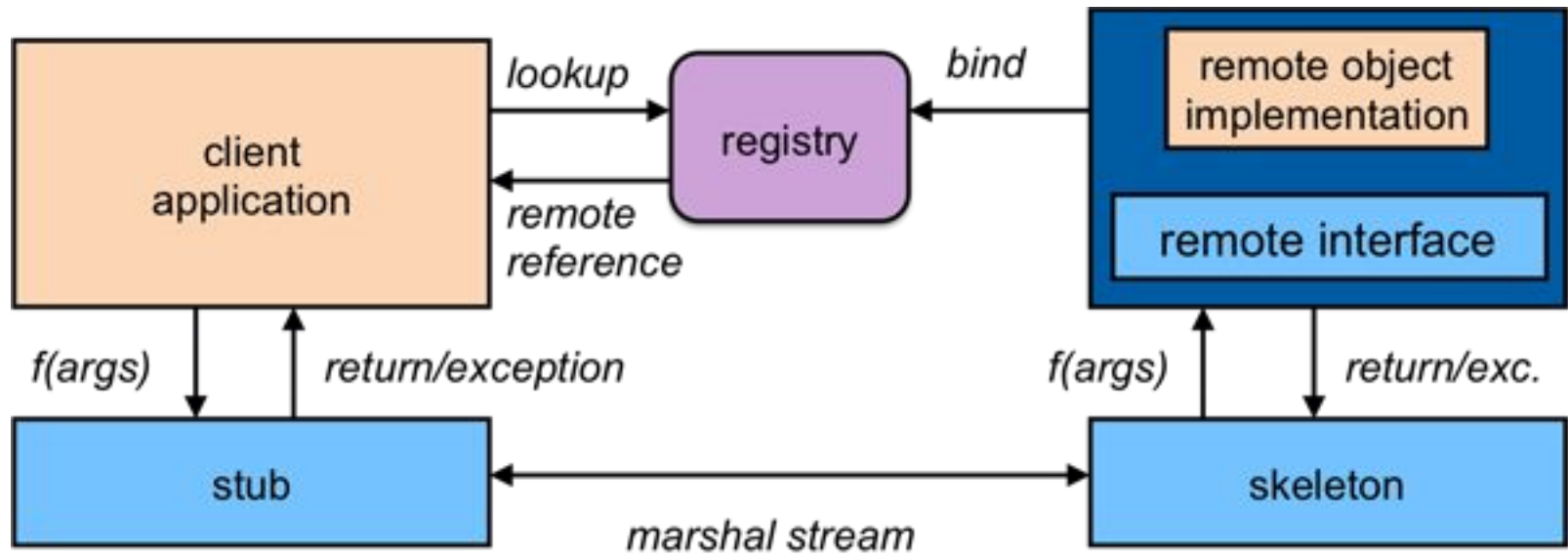
Figure 1: Bank IDL Example

RPC Compiler



Chamada Remota de Procedimentos

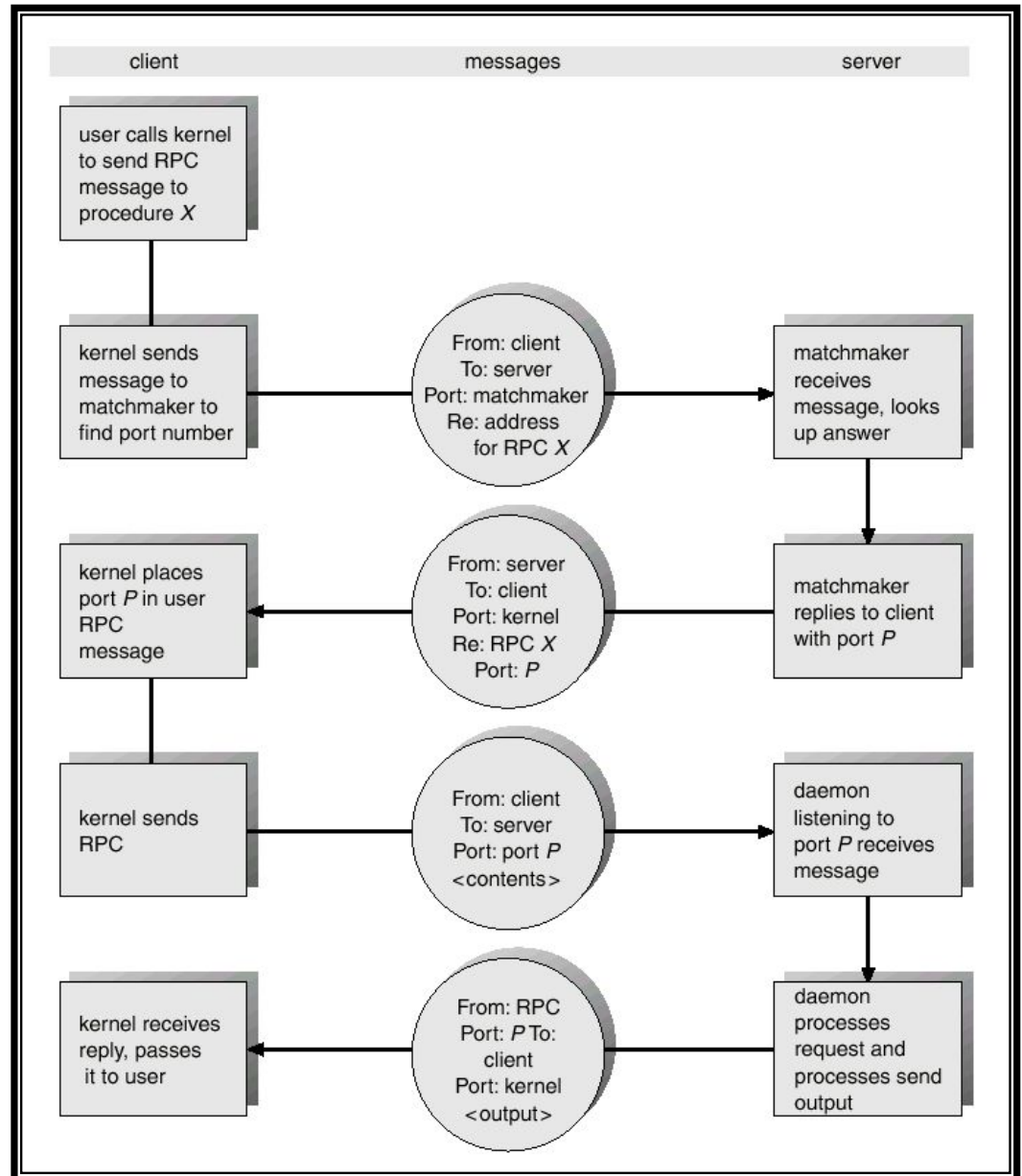
Como o cliente descobre onde está o servidor?



Geralmente envolve um broker (Matchmaker/Registry), que conhece o endereço IP+porta do processo servidor, i.e. que implementa o procedimento procurado.

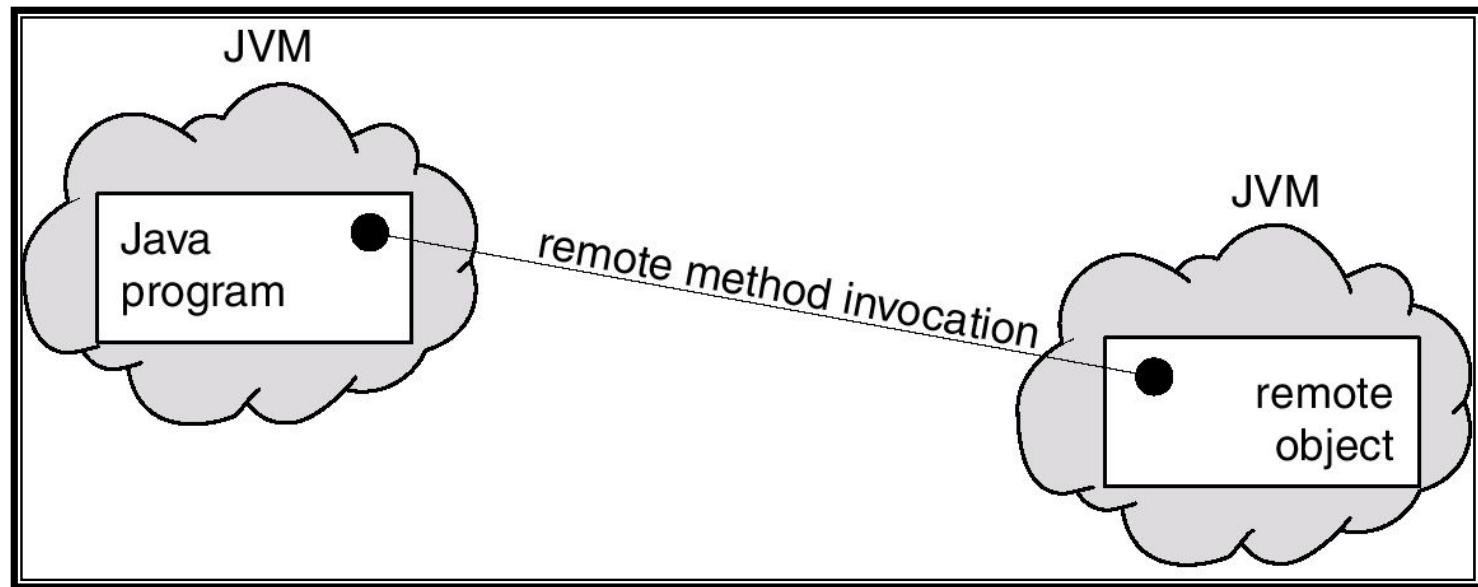
Execução de um RPC

Geralmente envolve um broker (Matchmaker/Registry), que conhece o endereço IP+porta do processo servidor, i.e. que implementa o procedimento procurado.

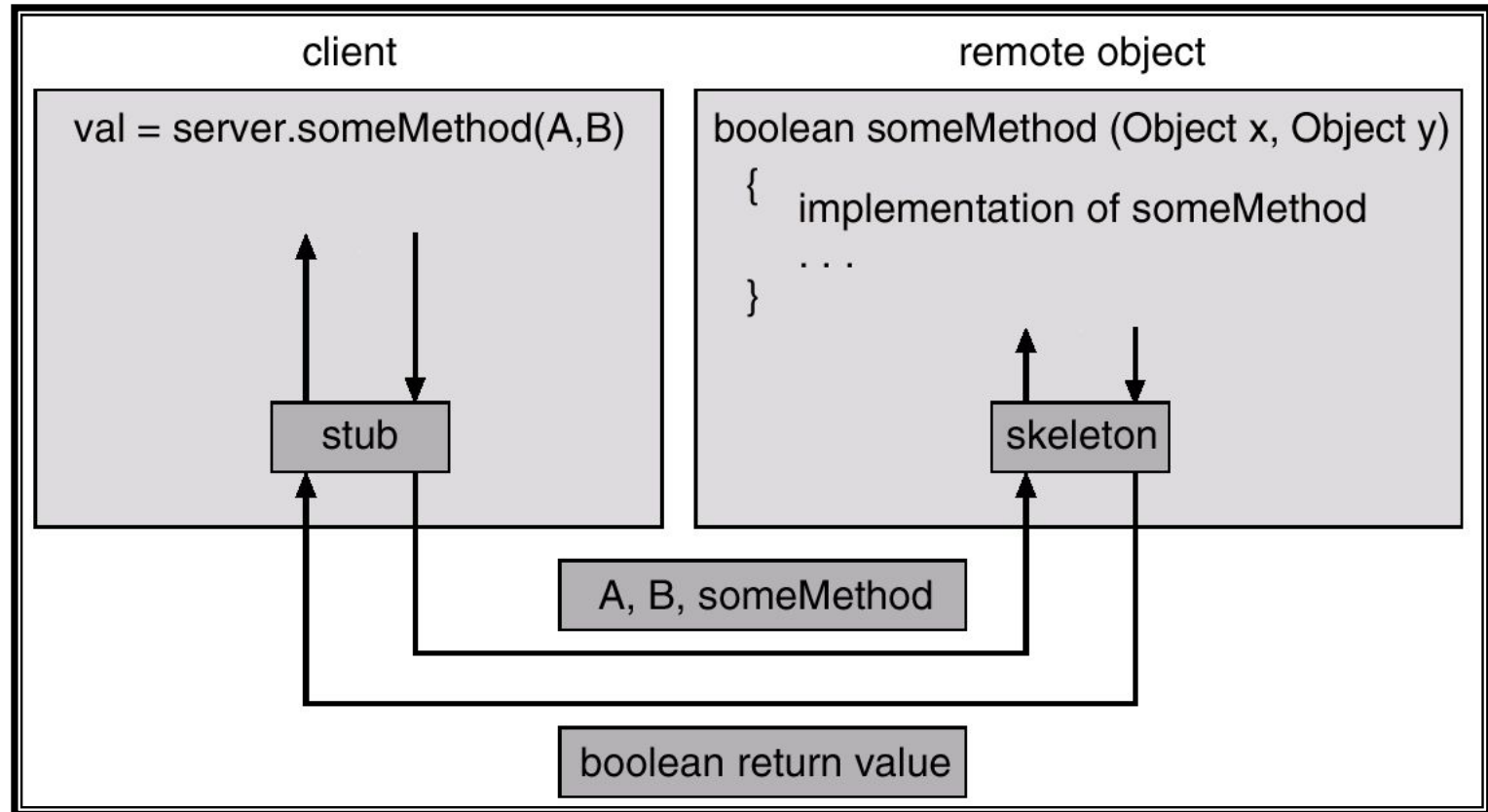


Remote Method Invocation

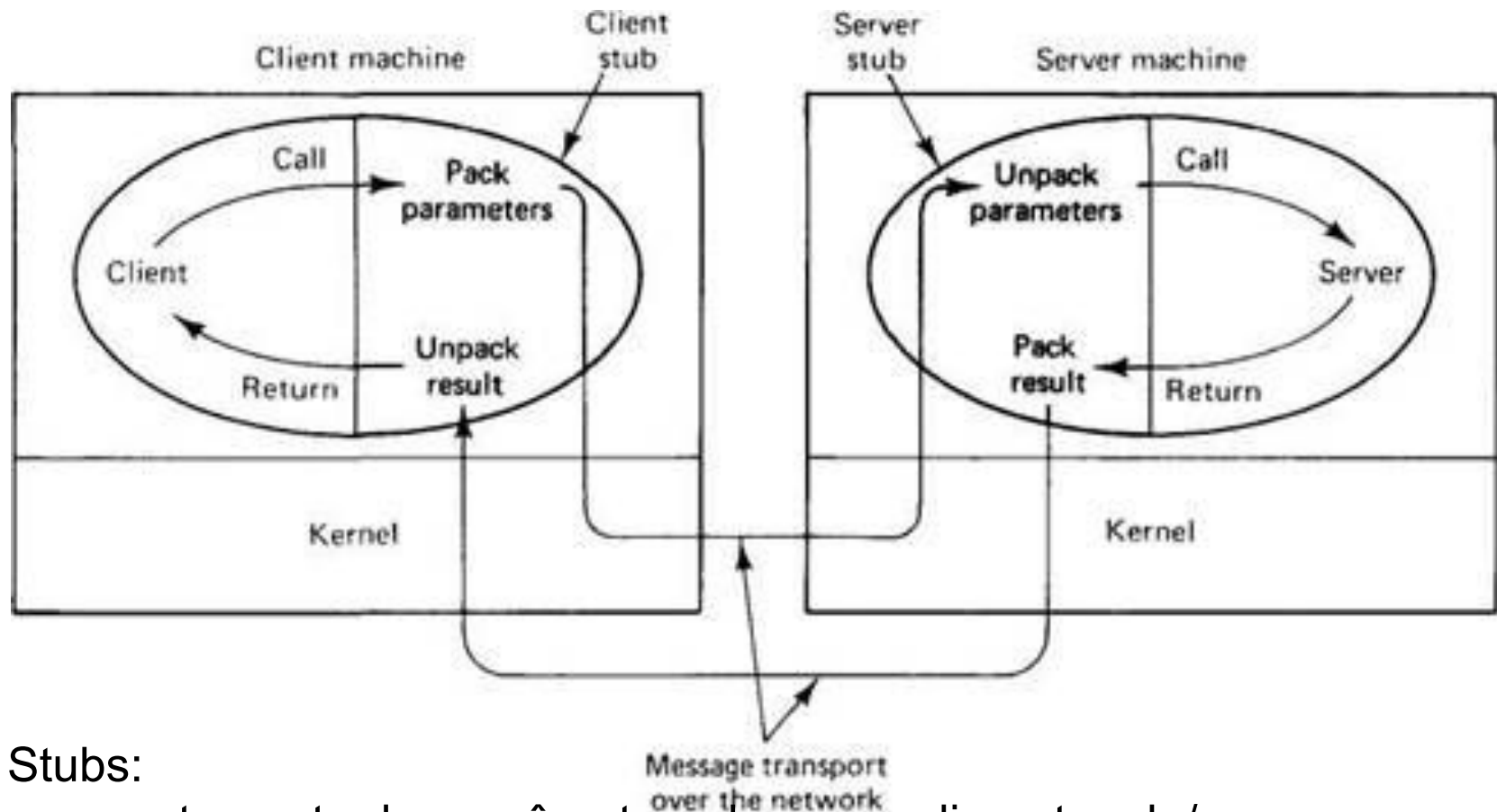
- Remote Method Invocation (RMI) is a Java mechanism similar to RPCs.
- RMI allows a Java program on one machine to invoke a method on a remote object.



RPC: Funcionamento



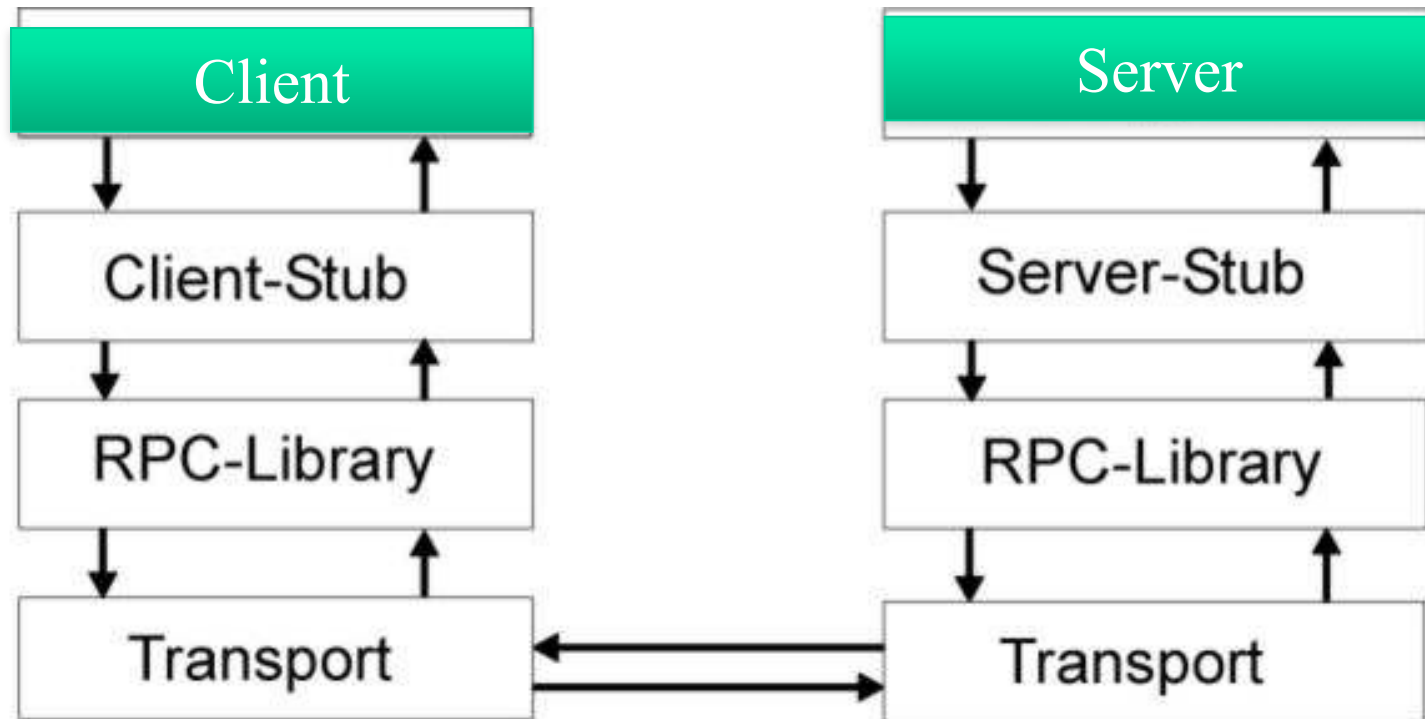
RPC: Funcionamento



Stubs:

- empacotamento dos parâmetros dos procedimentos de/para mensagens;
- conversão de dados da representação específica da lingu. de programação para uma representação intermediária, e vice-versa

RPC



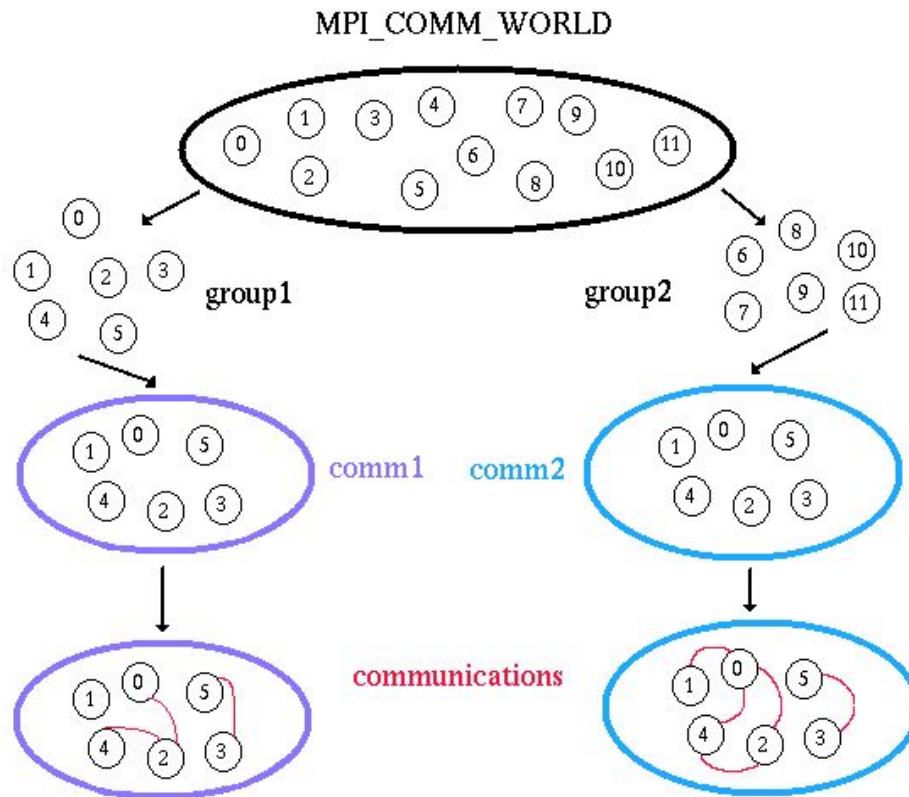
Biblioteca RPC: para registro dos stubs, multiplexação de sockets, e controle do protocolo de transporte;

MPI

MPI é uma biblioteca para comunicar e sincronizar processos apenas através de uma rede (ou barramento) geralmente para aplicações paralelas.

- MPI têm a noção de “communicator”
 - Consiste de um grupo de processos e um contexto
 - Uma mensagem deve ser enviada e recebida no mesmo contexto.
 - Communicator é uma forma de agrupar dinamicamente o conjunto de processos que irão interagir
 - Existe um communicator default que engloba inicialmente todos os processos criados.
-
- Cada mensagem é composta de (address, count, datatype)
 - Um datatype pode ser tipo básico da linguagem; um vetor de *datatypes*; um vetor indexado de blocos de *datatype*; uma estrutura arbitária de *datatypes*
 - MPI provê primitivas para construir datatypes customizados

MPI Communicator



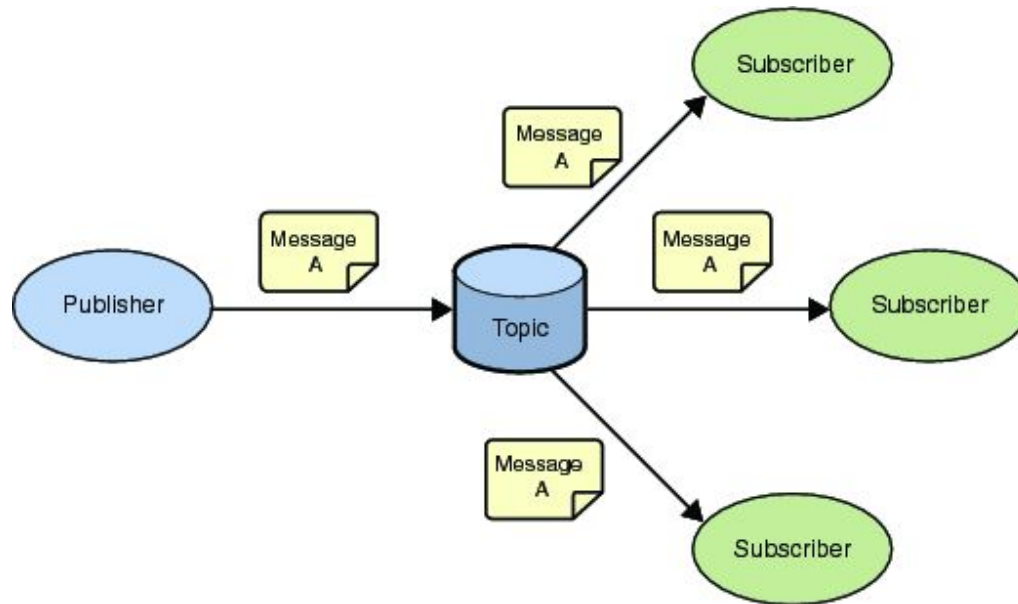
Collective Operations

These operations are all executed in a collective fashion, meaning that each process in a process group calls the communication routine

- **Barrier:** Synchronize all processes.
- **Broadcast:** Send data from one process to all processes.
- **Gather:** Gather data from all processes to one process.
- **Scatter:** Scatter data from one process to all processes.
- **Reduction operations:** addition, multiplication, etc. of distributed data.

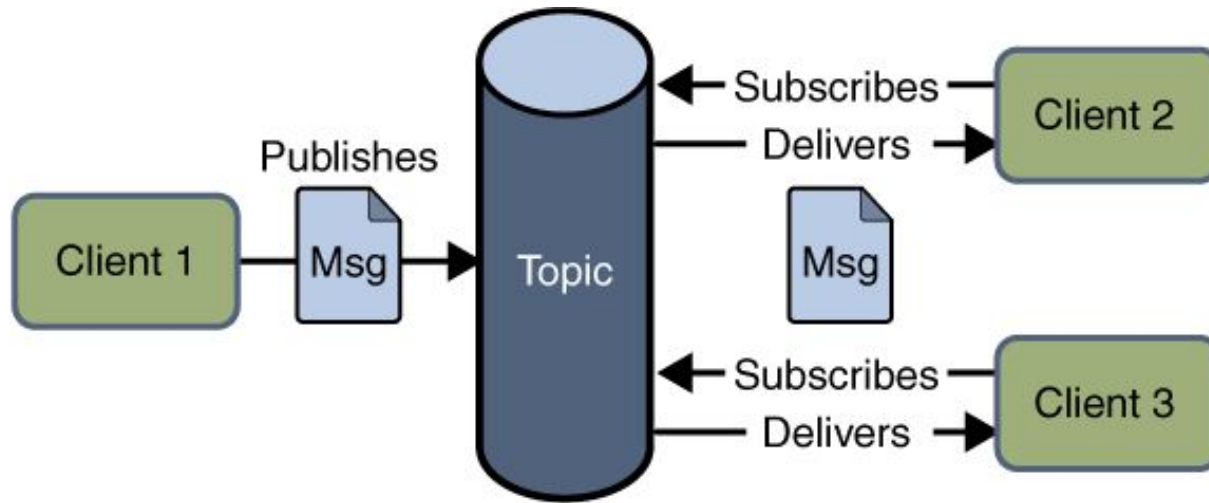
Publish/Subscribe

- Clientes se registram em um tópico para receber mensagens (subscribe), possivelmente com um *filtro*
- Publicadores postam mensagens nos tópicos, que middleware distribui para todos os subscribers
- Comunicação 1-para-1, 1-para-N, M-para-N
- Publishers e Subscribers não se conhecem:
desacoplamento referencial e espacial



- Possivelmente, também com **desacoplamento temporal**

Publish Subscribe



Exemplos: XMPP. DDS. MQTT, Amazon SNS. Azure Service Bus, etc.

Exemplo em C de Pub-Sub usando fila de mensagens

```
// --- Message Structure ---
typedef struct Message {
    char topic[50];
    char content[100];
    struct Message *next;
} Message;

// --- Broker's Message Queue ---
Message *message_queue = NULL;
pthread_mutex_t queue_mutex = PTHREAD_MUTEX_INITIALIZER;

// --- Function to add a message to the queue ---
void enqueue_message(const char *topic, const char *content) {
    Message *new_msg = (Message *)malloc(sizeof(Message));
    if (new_msg == NULL) {
        perror("Failed to allocate message");
        return;
    }
    strncpy(new_msg->topic, topic, sizeof(new_msg->topic) - 1);
    new_msg->topic[sizeof(new_msg->topic) - 1] = '\0';
    strncpy(new_msg->content, content, sizeof(new_msg->content));
    new_msg->content[sizeof(new_msg->content) - 1] = '\0';
    new_msg->next = NULL;

    pthread_mutex_lock(&queue_mutex);
    if (message_queue == NULL) {
        message_queue = new_msg;
    } else {
        Message *current = message_queue;
        while (current->next != NULL) {
            current = current->next;
        }
        current->next = new_msg;
    }
    pthread_mutex_unlock(&queue_mutex);
}
```

```
// --- Function to retrieve a message for a specific topic (subs
Message *dequeue_message(const char *subscribed_topic) {
    pthread_mutex_lock(&queue_mutex);
    Message *current = message_queue;
    Message *prev = NULL;
    Message *found_msg = NULL;

    while (current != NULL) {
        if (strcmp(current->topic, subscribed_topic) == 0) {
            found_msg = current;
            if (prev == NULL) {
                message_queue = current->next;
            } else {
                prev->next = current->next;
            }
            break;
        }
        prev = current;
        current = current->next;
    }
    pthread_mutex_unlock(&queue_mutex);
    return found_msg;
}
```

Exemplo em C de Pub-Sub usando fila de mensagens

```
// --- Publisher Thread Function ---
void *publisher(void *arg) {
    char *topic = (char *)arg;
    int i = 0;
    while (i < 5) {
        char content[100];
        sprintf(content, "Message %d from publisher on topic %s", i, topic);
        enqueue_message(topic, content);
        printf("Publisher on topic '%s' published: %s\n", topic, content);
        i++;
        sleep(1); // Simulate work
    }
    return NULL;
}

// --- Subscriber Thread Function ---
void *subscriber(void *arg) {
    char *subscribed_topic = (char *)arg;
    printf("Subscriber subscribed to topic: '%s'\n", subscribed_topic);
    while (1) {
        Message *msg = dequeue_message(subscribed_topic);
        if (msg != NULL) {
            printf("Subscriber on topic '%s' received: %s\n", subscribed_topic, msg->content);
            free(msg); // Free the message after processing
        }
        usleep(100000); // Check for messages every 100ms
    }
    return NULL;
}
```

Exemplo em C de Pub-Sub usando fila de mensagens

```
int main() {
    pthread_t pub_thread1, pub_thread2, sub_thread1, sub_thread2;

    // Create publisher threads
    pthread_create(&pub_thread1, NULL, publisher, (void *)"weather");
    pthread_create(&pub_thread2, NULL, publisher, (void *)"news");

    // Create subscriber threads
    pthread_create(&sub_thread1, NULL, subscriber, (void *)"weather");
    pthread_create(&sub_thread2, NULL, subscriber, (void *)"news");

    // Join publisher threads (subscribers run indefinitely in the background)
    pthread_join(pub_thread1, NULL);
    pthread_join(pub_thread2, NULL);

    // In a real application, you would have a mechanism to gracefully shutdown
    // For this example, we'll just let them run for a bit and then return
    sleep(10); // Let subscribers receive some messages

    // Clean up remaining messages in the queue (if any)
    pthread_mutex_lock(&queue_mutex);
    Message *current = message_queue;
    while (current != NULL) {
        Message *temp = current;
        current = current->next;
        free(temp);
    }
    message_queue = NULL;
```